

D79232GC10

Edition 1.0

February 2014

D85497

ORACLE®

Oracle Database 12c: Data Guard Administration

Student Guide – Volume II

Author

Mark Fuller

Technical Contributors and Reviewers

Christopher Andrews

Maria Billings

Trevor Bowen

Harald van Breederode

Larry Carpenter

Al Flournoy

Joe Fong

Gerlinde Frenzen

Joel Goodman

Uwe Hesse

Nitin Karkhanis

Donna Keesling

Sean Kim

Jerry Lee,

Stephen Moriarty

Juan Quezada Nunez

Veerabhadra Rao Putrevu

Puneet Sangar

Thorsten Senft

Ira Singer

Branislav Valny

Editors

Daniel Milne

Raj Kumar

Anwesha Ray

Publishers

Joseph Fernandez

Veena Narasimhan

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Disclaimer

This document contains proprietary information and is protected by copyright and other intellectual property laws. You may copy and print this document solely for your own use in an Oracle training course. The document may not be modified or altered in any way. Except where your use constitutes "fair use" under copyright law, you may not use, share, download, upload, copy, print, display, perform, reproduce, publish, license, post, transmit, or distribute this document in whole or in part without the express authorization of Oracle.

The information contained in this document is subject to change without notice. If you find any problems in the document, please report them in writing to: Oracle University, 500 Oracle Parkway, Redwood Shores, California 94065 USA. This document is not warranted to be error-free.

Restricted Rights Notice

If this documentation is delivered to the United States Government or anyone using the documentation on behalf of the United States Government, the following notice is applicable:

U.S. GOVERNMENT RIGHTS

The U.S. Government's rights to use, modify, reproduce, release, perform, display, or disclose these training materials are restricted by the terms of the applicable Oracle license agreement and/or the applicable U.S. Government contract.

Trademark Notice

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Contents

1 Introduction to Oracle Data Guard

- Objectives 1-2
- What Is Oracle Data Guard? 1-3
- Types of Standby Databases 1-4
- Types of Data Guard Services 1-6
- Role Transitions: Switchover and Failover 1-7
- Oracle Data Guard Broker Framework 1-9
- Choosing an Interface for Administering a Data Guard Configuration 1-10
- Oracle Data Guard: Architecture (Overview) 1-11
- Primary Database Processes 1-12
- Standby Database Processes 1-13
- Physical Standby Database: Redo Apply Architecture 1-14
- Logical Standby Database: SQL Apply Architecture 1-15
- Automatic Gap Detection and Resolution 1-16
- Data Protection Modes 1-17
- Data Guard Operational Requirements: Hardware and Operating System 1-19
- Data Guard Operational Requirements: Oracle Database Software 1-20
- Benefits of Implementing Oracle Data Guard 1-21
- Quiz 1-22
- Summary 1-24
- Practice 1: Overview 1-25

2 Creating a Physical Standby Database by Using Enterprise Manager Cloud Control

- Objectives 2-2
- Prerequisites for Using Oracle Enterprise Manager Cloud Control 12c 2-3
- Accessing the Add Standby Database Wizard 2-4
- Creating a New Standby Database 2-6
- Creating a Physical Standby Database Step 1: Backup Type 2-7
- Creating a Physical Standby Database Step 2: Backup Options 2-8
- Creating a Physical Standby Database Step 3: Database Location 2-9
- Creating a Physical Standby Database Step 4: File Locations 2-10
- Creating a Physical Standby Database Step 5: Configuration 2-12
- Creating a Physical Standby Database Step 6: Review 2-13
- Creating a Physical Standby Database Job Activity 2-14

Creating a Physical Standby Database Job Task Details	2-15
Data Guard Administration Page	2-16
Verifying Configuration	2-17
Performing Routine Maintenance	2-18
Configuring Fast-Start Failover	2-19
Editing Primary Database Properties: General Properties	2-20
Editing Primary Database Properties: Standby Role Properties	2-21
Editing Primary Database Properties: Common Properties	2-22
Test Application	2-23
Quiz	2-24
Summary	2-26
Practice 2: Overview	2-27

3 Oracle Net Services in a Data Guard Environment

Objectives	3-2
Oracle Net Services Overview	3-3
Understanding Name Resolution	3-4
Local Naming Configuration Files	3-5
Local Naming: tnsnames.ora	3-6
Connect-Time Failover: Planning for Role Reversal	3-7
Listener Configuration: listener.ora	3-8
Dynamic Service Registration	3-9
Static Listener Entries: listener.ora	3-10
Optimizing Oracle Net for Data Guard	3-11
Quiz	3-13
Summary	3-15
Practice 3: Overview	3-16

4 Creating a Physical Standby Database by Using SQL and RMAN Commands

Objectives	4-2
Steps to Create a Physical Standby Database	4-3
Preparing the Primary Database	4-4
FORCE LOGGING Mode	4-5
Configuring Standby Redo Logs	4-7
Standby Redo Log Usage	4-8
Using SQL to Create Standby Redo Logs	4-9
Viewing Standby Redo Log Information	4-10
Setting Initialization Parameters on the Primary Database to Control Redo Transport	4-11
Setting LOG_ARCHIVE_CONFIG	4-12
Setting LOG_ARCHIVE_DEST_n	4-14

Specifying Role-Based Destinations	4-15
Combinations for VALID_FOR	4-17
Defining the Redo Transport Mode	4-18
Setting Initialization Parameters on the Primary Database	4-19
Specifying Values for DB_FILE_NAME_CONVERT	4-20
Specifying Values for LOG_FILE_NAME_CONVERT	4-21
Specifying a Value for STANDBY_FILE_MANAGEMENT	4-22
Specifying a Value for FAL_SERVER	4-23
Example: Setting Initialization Parameters on the Primary Database	4-24
Creating an Oracle Net Service Name for Your Physical Standby Database	4-25
Creating a Listener Entry for Your Standby Database	4-26
Copying Your Primary Database Password File to the Physical Standby Database Host	4-27
Creating an Initialization Parameter File for the Physical Standby Database	4-28
Creating Directories for the Physical Standby Database	4-29
Starting the Physical Standby Database	4-30
Creating an RMAN Script to Create the Physical Standby Database	4-31
Creating the Physical Standby Database	4-33
Real-Time Apply (Default)	4-34
Starting Redo Apply in Real-Time	4-36
Preventing Primary Database Data Corruption from Affecting the Standby Database	4-37
Special Note: Data Guard Support for Oracle Multitenant	4-39
Special Note: Standby Database on the Same System	4-40
Creating a Physical Standby Without Using RMAN	4-41
Quiz	4-42
Summary	4-44
Practice 4: Overview	4-45

5 Using Oracle Active Data Guard

Objectives	5-2
Oracle Active Data Guard	5-3
Using Real-Time Query	5-4
Enabling Real-Time Query	5-5
Disabling Real-Time Query	5-6
Checking the Standby's Open Mode	5-7
Understanding Lag in an Active Data Guard Configuration	5-8
Monitoring Apply Lag: V\$DATAGUARD_STATS	5-9
Monitoring Apply Lag: V\$STANDBY_EVENT_HISTOGRAM	5-10
Allowed Staleness of Standby Query Data	5-11
Configuring Zero Lag Between the Primary and Standby Databases	5-12

Setting STANDBY_MAX_DATA_DELAY by Using an AFTER LOGON Trigger	5-13
Example: Setting STANDBY_MAX_DATA_DELAY by Using an AFTER LOGON Trigger	5-14
Forcing Redo Apply Synchronization	5-15
Creating an AFTER LOGON Trigger for Synchronization	5-16
Active Data Guard: DML on Temporary Tables	5-17
Active Data Guard: Support for Global Sequences	5-18
Active Data Guard: Support for Session Sequences	5-19
Benefits: Temporary Undo and Sequences	5-20
Supporting Read-Mostly Applications	5-21
Example: Transparently Redirecting Writes to the Primary Database	5-22
Enabling Block Change Tracking on a Physical Standby Database	5-23
Creating Fast Incremental Backups	5-24
Enabling Block Change Tracking	5-25
Monitoring Block Change Tracking	5-26
Data Guard 12c: Far Sync	5-27
Far Sync: Redo Transport	5-28
Far Sync: Alternate Redo Transport Routes	5-30
Oracle Data Guard 12c: Far Sync Creation	5-31
Benefits: Far Sync	5-32
Far Sync: Alternate Design	5-33
Real-Time Cascade	5-34
Traditional Multi-Standby Database Architecture	5-35
Benefits: Real-Time Cascade	5-36
Quiz	5-37
Summary	5-38
Practice 5: Overview	5-39

6 Creating and Managing a Snapshot Standby Database

Objectives	6-2
Snapshot Standby Databases: Overview	6-3
Snapshot Standby Database: Architecture	6-4
Converting a Physical Standby Database to a Snapshot Standby Database	6-5
Activating a Snapshot Standby Database: Issues and Cautions	6-6
Snapshot Standby Database: Target Restrictions	6-7
Viewing Snapshot Standby Database Information	6-8
Snapshot Standby Space Requirements	6-9
Converting a Snapshot Standby Database to a Physical Standby Database	6-10
Quiz	6-11
Summary	6-12
Practice 6: Overview	6-13

7 Creating a Logical Standby Database

- Objectives 7-2
- Benefits of Implementing a Logical Standby Database 7-3
- Logical Standby Database: SQL Apply Architecture 7-5
- SQL Apply Process: Architecture 7-6
- Preparing to Create a Logical Standby Database 7-7
- Unsupported Objects 7-8
- Unsupported Data Types 7-9
- Identifying Internal Schemas 7-10
- Checking for Unsupported Tables 7-11
- Checking for Tables with Unsupported Data Types 7-12
- SQL Commands That Do Not Execute on the Standby Database 7-13
- Unsupported PL/SQL-Supplied Packages 7-14
- Ensuring Unique Row Identifiers 7-15
- Adding a Disabled Primary Key RELY Constraint 7-17
- Creating a Logical Standby Database by Using SQL Commands 7-18
- Step 1: Create a Physical Standby Database 7-19
- Step 2: Stop Redo Apply on the Physical Standby Database 7-20
- Step 3: Prepare the Primary Database to Support Role Transitions 7-21
- Step 4: Build a LogMiner Dictionary in the Redo Data 7-22
- Step 5: Transition to a Logical Standby Database 7-23
- Step 6: Open the Logical Standby Database 7-24
- Step 7: Verify That the Logical Standby Database Is Performing Properly 7-25
- Creating a Logical Standby Database by Using Enterprise Manager 7-27
- Using the Add Standby Database Wizard 7-28
- Securing Your Logical Standby Database 7-31
- Automatic Deletion of Redo Log Files by SQL Apply 7-32
- Managing Remote Archived Log File Retention 7-33
- Creating SQL Apply Filtering Rules 7-34
- Deleting SQL Apply Filtering Rules 7-35
- Viewing SQL Apply Filtering Settings 7-36
- Using DBMS_SCHEDULER to Create Jobs on a Logical Standby Database 7-37
- Quiz 7-38
- Summary 7-40
- Practice 7: Overview 7-41

8 Oracle Data Guard Broker: Overview

- Objectives 8-2
- Oracle Data Guard Broker: Features 8-3
- Data Guard Broker: Components 8-4
- Data Guard Broker: Configurations 8-5

Data Guard Broker: Management Model	8-6
Data Guard Broker: Architecture	8-7
Data Guard Monitor: DMON Process	8-8
Benefits of Using the Data Guard Broker	8-9
Comparing Configuration Management With and Without the Data Guard Broker	8-10
Data Guard Broker Interfaces	8-11
Broker Controlled Database Initialization Parameters	8-12
Using the Command-Line Interface of the Data Guard Broker	8-13
Using Oracle Enterprise Manager Cloud Control	8-16
Data Guard Overview Page	8-17
Benefits of Using Enterprise Manager	8-18
Quiz	8-19
Summary	8-21

9 Creating a Data Guard Broker Configuration

Objectives	9-2
Data Guard Broker: Requirements	9-3
Data Guard Broker and the SPFILE	9-5
Data Guard Monitor: Configuration File	9-7
Data Guard Broker: Log Files	9-8
Creating a Broker Configuration	9-9
Clear Redo Transport Network Locations on Primary	9-10
Connecting to the Primary Database with DGMGRL	9-11
Defining the Broker Configuration and the Primary Database Profile	9-12
Adding a Standby Database to the Configuration	9-13
Adding a Far Sync to the Configuration	9-14
Enabling the Configuration	9-15
Broker Support for Complex Redo Routing	9-16
Defining RedoRoutes by Using DGMGRL	9-17
RedoRoutes Usage Guidelines	9-18
How to Read Redo Routing Rules	9-19
Far Sync Example with RedoRoutes	9-20
Changing Database Properties and States	9-21
Managing Redo Transport Services by Using DGMGRL	9-22
Managing the Redo Transport Service by Using the LogXptMode Property	9-26
Setting LogXptMode to ASYNC	9-27
Setting LogXptMode to SYNC	9-28
Setting LogXptMode to FASTSYNC (New in Oracle Database 12c)	9-29
Disabling Broker Management of the Configuration or Standby Database	9-30
Removing the Configuration or Standby Database	9-31

Quiz 9-32
 Summary 9-33
 Practice 9: Overview 9-34

10 Monitoring a Data Guard Broker Configuration

Objectives 10-2
 Monitoring the Data Guard Configuration by Using Enterprise Manager Cloud
 Control 10-3
 Viewing the Data Guard Configuration Status 10-4
 Monitoring Data Guard Performance 10-5
 Viewing Log File Details 10-6
 Enterprise Manager Metrics and Alerts 10-7
 Data Guard Metrics 10-8
 Managing Data Guard Metrics 10-9
 Viewing Metric Value History 10-10
 Viewing Data Guard Diagnostic Information 10-11
 Using Monitorable Database Properties to Identify a Failure 10-13
 Using the SHOW CONFIGURATION DGMGRL Command to Monitor the
 Configuration 10-14
 Using the SHOW DATABASE VERBOSE DGMGRL Command to Monitor the
 Configuration 10-15
 Viewing Standby Redo Log Information in V\$LOGFILE 10-17
 Viewing Standby Redo Log Information in V\$STANDBY_LOG 10-18
 Identifying Destination Settings 10-19
 Setting the LOG_ARCHIVE_TRACE Initialization Parameter 10-20
 Viewing Redo Transport Errors by Querying V\$ARCHIVED_DEST 10-22
 Evaluating Redo Data by Querying V\$DATAGUARD_STATS 10-23
 Viewing Data Guard Status Information by Querying
 V\$DATAGUARD_STATUS 10-24
 Monitoring Redo Apply by Querying V\$MANAGED_STANDBY 10-25
 Monitoring SQL Apply by Querying V\$LOGSTDBY_TRANSACTION 10-26
 Quiz 10-27
 Summary 10-29
 Practice 10: Overview 10-30

11 Configuring Data Protection Modes

Objectives 11-2
 Data Protection Modes and Redo Transport Modes 11-3
 Data Protection Modes 11-4
 Maximum Protection Mode 11-5
 Maximum Availability Mode 11-6

Maximum Performance Mode 11-7
 Comparing Data Protection Modes 11-8
 Setting the Data Protection Mode by Using Enterprise Manager 11-9
 Setting the Data Protection Mode by Using DGMGRL 11-11
 Setting the Data Protection Mode by Using SQL 11-12
 Quiz 11-13
 Summary 11-14
 Practice 11: Overview 11-15

12 Performing Role Transitions

Objectives 12-2
 Role Management Services 12-3
 Role Transitions: Switchover and Failover 12-4
 Switchover 12-5
 Switchover: Before 12-6
 Switchover: After 12-7
 Performing a Switchover by Using Enterprise Manager 12-8
 Validating Databases for Switchover by Using DGMGRL 12-11
 Performing a Switchover by Using DGMGRL 12-14
 Preparing for a Switchover Using SQL 12-15
 Performing a Switchover by Using SQL 12-16
 Considerations When Performing a Switchover to a Logical Standby Database 12-17
 Situations That Prevent a Switchover 12-18
 Failover 12-19
 Types of Failovers 12-20
 Failover Considerations 12-21
 Performing a Failover by Using Enterprise Manager 12-22
 Performing a Failover to a Physical Standby Database 12-25
 Performing a Failover to a Logical Standby Database 12-26
 Performing a Manual Failover by Using DGMGRL 12-27
 Reenabling Disabled Databases by Using DGMGRL 12-28
 Quiz 12-29
 Summary 12-31
 Practice 12: Overview 12-32

13 Using Flashback Database in a Data Guard Configuration

Objectives 13-2
 Using Flashback Database in a Data Guard Configuration 13-3
 Overview of Flashback Database 13-4
 Configuring Flashback Database 13-5

Configuring Flashback Database by Using Enterprise Manager	13-6
Using Flashback Database Instead of Apply Delay	13-8
Using Flashback Database and Real-Time Apply	13-9
Using Flashback Database After RESETLOGS	13-10
Flashback Through Standby Database Role Transitions	13-12
Using Flashback Database After Failover	13-13
Quiz	13-14
Summary	13-15
Practice 13: Overview	13-16

14 Enabling Fast-Start Failover

Objectives	14-2
Fast-Start Failover: Overview	14-3
When Does Fast-Start Failover Occur?	14-4
Installing the Observer Software	14-5
Fast-Start Failover Prerequisites	14-6
Configuring Fast-Start Failover	14-7
Step 1: Specify the Target Standby Database	14-8
Step 2: Set the Protection Mode	14-9
Step 3: Set the Fast-Start Failover Threshold	14-10
Step 4: (Optional) Set Additional Fast-Start Failover Properties	14-11
Setting the Lag-Time Limit	14-12
Configuring the Primary Database to Shut Down Automatically	14-14
Automatic Reinstatement After Fast-Start Failover	14-15
Configuring Automatic Reinstatement of the Primary Database	14-17
Setting a Connect Identifier for the Observer	14-18
Setting an Observer Override	14-19
Setting Observer Reconnection Frequency	14-20
Step 5: Configure Additional Fast-Start Failover Conditions	14-21
Configuring Fast-Start Failover Conditions	14-23
Step 6: Enable Fast-Start Failover	14-24
Step 7: Start the Observer	14-25
Step 8: Verify the Configuration	14-27
Initiating Fast-Start Failover from an Application	14-28
Viewing Fast-Start Failover Information	14-30
Determining the Reason for a Fast-Start Failover	14-32
Prohibited Operations After Enabling Fast-Start Failover	14-33
Disabling Fast-Start Failover	14-34
Disabling Fast-Start Failover Conditions	14-36
Using the FORCE Option	14-37
Stopping the Observer	14-38

- Performing Manual Role Changes 14-39
- Manually Reinstating the Database 14-40
- Using Enterprise Manager to Enable Fast-Start Failover 14-41
- Changing the Protection Mode and Disabling Fast-Start Failover 14-47
- Using Enterprise Manager to Disable Fast-Start Failover 14-48
- Using Enterprise Manager to Suspend Fast-Start Failover 14-49
- Moving the Observer to a New Host 14-50
- Quiz 14-51
- Summary 14-53
- Practice 14: Overview 14-54

15 Backup and Recovery Considerations in an Oracle Data Guard Configuration

- Objectives 15-2
- Using RMAN to Back Up and Restore Files in a Data Guard Configuration 15-3
- Offloading Backups to a Physical Standby 15-4
- Restrictions and Usage Notes 15-5
- Association and Accessibility of RMAN Backups 15-6
- Backup and Recovery of a Logical Standby Database 15-7
- Using the RMAN Recovery Catalog in a Data Guard Configuration 15-8
- Creating the Recovery Catalog 15-9
- Registering a Database in the Recovery Catalog 15-11
- Setting Persistent Configuration Settings 15-12
- Setting RMAN Persistent Configuration Parameters on the Primary Database 15-14
- Setting RMAN Persistent Configuration Parameters on the Physical Standby Database 15-16
- Setting RMAN Persistent Configuration Parameters on the Other Standby Databases 15-17
- Configuring Daily Incremental Backups 15-18
- Recovering from the Loss of a Data File on the Primary Database 15-20
- Using a Backup to Recover a Data File on the Primary Database 15-21
- Using a Physical Standby Database Data File to Recover a Data File on the Primary Database 15-22
- Recovering a Data File on the Standby Database 15-24
- Enhancements to Block Media Recovery 15-25
- Executing the RECOVER BLOCK Command 15-27
- Excluding the Standby Database 15-28
- Quiz 15-29
- Summary 15-31
- Practice 15: Overview 15-32

16 Enhanced Client Connectivity in a Data Guard Environment

- Objectives 16-2
- Connecting to the Appropriate Environment 16-3
- Understanding Client Connectivity in a Data Guard Configuration 16-4
- Preventing Clients from Connecting to the Wrong Database 16-5
- Managing Services 16-6
- Creating Services for the Data Guard Configuration Databases 16-7
- Connecting Clients to the Correct Database 16-8
- Creating the AFTER STARTUP Trigger for Role-Based Services 16-9
- Configuring Service Names in the tnsnames.ora File 16-10
- Configuring Role-Based Services Using Oracle Clusterware 16-11
- Adding Standby Databases to Oracle Restart Configuration 16-13
- Example: Configuring Role-Based Services 16-14
- Automatic Failover of Applications to a New Primary Database 16-15
- Data Guard Broker and Fast Application Notification (FAN) 16-16
- Automating Client Failover in a Data Guard Configuration 16-17
- Client Failover: Components 16-18
- Client Failover: Best Practices 16-19
- Automating Failover for OCI Clients 16-20
- Automating Failover for OLE DB Clients 16-22
- Configuring OLE DB Clients for Failover 16-23
- Automating Failover for JDBC Clients 16-24
- Configuring JDBC Clients for Failover 16-25
- Quiz 16-26
- Summary 16-28
- Practice 16: Overview 16-29

17 Patching and Upgrading Databases in a Data Guard Configuration

- Objectives 17-2
- Data Guard Standby-First Patch Apply 17-3
- Upgrading an Oracle Data Guard Broker Configuration 17-6
- Upgrading Oracle Database in a Data Guard Configuration with a Physical Standby Database 17-7
- Upgrading Oracle Database in a Data Guard Configuration with a Logical Standby Database 17-9
- Using SQL Apply to Upgrade the Oracle Database 17-10
- Requirements for Using SQL Apply to Perform a Rolling Upgrade 17-11
- Performing a Rolling Upgrade by Using SQL Apply 17-12
- Identifying Unsupported Data Types 17-13
- Logical Standby: New Data Type Support for Oracle Database 12c 17-14

Performing a Rolling Upgrade by Using an Existing Logical Standby Database	17-15
Performing a Rolling Upgrade by Creating a New Logical Standby Database	17-21
Performing a Rolling Upgrade by Using a Physical Standby Database	17-22
Rolling Upgrades Using DBMS_ROLLING and Active Data Guard	17-28
DBMS_ROLLING: Concepts	17-29
DBMS_ROLLING: Key Features	17-31
Database Rolling Upgrade: Specification and Compilation Stages	17-32
Specification Stage Examples	17-33
Compilation Stage Examples	17-34
Database Rolling Upgrade: Execution Stage	17-35
Quiz	17-36
Summary	17-38

18 Optimizing a Data Guard Configuration

Objectives	18-2
Monitoring Configuration Performance by Using Enterprise Manager Cloud Control	18-3
Optimizing Redo Transport Services	18-4
Setting the ReopenSecs Database Property	18-5
Setting the NetTimeout Database Property	18-6
Optimizing Redo Transmission by Setting MaxConnections	18-7
Setting the MaxConnections Database Property	18-8
Compressing Redo Data by Setting the RedoCompression Property	18-9
Delaying the Application of Redo	18-10
Setting the DelayMins Database Property to Delay the Application of Redo	18-11
Using Enterprise Manager to Delay the Application of Redo	18-12
Optimizing SQL Apply	18-13
Adjusting the Number of APPLIER Processes	18-14
Adjusting the Number of PREPARER Processes	18-15
Quiz	18-17
Summary	18-19
Practice 18: Overview	18-20

Monitoring a Data Guard Broker Configuration

10

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Objectives

After completing this lesson, you should be able to:

- Use Enterprise Manager Cloud Control to monitor the configuration
- Use DGMGRL to monitor the configuration
- Use SQL to monitor the configuration

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Monitoring the Data Guard Configuration by Using Enterprise Manager Cloud Control

On the Data Guard Overview page, you can:

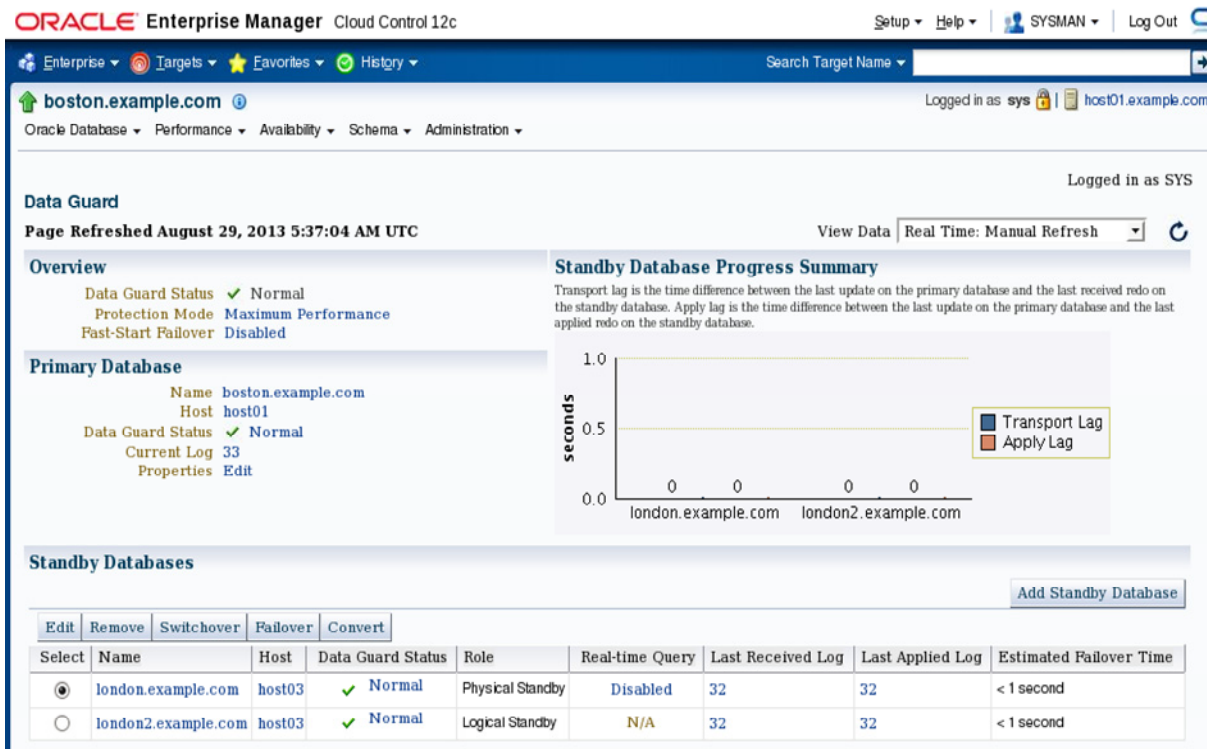
- View the Standby Progress Summary graph that shows the transport lag and the apply lag
- Access additional performance and configuration information
 - Performance Overview page: Information about data archived and applied, standby database progress, and log services
 - Log File Details page: Information about the primary database online redo log file
- Perform a Verify operation

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Enterprise Manager Cloud Control provides a graphical user interface for monitoring the Data Guard configuration. The following pages in this lesson describe the information that you can view on the Data Guard Overview page and its related pages.

Viewing the Data Guard Configuration Status



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

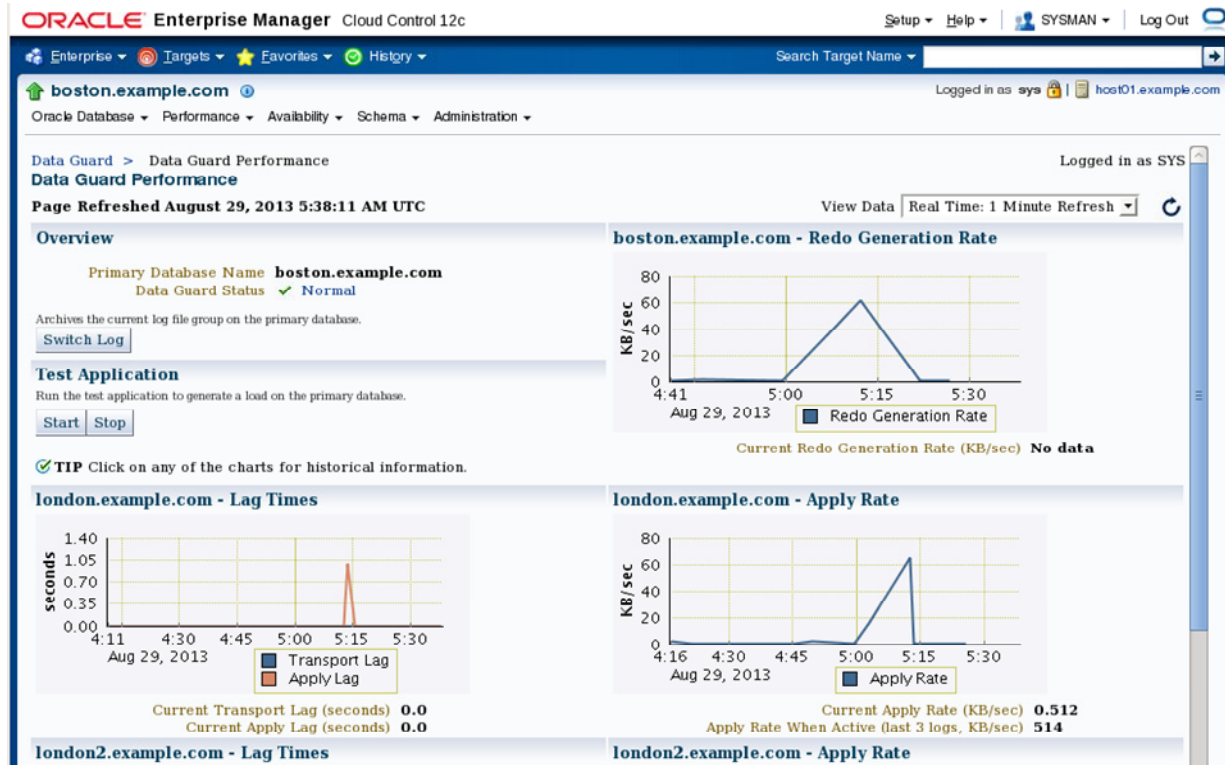
On the Data Guard Overview page, you can view the status of the primary database and the standby databases in a configuration. In the Standby Database Progress Summary section, you can view information about transport and apply lags.

The apply lag shows the approximate number of seconds that the standby database is behind the primary database. A large apply lag may indicate that Redo Apply needs to be tuned or that there is a gap.

The transport lag shows the approximate number of seconds of redo not yet available on the standby database and provides an indication of potential data loss in the event of a disaster. A significant transport lag may be caused by:

- A gap in the redo
- Network bandwidth constraints
- An overloaded log writer network server (LNS) process on the primary database
- Redo generation that is faster than the LNS and network can handle
- I/O issues on the remote file server (RFS) process side
- An overloaded RFS process
- An inadequate number of standby redo log files causing the RFS process to stall or to use archived log files

Monitoring Data Guard Performance



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Click Data Guard Performance in the Performance section of the Data Guard Overview page to access the Performance Overview page.

The Data Guard Performance page displays detailed performance-related statistics for the Data Guard configuration. The performance charts provide a graphical summary of all redo log activity in the configuration. You can use the View Data drop-down list to set the collection interval (which causes the charts to be refreshed) to determine the rate of sampling of the primary database.

The following charts display performance information for all databases in the configuration:

- **Redo Generation Rate:** The redo generation rate in KB per second
- **Lag Times:** The approximate amount of potential data loss. *Lag Times* shows the transport lag (the approximate number seconds of redo not yet available on the standby database) and Apply Lag (the approximate number of seconds the standby is behind the primary database).
- **Apply Rate:** The data applied on each standby database in the configuration. Each point on the chart represents the amount of redo data that has been applied since the last time it was refreshed.

Click any of the charts to obtain historical information.

Viewing Log File Details

ORACLE Enterprise Manager Cloud Control 12c

Setup ▾ Help ▾ | SYSMAN ▾ | Log Out

Enterprise ▾ Targets ▾ Favorites ▾ History ▾ Search Target Name ▾

boston.example.com Logged in as sys host01.example.com

Oracle Database ▾ Performance ▾ Availability ▾ Schema ▾ Administration ▾

Data Guard > Log File Details

View Data Real Time: Manual Refresh

Page Refreshed Aug 29, 2013 5:40:03 AM UTC

A table is shown below for each standby database in the configuration, listing redo log files that have not been received by the standby database and redo log files that have been received but not applied to the standby database. Related redo transport and apply information is also displayed for diagnostic purposes.

Primary Current Log 33

london.example.com

Redo Transport Services On
Redo Transport Status Normal

Redo Apply Services On
Status Normal
Apply Delay (minutes) 0

Log	Status	ResetLogs ID #	First Change # (SCN)	Last Change # (SCN)	Size (KB)	Time Generated	Time Completed
All logs have been received and applied.							

london2.example.com

Redo Transport Services On
Redo Transport Status Normal

Redo Apply Services On
Status Normal
Apply Delay (minutes) 0

Log	Status	ResetLogs ID #	First Change # (SCN)	Last Change # (SCN)	Size (KB)	Time Generated	Time Completed
All logs have been received and applied.							

ORACLE

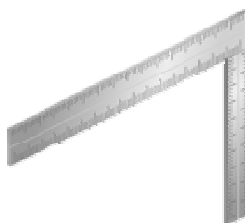
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Log File Details page displays information about the log files that were generated on the primary database and received by the standby database. The information is presented in a tabular format for each standby database in the configuration. The table contains the following columns:

- **Log:** Contains the log file sequence number
- **Status:** “Partially Applied,” “Not Applied,” “Not Received”
- **ResetLogs ID:** Identifier that changes after a terminal recovery and an open with RESETLOGS
- **First Change # (SCN):** First system change number (SCN) in the log file
- **Last Change # (SCN):** Last SCN in the log file
- **Size (KB):** Size of the log file
- **Time Generated:** Time at which the log file was generated
- **Time Completed:** Time at which the log file completed

Enterprise Manager Metrics and Alerts

- Metrics: Units of measurement that are used to assess the health of your system
- Thresholds: Boundary values against which monitored metric values are compared
- Alert: An indication that is generated when a threshold is reached



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Metrics are units of measurement that are used to assess the health of your system. Each target comes with a predefined set of metrics.

Metric thresholds are boundary values against which monitored metric values are compared. Some thresholds are predefined by Oracle; others are not.

When a threshold is reached, an alert is generated to indicate that a particular condition was encountered. An alert is triggered when one of the following conditions is true:

- A threshold is reached.
- An alert was cleared.
- The availability of a monitored service changes.
- A specific condition occurs. (Example: An alert is triggered when an error message is written to a database alert log file.)

Alerts are detected through a polling-based mechanism by checking for the monitored condition from a separate process at regular, predefined intervals.

You can associate an alert with a notification, with the automatic execution of a job, and so on.

Data Guard Metrics

Enterprise Manager provides Data Guard metrics:

Metric	Comparison Operator	Warning Threshold	Critical Threshold	Corrective Actions	Collection Schedule	Edit
▼ boston.example.com						
▶ Alert Log					Disabled	
▶ Alert Log Error Status					Disabled	
▶ Archive Area					Every 15 Minutes	
▶ Database Files					Disabled	
▶ Database Job Status					Every 30 Minutes	
▶ Database Limits					Every 10 Minutes	
▶ Database Services					Every 10 Minutes	
▶ Database Vault Attempted Violations - Command Rules					Every 1 Hour	
▶ Database Vault Attempted Violations - Realms					Every 1 Hour	
▶ Database Vault Configuration Issues - Command Rules					Every 1 Hour	
▶ Database Vault Configuration Issues - Realms					Every 1 Hour	
▶ Database Vault Policy Changes					Every 1 Hour	
▶ Data Failure					Every 5 Minutes	
▼ Data Guard Fast-Start Failover Observer					Every 1 Minute	
Observer Status	Contains		Error	None		
▼ Data Guard Performance					Every 5 Minutes	
Redo Generation Rate (KB/second)	>			None		
▼ Data Guard Status					Every 5 Minutes	
Data Guard Status	Contains	Warning	Error	None		

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can use Enterprise Manager to monitor status and log file activity. In addition, Enterprise Manager automatically monitors the status and archived redo log file activity on the primary and standby databases and provides the following metrics:

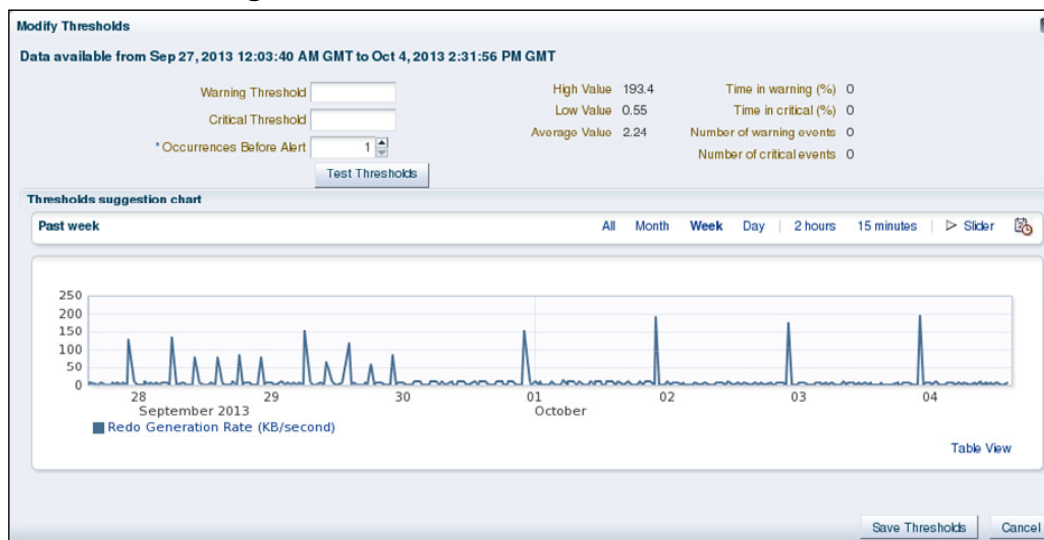
- **Data Guard Failover:** When fast-start failover is enabled, this metric generates a critical alert on the new primary database if a fast-start failover occurs.
- **Data Guard Fast-Start Failover Observer:** This metric displays the current status of the fast-start failover observer. A down status triggers an alert. The observer is a separate OCI client-side component that monitors the availability of the primary database in a fast-start failover configuration. Fast-start failover is discussed in the lesson titled “Enabling Fast-Start Failover.”
- **Data Guard Performance:** This metric provides alerts for performance that is related to redo log activity in the configuration.
- **Data Guard Status:** This metric checks the status of each database in the broker configuration and triggers a warning or critical alert if necessary.

You can set up Email Services to send you a message if a metric is triggered. See *Oracle Data Guard Broker* for detailed information.

Note: These metrics are seen on the primary database only.

Managing Data Guard Metrics

1. Configure notification methods.
2. View the All Metrics page.
3. Set or change Data Guard metric thresholds.



ORACLE

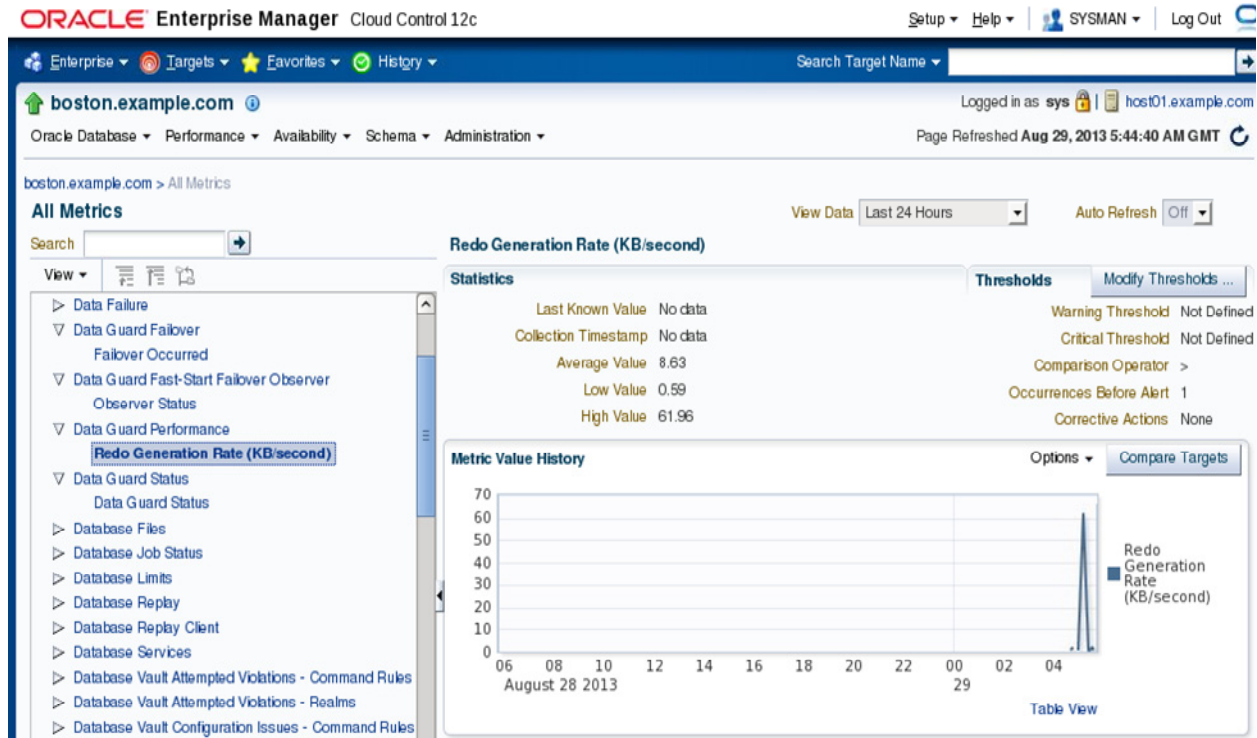
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can specify that an email notification be sent to you when a Data Guard metric is triggered. To configure the notification:

1. Configure notification methods in Enterprise Manager.
 - a. Click Setup at the top of the Database Home page, followed by Notifications and then Notification Methods on the menu tree that appears
 - b. Enter the appropriate information in the Mail Server section and click Apply. Next, click Test Mail Servers to verify your configuration.
2. View the All Metrics page by clicking Database > Monitoring > All Metrics using the menu on the Database Home page.
3. Set or change Data Guard metric thresholds by selecting an individual metric and then clicking the modify Thresholds button within the section dedicated for that metric. Not all metrics have thresholds that can be modified.

If a metric condition is triggered or a threshold value is exceeded, an alert is issued. Notifications determine how the alert will be received.

Viewing Metric Value History



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When an alert is triggered, you can view detailed information by clicking the link in the Message field. You can also view metric value history by clicking the metric link in the All Metrics tree view.

Viewing Data Guard Diagnostic Information

- The Data Guard broker records information in:
 - Oracle database alert log files
 - Data Guard broker log files
- Database status information is available by issuing the `SHOW DATABASE` command.
- Use the following database properties to obtain additional information:

– <code>StatusReport</code>	– <code>LsbyFailedTxnInfo</code>
– <code>LogXptStatus</code>	– <code>LsbyParameters</code>
– <code>InconsistentProperties</code>	– <code>[Recv/Send] QEntries</code>
– <code>InconsistentLogXptProps</code>	– <code>TopWaitEvents</code>

```
DGMGRL> SHOW DATABASE london 'RecvQEntries';
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Data Guard broker records activity information in the Oracle database alert log file and in the Data Guard broker log files. The broker log files are named `drc<$ORACLE_SID>.log` and are located in the same directory as the alert log file.

You can obtain information about the health of the database (referred to as the *database status*) by issuing the `SHOW DATABASE DGMGRL` command.

The following monitorable database properties can be used to query the database status:

- **StatusReport:** Lists all problems detected by the broker during a database health check
- **LogXptStatus:** Lists all log transport errors detected on all instances of the primary database
- **InconsistentProperties:** Lists all properties that have inconsistent values between the broker configuration file and the database settings
- **InconsistentLogXptProps:** Lists all redo transport–related properties of standby databases that have inconsistent values between the broker configuration file and the redo transport settings
- **LsbyFailedTxnInfo:** Identifies a failed transaction that caused log apply service to stop

- **LogbyParameters:** Identifies the value of MAX_SGA (maximum system global area) and MAX_SERVERS (maximum number of parallel query servers) specifically reserved for log apply services
- **RecvQEntries:** Returns a table indicating all log files that were received by the standby database but have not yet been applied
- **SendQEntries:** Returns a table that shows all log files on the primary database that were not successfully archived to one or more standby databases
- **TopWaitEvents:** Specifies the 5 events with the longest waiting time in the specified instance

Using Monitorable Database Properties to Identify a Failure

1. Check the configuration status:

```
DGMGRL> SHOW CONFIGURATION;
```

2. Check the database status:

```
DGMGRL> SHOW DATABASE london;
```

3. Check a Far Sync status:

```
DGMGRL> SHOW FAR_SYNC 'bostonFS'
```

4. Check additional monitorable database properties

```
DGMGRL> SHOW DATABASE london 'InconsistentProperties';
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can use the `SHOW CONFIGURATION`, `SHOW DATABASE`, and `SHOW FAR_SYNC` commands along with additional monitorable database properties to identify and determine an appropriate resolution for a failure in your Database Guard configuration.

1. Use the `SHOW CONFIGURATION` command to check the status of the configuration. The status is an aggregated status of all databases and instances in the broker configuration. If everything is working properly in the configuration, the output of this command with respect to status is "SUCCESS." If there is a problem in the configuration, you receive an error message and it will indicate which databases are in warning or error states.
2. If you receive an error message when you execute the `SHOW CONFIGURATION` command, execute the `SHOW DATABASE` command for each database or the `SHOW FAR_SYNC` command for each Far Sync to view a partial list of the warnings and errors for the database.
3. After viewing the `StatusReport` output, you can view the other monitorable database properties such as `InconsistentProperties`, `LogXptStatus`, and `InconsistentLogXptProps`.

Using the SHOW CONFIGURATION DGMGRL Command to Monitor the Configuration

Use the SHOW CONFIGURATION command to display a brief description of the configuration status:

```
DGMGRL> show configuration
Configuration - DRSolution
Protection Mode: MaxPerformance
Databases:
boston      - Primary database
              WARNING: ORA-16809: multiple warnings detected for
              this database
bostonFS    - Far Sync
              london      - Physical standby database
              london2     - Logical standby database
              londonFS    - Far Sync (inactive)

Fast-Start Failover: DISABLED
Configuration Status:
WARNING
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The SHOW CONFIGURATION DGMGRL command provides a brief description of the configuration, including the state of the configuration, the protection mode, and the state of fast-start failover. The display also lists the databases that are part of the configuration.

In addition, the current status of the configuration is provided; if there is a problem with the configuration, a warning message appears. You can investigate the problem as described on the following slides.

Indentation is used in the database listing to show redo sources and redo destinations.

Using the SHOW DATABASE VERBOSE DGMGRL Command to Monitor the Configuration

Use the SHOW DATABASE VERBOSE command to display all property values for a database:

```
DGMGRL> show database verbose london
Database - london

Role:                PHYSICAL STANDBY
Intended State:      APPLY-ON
Transport Lag:       0 seconds (computed 1 second ago)
Apply Lag:           0 seconds (computed 1 second ago)
Apply Rate:          1.52 MByte/s
Real Time Query:     OFF
Instance(s):
    london

Properties:
    DGConnectIdentifier          = 'london'
...
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Use the SHOW DATABASE VERBOSE DGMGRL command to view a brief summary and the properties of the specified database. An example of the output:

```
DGMGRL> show database verbose london
```

Database - london

```
Role:                PHYSICAL STANDBY
Intended State:      APPLY-ON
Transport Lag:       0 seconds (computed 1 second ago)
Apply Lag:           0 seconds (computed 1 second ago)
Apply Rate:          1.52 MByte/s
Real Time Query:     OFF
Instance(s):
    london
```

Properties:

```

DGConnectIdentifier           = 'london'
ObserverConnectIdentifier     = ''
LogXptMode                    = 'ASYNC'
RedoRoutes                    = '(Local:londonFS SYNC)'
DelayMins                     = '0'
Binding                       = 'optional'
MaxFailure                    = '0'
MaxConnections                = '1'
ReopenSecs                    = '300'
NetTimeout                    = '30'
RedoCompression               = 'DISABLE'
LogShipping                   = 'ON'
PreferredApplyInstance        = ''
ApplyInstanceTimeout          = '0'
ApplyLagThreshold             = '0'
TransportLagThreshold         = '0'
TransportDisconnectedThreshold = '30'
ApplyParallel                 = 'AUTO'
StandbyFileManagement         = 'AUTO'
ArchiveLagTarget              = '0'
LogArchiveMaxProcesses        = '10'
LogArchiveMinSucceedDest      = '1'
DbFileNameConvert             = 'boston, london'
LogFileNameConvert            = 'boston, london'
FastStartFailoverTarget       = ''
InconsistentProperties         = '(monitor)'
InconsistentLogXptProps       = '(monitor)'
SendQEntries                  = '(monitor)'
LogXptStatus                   = '(monitor)'
RecvQEntries                  = '(monitor)'
StaticConnectIdentifier        =
'(DESCRIPTION=(ADDRESS=(PROTOCOL=tcp)(HOST=host03)(PORT=1521))(CONNECT_
DATA=(SERVICE_NAME=london_DGMGRL.example.com)(INSTANCE_NAME=london)(SER
VER=DEDICATED)))'
StandbyArchiveLocation         = 'USE_DB_RECOVERY_FILE_DEST'
AlternateLocation              = ''
LogArchiveTrace                = '0'
LogArchiveFormat               = 'arch_%t_%s_%r.log'
TopWaitEvents                  = '(monitor)'

```

Database Status:

SUCCESS

Viewing Standby Redo Log Information in V\$LOGFILE

```
SQL> SELECT group#, member
2 FROM v$logfile
3 WHERE type = 'STANDBY';
```

GROUP#	STATUS	MEMBER
-----	-----	-----
4		+DATA/boston/onlinelog/group_4.278.711989145
5		+DATA/boston/onlinelog/group_5.279.711989151
6		+DATA/boston/onlinelog/group_6.280.711989159
7		+DATA/boston/onlinelog/group_7.281.711989165

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Obtain information about the standby redo log by querying V\$LOGFILE. The STATUS column contains the following possible values:

- **INVALID:** The file is inaccessible.
- **STALE:** The file contents are incomplete.
- **DELETED:** The file is no longer used.
- **Null:** The file is in use.

The CON_ID column of this view provides information useful in a multitenant architecture. Possible values for the CON_ID column include:

- **0:** This value is used for rows containing data that pertain to the entire CDB. This value is also used for rows in non-CDBs.
- **1:** This value is used for rows containing data that pertain to only the root
- **n:** Where n is the applicable container ID for the rows containing data

Viewing Standby Redo Log Information in V\$STANDBY_LOG

```
SQL> SELECT group#, dbid, archived, status
2 FROM v$standby_log;
```

GROUP#	DBID	ARC	STATUS
4	UNASSIGNED	NO	UNASSIGNED
5	3303427449	YES	ACTIVE
6	UNASSIGNED	YES	UNASSIGNED
7	UNASSIGNED	YES	UNASSIGNED

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Query V\$STANDBY_LOG to obtain information about the standby redo log. Columns of interest include:

- **GROUP#:** Log group number
- **DBID:** Database ID of the primary database to which the standby redo log file is assigned. If the standby redo log file is unassigned, the value UNASSIGNED is displayed.
- **ARCHIVED:** Contains a value of YES or NO. See STATUS for additional information.
- **STATUS:** Contains a value of UNASSIGNED or ACTIVE
 - **UNASSIGNED:** If the value of ARCHIVED is NO, the standby redo log was archived and is again available. If the value of ARCHIVED is YES, the standby redo log was never used and is available.
 - **ACTIVE:** If the value of ARCHIVED is NO, the standby redo log is complete and waiting to be archived. If the value of ARCHIVED is YES, the standby redo log is currently being written to and is not ready to be archived.

The CON_ID column of this view provides information useful in a multitenant architecture.

Identifying Destination Settings

```
SQL> SELECT dest_id,valid_type,valid_role,valid_now
2 FROM v$archive_dest;
DEST_ID VALID_TYPE          VALID_ROLE    VALID_NOW
-----
1 ALL_LOGFILES             ALL_ROLES     YES
2 STANDBY_LOGFILE          STANDBY_ROLE  WRONG VALID_TYPE
3 ONLINE_LOGFILE           STANDBY_ROLE  WRONG VALID_ROLE
4 ALL_LOGFILES             ALL_ROLES     UNKNOWN
5 ALL_LOGFILES             ALL_ROLES     UNKNOWN
6 ALL_LOGFILES             ALL_ROLES     UNKNOWN
7 ALL_LOGFILES             ALL_ROLES     UNKNOWN
...
9 ALL_LOGFILES             ALL_ROLES     UNKNOWN
30 ALL_LOGFILES            ALL_ROLES     UNKNOWN
31 ALL_LOGFILES            ALL_ROLES     YES
31 rows selected.
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The `VALID_NOW` column in `V$ARCHIVE_DEST` indicates whether the archive log destination is used. The column contains the following values:

- **YES:** The archive log destination is appropriately defined for the current database role.
- **WRONG VALID_TYPE:** The archive log destination is appropriately defined for the current database role but cannot be used. For example, `LOG_ARCHIVE_DEST_2` is set to `(STANDBY_LOGFILES,STANDBY_ROLE)`, but `WRONG VALID_TYPE` is returned because this standby destination does not have an implemented standby redo log.
- **WRONG VALID_ROLE:** The archive log destination is not appropriately defined for the current database role. For example, `LOG_ARCHIVE_DEST_3` is set to `(ONLINE_LOGFILES,STANDBY_ROLE)`, but `WRONG VALID_ROLE` is returned because this destination is currently running in the primary database role.
- **UNKNOWN:** The archive log destination is not defined.

The `VALID_TYPE` and `VALID_ROLE` columns are the values from the `VALID_FOR` attribute that is specified for each archive log destination.

Setting the LOG_ARCHIVE_TRACE Initialization Parameter

- LOG_ARCHIVE_TRACE is optional and is used for diagnostic purposes.
- Set this parameter to an integer value to see the progression of redo log archiving to the standby system.
 - On the primary database, processes write an audit trail of the archived logs sent to the standby system into a trace file and information into the alert log.
 - On the standby database, processes write an audit trail of the archived logs received from the primary database into a trace file and information into the alert log.
- Trace files are written to the Automatic Diagnostic Repository, the location of which is specified by the DIAGNOSTIC_DEST initialization parameter.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Set this parameter to trace the transmission of redo data to the standby system.

To enable, disable, or modify the LOG_ARCHIVE_TRACE parameter in a primary database, issue the ALTER SYSTEM SET LOG_ARCHIVE_TRACE=trace_level statement while the database is open or mounted. If you change the value of this parameter dynamically with an ALTER SYSTEM statement, the changes take effect at the start of the next archive operation.

See *Oracle Data Guard Concepts and Administration* for additional information.

The integer values for the `LOG_ARCHIVE_TRACE` parameter represent levels of tracing data. In general, a higher level indicates more detailed information. You can combine tracing levels by setting the value of the `LOG_ARCHIVE_TRACE` parameter to the sum of the individual levels. For example, setting the parameter to 6 generates level-2 and level-4 trace output.

The following integer levels are available:

Level	Meaning
0	Disables archived redo log tracing (default setting)
1	Tracks archiving of redo log file
2	Tracks archival status per archived redo log destination
4	Tracks archival operational phase
8	Tracks archived redo log destination activity
16	Tracks detailed archived redo log destination activity
32	Tracks archived redo log destination parameter modifications
64	Tracks ARCn process state activity
128	Tracks FAL server process activity
256	Tracks RFS logical client
512	Tracks LGWR redo shipping network activity
1024	Tracks RFS physical client
2048	Tracks ARCn or RFS ping heartbeat
4096	Tracks real-time apply activity
8192	Tracks redo apply activity (media recovery or physical standby)
16384	Tracks redo transport buffer management
32768	Tracks LogMiner dictionary

Viewing Redo Transport Errors by Querying V\$ARCHIVED_DEST

```
SQL> SELECT DEST_ID "ID", STATUS "DB_status",
DESTINATION "Archive_dest", ERROR "Error" FROM
V$ARCHIVE_DEST WHERE DEST_ID <=5;
```

ID	DB_status	Archive_dest	Error
1	VALID	/u01/app/oracle/ fast_recovery_area/ london	
2	ERROR	standby1	ORA-16012: Archivelog standby database identifier mismatch
3	INACTIVE		
4	INACTIVE		
5	INACTIVE		

5 rows selected.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If the standby site is not receiving redo data, query the V\$ARCHIVE_DEST view and check for error messages. If the output of the query does not help you, check the following list of possible issues. If any of the following conditions exist, redo transport services will fail to transmit redo data to the standby database:

- The service name for the standby instance is not configured correctly in the `tnsnames.ora` file for the primary database.
- The Oracle Net service name specified by the `LOG_ARCHIVE_DEST_n` parameter for the primary database is incorrect.
- The `LOG_ARCHIVE_DEST_STATE_n` parameter for the standby database is not set to the value `ENABLE`.
- The `listener.ora` file has not been configured correctly for the standby database.
- The listener is not started at the standby site.
- The standby instance is not started.
- You have added a standby archiving destination to the primary SPFILE or text initialization parameter file, but have not yet enabled the change.
- Redo transport authentication has not been configured properly.

Evaluating Redo Data by Querying V\$DATAGUARD_STATS

```
SQL> SELECT name, value, time_computed FROM  
v$dataguard_stats;
```

NAME	VALUE	TIME_COMPUTED
-----	-----	-----
transport lag	+00 00:00:00	10/08/2013 20:07:05
apply lag	+00 00:00:00	10/08/2013 20:07:05
apply finish time	+00 00:00:00	10/08/2013 20:07:05
estimated startup time	10	10/08/2013 20:07:05



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Query V\$DATAGUARD_STATS to evaluate each standby database in terms of the currency of the data in the standby database and the time it takes to perform a role transition if all available redo data is applied to the standby database.

V\$DATAGUARD_STATS displays the amount of redo data generated by the primary database that is not yet available on the standby database. This information enables you to determine how much redo data would be lost if the primary database were to crash when you queried this view.

See the *Oracle Database Reference* for detailed information about column values.

Viewing Data Guard Status Information by Querying V\$DATAGUARD_STATUS

```
SQL> SELECT timestamp, facility, message FROM
v$dataguard_status ORDER by timestamp;
```

```
TIMESTAMP FACILITY
```

```
-----
```

```
MESSAGE
```

```
-----
```

```
...
```

```
ARC9: Beginning to archive thread 1 sequence 122
(5336651-5404240)
```

```
08-OCT-13 Log Apply Services
Media Recovery Waiting for thread 1 sequence 123 (in
transit)
```

```
08-OCT-13 Log Transport Services
ARC9: Completed archiving thread 1 sequence 122 (0-0)
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Query V\$DATAGUARD_STATUS to view messages that were recently written to the alert log or server process trace files that concern physical standby databases or redo transport services for all standby database types.

See the *Oracle Database Reference* for detailed information about column values.

Monitoring Redo Apply by Querying V\$MANAGED_STANDBY

```
SQL> SELECT process, status, group#, thread#, sequence#
2  FROM v$managed_standby
3  order by process, group#, thread#, sequence#;
```

PROCESS	STATUS	GROUP#	THREAD#	SEQUENCE#
-----	-----	-----	-----	-----
ARCH	CLOSING	4	1	142
ARCH	CLOSING	4	1	146
ARCH	CLOSING	4	1	148
ARCH	CLOSING	5	1	141
ARCH	CLOSING	5	1	147
MRP0	APPLYING_LOG	N/A	1	149
RFS	IDLE	2	1	149
RFS	IDLE	N/A	0	0
RFS	IDLE	N/A	0	0

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Query V\$MANAGED_STANDBY to view information about Redo Apply and redo transport status on a physical standby database. See the *Oracle Database Reference* for detailed information about the values in each column.

Monitoring SQL Apply by Querying V\$LOGSTDBY_TRANSACTION

Query V\$LOGSTDBY_TRANSACTION to view information about all transactions that are actively being processed by SQL Apply:

```
SQL> SELECT primary_xid, type,  
2      mining_status, apply_status  
3      FROM v$logstdby_transaction;
```

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is centered on a solid red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Query V\$LOGSTDBY_TRANSACTION to view information about the transactions that are actively being processed by SQL Apply. The transaction identifiers shown in this view correspond to transaction identifiers assigned at the primary database and do not correspond to the transactions that are active at the logical standby database. Query the V\$TRANSACTION view on the logical standby database for information about transactions that are active in the logical standby database, including those that were created as part of SQL Apply.

Quiz

You can click the Data Guard performance charts that show real-time data in Oracle Enterprise Manager to view historical data.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: a

Quiz

The `LOG_ARCHIVE_TRACE` initialization parameter is valid only on the primary database to display diagnostic information regarding sending of redo data.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: b

Summary

In this lesson, you should have learned how to:

- Use Enterprise Manager Grid Control to monitor the configuration
- Use DGMGRL to monitor the configuration
- Use SQL to monitor the configuration

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 10: Overview

This practice covers the following topics:

- Verifying the broker configuration
- Viewing log file details
- Checking the configuration status
- Checking the database status

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

11

Configuring Data Protection Modes

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Objectives

After completing this lesson, you should be able to:

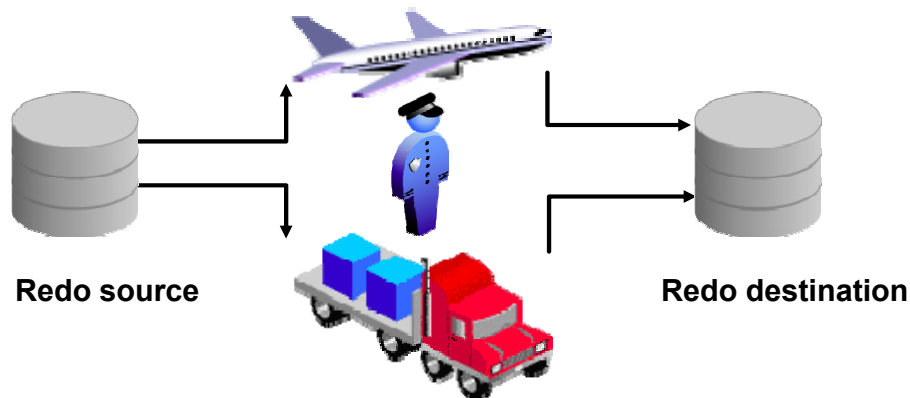
- Describe the data protection modes
- Change the data protection mode of your configuration
 - Using Enterprise Manager Cloud Control
 - Using DGMGRL
 - Using SQL

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Data Protection Modes and Redo Transport Modes

- A data protection mode requires a specific redo transport mode.
- A redo transport mode alone does not define a data protection mode.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When you define a redo transport mode, you are configuring the shipment of log files from a redo source to a redo destination. The redo source is usually a primary database or a Far Sync instance. A redo destination is usually a physical or logical standby database. It is possible for a standby database to be considered a redo source to cascaded standby databases. You must set your redo transport mode to support the protection mode that you want for your configuration. However, configuring the redo transport mode alone does not set up the protection mode.

After setting up the redo transport mode, you can put the configuration into a data protection mode. The data protection mode setting causes internal rules to be implemented, ensuring that your configuration is protected at the necessary level.

Data Protection Modes

- Three data protection modes:
 - Maximum protection
 - Maximum availability
 - Maximum performance
- Help to balance data availability and system performance



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Data Guard offers maximum protection, maximum availability, and maximum performance modes to help enterprises balance data availability against system performance requirements.

In some situations, a business cannot afford to lose data. In other situations, the availability of the database may be more important than the loss of data. Some applications require maximum database performance and can tolerate the potential loss of data.

Maximum Protection Mode

- Maximum protection mode ensures zero data loss in the event of a failure of the primary database, the network, or all standby databases.
- The primary database shuts down if a fault prevents it from writing its redo stream to at least one synchronized standby database.
- Redo data must be written to both the local online redo log and the remote standby redo log on at least one synchronized standby database.
- Configuration requirements: At least one standby database must have a standby redo log, and that standby database destination must be configured with the `SYNC` and `AFFIRM` redo transport attributes.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

This protection mode ensures that no data loss occurs if the primary database fails. To provide this level of protection, the redo data that is needed to recover each transaction must be written to both the local online redo log and the standby redo log on at least one standby database before the transaction commits. To ensure that data loss cannot occur, the primary database shuts down if a fault prevents it from writing its redo stream to at least one remote standby redo log. For multiple-instance RAC databases, Data Guard shuts down the primary database if it is unable to write the redo records to at least one properly configured database instance.

Maximum protection mode requirements:

- Configure standby redo log files on at least one standby database.
- Set the `SYNC` and `AFFIRM` attributes of the `LOG_ARCHIVE_DEST_n` parameter for at least one standby database destination.

Note: Oracle recommends a minimum of two standby databases for maximum protection mode.

Far Sync does not support the maximum protection mode.

Maximum Availability Mode

- Maximum availability mode ensures zero data loss without compromising the availability of the primary database.
- Redo data must be written to both the local online redo log and received by the remote RFS process of a standby database.
- The primary database does not shut down if it cannot receive acknowledgement from at least one standby.
- If no synchronized standby databases are available, the primary database operates in an unsynchronized mode until at least one standby database is synchronized.
- Configuration requirements: At least one standby database must have a standby redo log, and that standby database destination must be configured with the `SYNC` and `NOAFFIRM` redo transport attributes as a minimum.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

This protection mode provides the highest possible level of data protection without compromising the availability of the primary database. A transaction does not commit until the redo that is needed to recover that transaction is written to the local online redo log and has been received at the standby. The standby must initiate I/O to the standby redo log and send acknowledgement back to the primary. It does not have to wait on the I/O to complete. This feature is known as FastSync in Oracle Database 12c. The primary database does not shut down if a fault prevents communicating with a standby instance. Instead, the primary database operates in an unsynchronized mode until the fault is corrected and all gaps in redo log files are resolved. When all gaps are resolved and the standby database is synchronized, the primary database automatically resumes operating in maximum availability mode.

This mode guarantees that no data loss occurs if the primary database fails—but only if a second fault does not prevent a complete set of redo data from being sent from the primary database to at least one standby database.

Maximum availability mode requirements:

- Configure standby redo log files on at least one standby database.
- Set the `SYNC` and `NOFFIRM` attributes of the `LOG_ARCHIVE_DEST_n` parameter for at least one standby database. The `SYNC` and `AFFIRM` attributes can also be used.

Maximum Performance Mode

- Maximum performance mode is the default level of data protection.
- This mode provides the highest possible level of data protection without affecting the performance of the primary database.
- Transactions can commit as soon as the redo data is written to the local online redo log.
- Redo data is shipped to the standby database asynchronously with respect to the commitment of the transactions that create the redo data.
- Configuration requirements:
 - Standby redo log on at least one standby database
 - At least one standby database that is configured with the `ASYNC` and `NOAFFIRM` redo transport attributes

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Maximum performance is the default protection mode and provides the highest possible level of data protection without affecting the performance of the primary database. This is accomplished by allowing a transaction to commit as soon as the redo data needed to recover that transaction is written to the local online redo log. The primary database's redo data is also written to at least one standby database, but that redo data is written asynchronously with respect to the commitment of the transactions that create the redo data.

When network links with sufficient bandwidth are used, this mode provides a level of data protection that approaches that of maximum availability mode with minimal impact on primary database performance.

Maximum performance mode requirement: Set the `ASYNC` and `NOAFFIRM` redo transport attributes of the `LOG_ARCHIVE_DEST_n` parameter on at least one standby database.

Comparing Data Protection Modes

Mode	Risk of Data Loss	Transport	If no acknowledgment is received:
Maximum Protection	Zero data loss Double failure protection	SYNC	Stall primary until an acknowledgement is received.
Maximum Availability	Zero data loss with single site failure	SYNC (AFFIRM)	Stall primary until threshold period expires, then resume processing.
Maximum Availability	Potential for data loss when multiple sites fail	SYNC (NOAFFIRM)	Stall primary until threshold period expires, then resume processing.
Maximum Performance	Potential for minimal data loss	ASync	Primary never waits for standby acknowledgement.

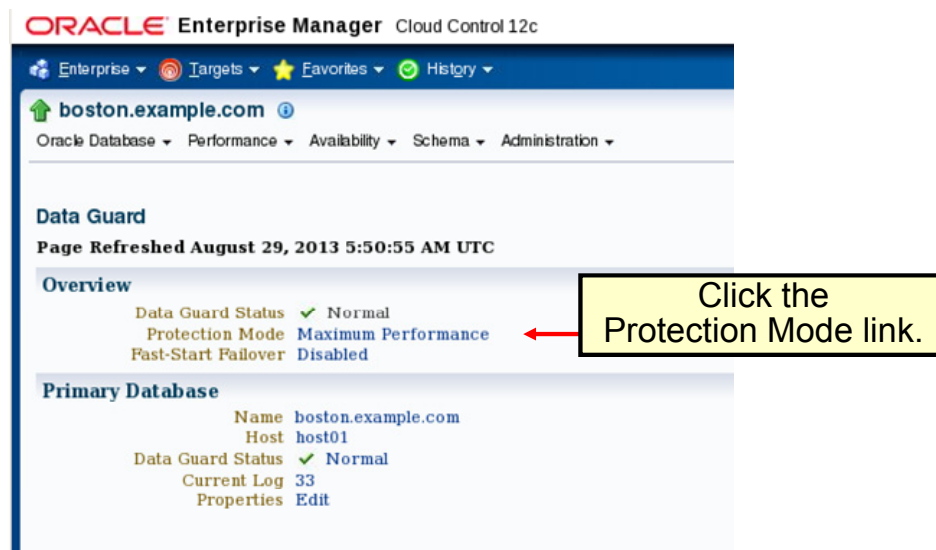


Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Consider the characteristics of each protection mode. You must balance cost, availability, performance, and transaction protection when choosing the protection mode.

Note: If you plan to enable fast-start failover, you must set the protection mode to maximum availability or maximum performance. See the lesson titled “Enabling Fast-Start Failover” for additional information.

Setting the Data Protection Mode by Using Enterprise Manager



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

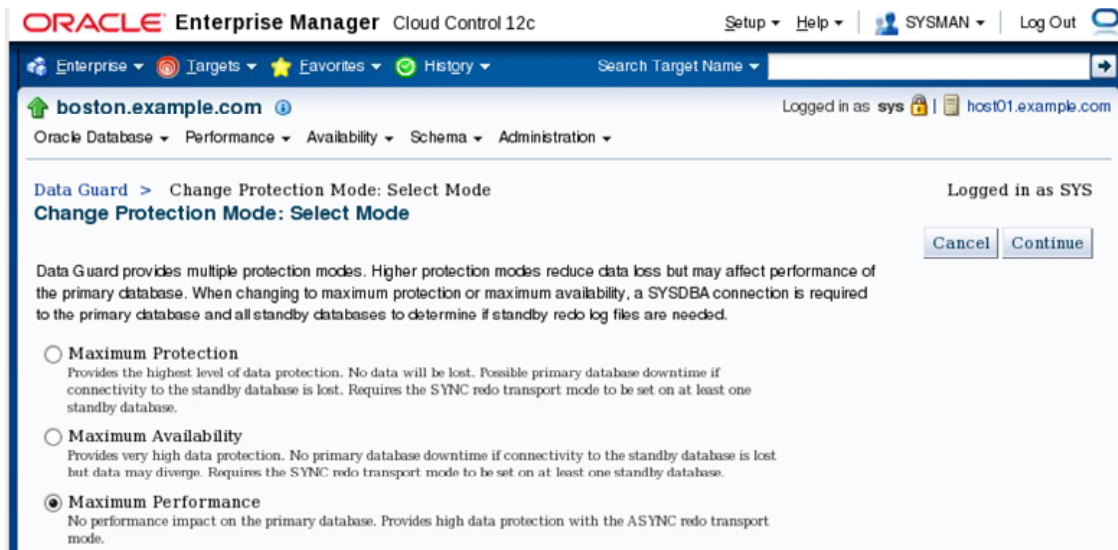
If the data protection mode that you need requires a standby database to use the SYNC or ASYNC redo transport mode, Enterprise Manager automatically sets the redo transport mode for the primary database and the selected standby databases.

Enterprise Manager automatically determines the correct number and size of standby redo log files that are needed for all databases in the configuration and adds those log files using the directory locations that you specify.

To set the data protection mode with Enterprise Manager:

1. Navigate to the Data Guard page.
2. Click the link in the Protection Mode field to access the Change Protection Mode: Select Mode page.

Setting the Data Protection Mode by Using Enterprise Manager



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

3. Select Maximum Protection, Maximum Availability, or Maximum Performance, and click Continue.
4. If prompted, enter the username and password of a user with SYSDBA privileges. Click Login.
5. Select one or more standby databases to support the protection mode that you selected. (If standby redo log files are needed, verify the names of the log files.) Click OK.
6. On the Confirmation page, click Yes.

Setting the Data Protection Mode by Using DGMGRL

1. Configure standby redo logs.
2. Set the `LogXptMode` property (if necessary).
 - Maximum protection: `SYNC`
 - Maximum availability: `SYNC` or `FASTSYNC`
 - Maximum performance: `ASYN`
3. Set the data protection mode.

```
DGMGRL> EDIT DATABASE 'london' SET PROPERTY
'LogXptMode'='FASTSYNC';
Property "LogXptMode" updated

DGMGRL> EDIT CONFIGURATION SET PROTECTION MODE AS
MAXAVAILABILITY;
Succeeded.
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

1. You must configure standby redo log files for the primary database or another standby database in the configuration to ensure that it can support the chosen protection mode after a switchover.
2. Use the `EDIT DATABASE SET PROPERTY` command to set the redo transport mode for the standby database. For example, if you are changing the data protection mode to maximum availability, use the `EDIT DATABASE` command to specify `SYNC` for redo transport services:

```
DGMGRL> EDIT DATABASE 'london' SET PROPERTY 'LogXptMode'='SYNC';
```

You must also set the redo transport services for the primary database or another standby database in the configuration to ensure that it can support the chosen protection mode after a switchover.

3. Use the `EDIT CONFIGURATION SET PROTECTION MODE AS` command to set the overall configuration protection mode. To set the protection mode to maximum availability, use the following command:

```
DGMGRL> EDIT CONFIGURATION SET PROTECTION MODE AS MAXAVAILABILITY;
```

Setting the Data Protection Mode by Using SQL

Set the data protection mode on the primary database:

```
SQL> ALTER DATABASE SET STANDBY DATABASE TO MAXIMIZE  
AVAILABILITY;  
Database Altered.
```

Confirm that the primary database is operating in the new protection mode:

```
SQL> SELECT protection_mode FROM v$database;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If you are not using the Data Guard broker, then you can use SQL to set the data protection mode. You must manually set the redo transport attributes correctly prior to setting the data protection mode. Note that the data protection mode can be set to `MAXIMUM PROTECTION` on an open database only if the current data protection mode is `MAXIMUM AVAILABILITY` and if there is at least one synchronized standby database. The `V$DATABASE` view can show the current protection mode.

Quiz

Which of the following are allowed Data Guard protection modes?

- a. Maximum Protection
- b. Maximum Availability
- c. Maximum Throughput
- d. Maximum Performance

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: a, b, d

Summary

In this lesson, you should have learned how to:

- Describe the data protection modes
- Change the data protection mode of your configuration
 - Using Enterprise Manager Cloud Control
 - Using DGMGRL
 - Using SQL

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 11: Overview

This practice covers setting the data protection mode:

- To meet specific business requirements
- By using DGMGRL

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

12

Performing Role Transitions

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Objectives

After completing this lesson, you should be able to:

- Explain the database roles
- Perform a switchover
- Perform a failover

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Role Management Services

- In a Data Guard configuration, a database operates in one of two mutually exclusive roles:
 - Primary role
 - Standby role (Physical, Logical, Snapshot subtypes)
- With role management services, you can change these roles dynamically.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can use role management services to change the primary and standby roles dynamically as a planned transition called a switchover operation, or as a result of a database failure through a failover operation.

For example, you might perform a switchover operation to transition the primary database to the standby role and transition a standby database to the primary role to perform routine maintenance tasks.

Role Transitions: Switchover and Failover

- Switchover
 - Planned role transition
 - Used for operating-system or hardware maintenance
 - Manually invoked on primary database
- Failover
 - Unplanned role transition
 - Used in an emergency
 - Minimal or no data loss (depending on the data-protection mode)
 - Fast-start failover can be enabled for automatic failover
 - Initiated at standby database

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Switchover

You can use the switchover feature to switch the role of the primary database to one of the available standby databases. The chosen standby database becomes the primary database, and the original primary database then becomes a standby database. There is no need to re-create any of the databases involved in the switchover operation. There is no data divergence between the original and new primary databases after successful completion of the switchover.

Failover

You invoke a failover operation when a failure occurs on the primary database and there is no possibility of recovering the primary database in a timely manner. During a failover operation, the standby database assumes the primary database role. You invoke the failover operation on the standby database that you want to fail over to the primary role.

Note: You can enable fast-start failover to fail over automatically when the primary database becomes unavailable. When fast-start failover is enabled, the broker determines whether a failover is necessary and initiates the failover to the specified target standby database automatically, with no need for DBA intervention and with no loss of data. For details, see the lesson titled “Enabling Fast-Start Failover.”

Switchover

- Transitions the roles of the primary and standby databases
- Requires no resetting of the online redo logs of the new primary database
- Incurs no data loss

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

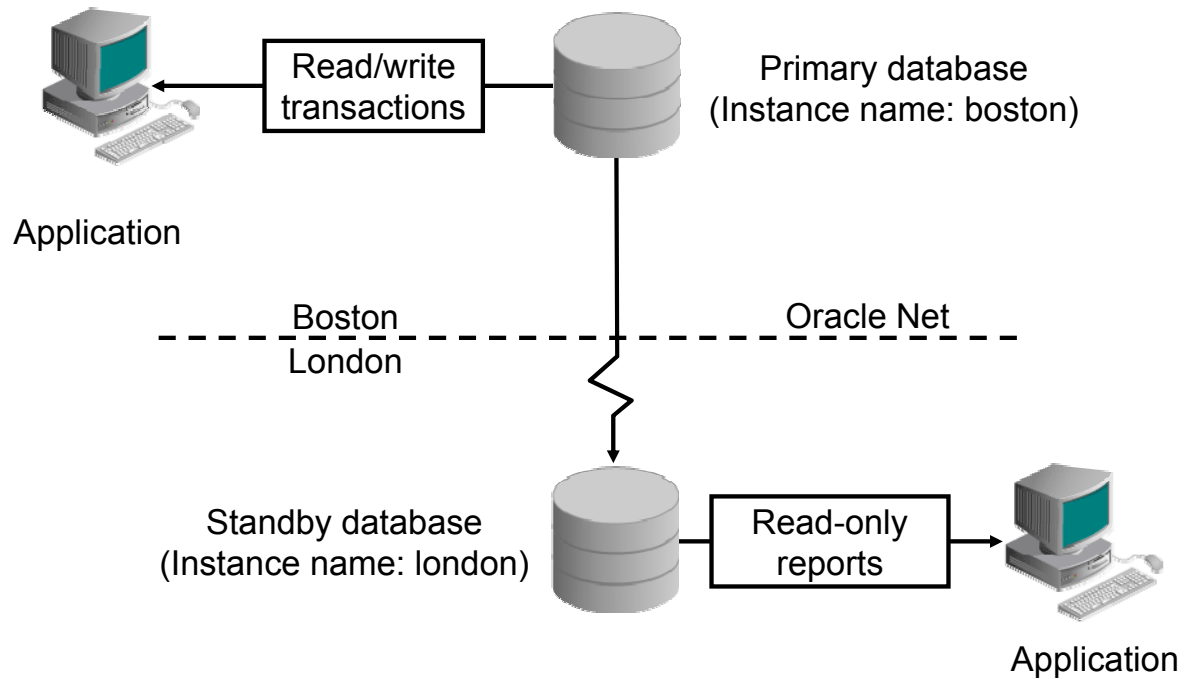
A switchover operation transitions the primary database to the standby role and transitions the standby database to the primary role, without resetting the online redo logs of the new primary database.

The primary database at the start of a switchover operation will need to be shut down and restarted. The physical standby database is not shut down and restarted during a switchover. If the physical standby database is in the Active Data Guard mode, it is closed for the transition and then opened again after the switchover completes, but it is never totally shut down to require a restart.

If the switchover operation involves a logical standby database, there is no need to shut down and restart either the primary database or any of the standby databases. Logical standby databases do not need to be shut down and restarted.

Note: When necessary, the broker automatically starts and stops all but one instance in a Real Application Clusters (RAC) environment for the primary database. If the broker is not used, this must be done manually. The secondary RAC instances of a physical standby database do not need to be shut down during the switchover. They can stay in the mounted or Active Data Guard state. The primary still requires only one instance to be running during a switchover.

Switchover: Before

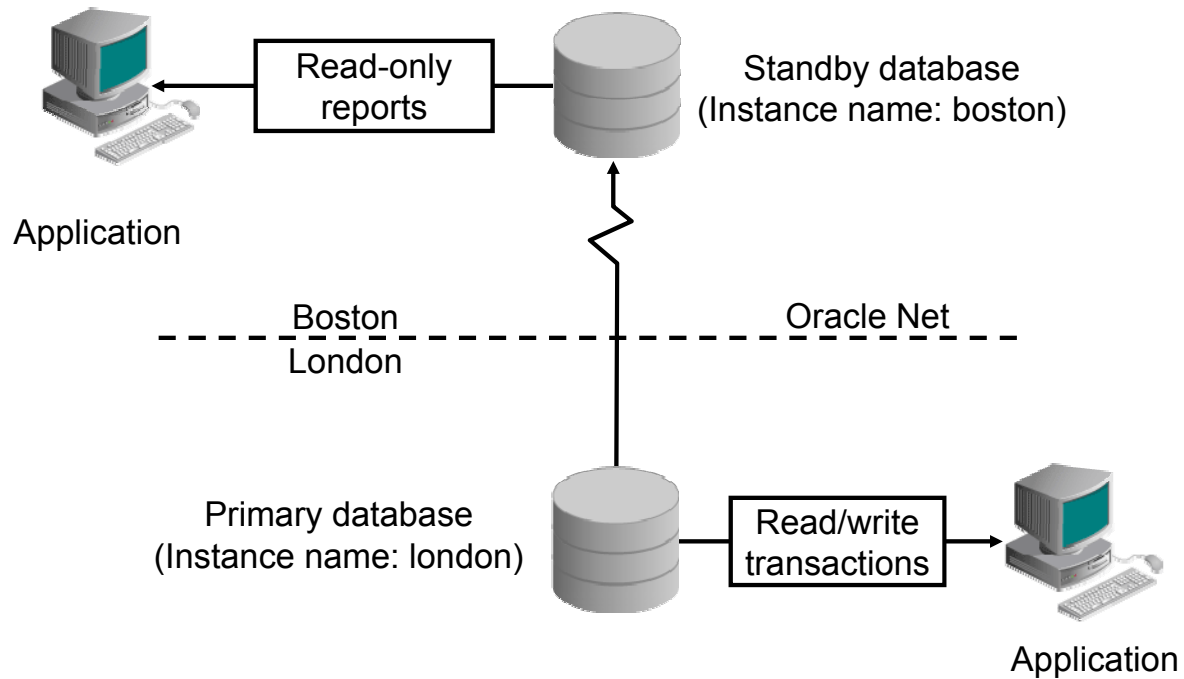


ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

As an example, suppose that the primary database is located in San Francisco and the physical standby database is located in Boston. Switchovers are started only on the primary database and completed on the target standby database. You can be connected to any database in the configuration through DGMGRL to execute the `SWITCHOVER` command.

Switchover: After



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

After the switchover is completed, each database has the role opposite to the one that it had before the switchover. In this example, Boston is now the primary database and San Francisco is the standby database.

Because Data Guard does not automatically switch over sessions from one database to the other, active sessions for each system need to reconnect. See the lesson titled “Enhanced Client Connectivity in a Data Guard Environment” for information about configuring automatic methods to reconnect sessions.

Performing a Switchover by Using Enterprise Manager

Oracle Enterprise Manager Cloud Control 12c

Setup Help SYSMAN Log Out

Enterprise Targets Favorites History Search Target Name

boston.example.com Logged in as sys host01.example.com

Oracle Database Performance Availability Schema Administration

Data Guard

Page Refreshed August 29, 2013 5:53:30 AM UTC View Data Real Time: Manual Refresh

Overview

- Data Guard Status ✓ Normal
- Protection Mode Maximum Performance
- Fast-Start Failover Disabled

Primary Database

- Name boston.example.com
- Host host01
- Data Guard Status ✓ Normal
- Current Log 33
- Properties Edit

Standby Database Progress Summary

Transport lag is the time difference between the last update on the primary database and the last received redo on the standby database. Apply lag is the time difference between the last update on the primary database and the last applied redo on the standby database.

seconds

1.0

0.5

0 0

london2.example.com

Transport Lag

Apply Lag

Standby Databases

Add Standby Database

Edit Remove **Switchover** Failover Convert

Select	Name	Host	Data Guard Status	Role	Real-time Query	Last Received Log	Last Applied Log	Estimated Failover Time
☒	london.example.com	host03	✓ Normal	Physical Standby	Disabled	32	32	< 1 second
☐	london2.example.com	host03	✓ Normal	Logical Standby	N/A	32	32	< 1 second

Select the database and click Switchover.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When it is used for the switchover, Enterprise Manager:

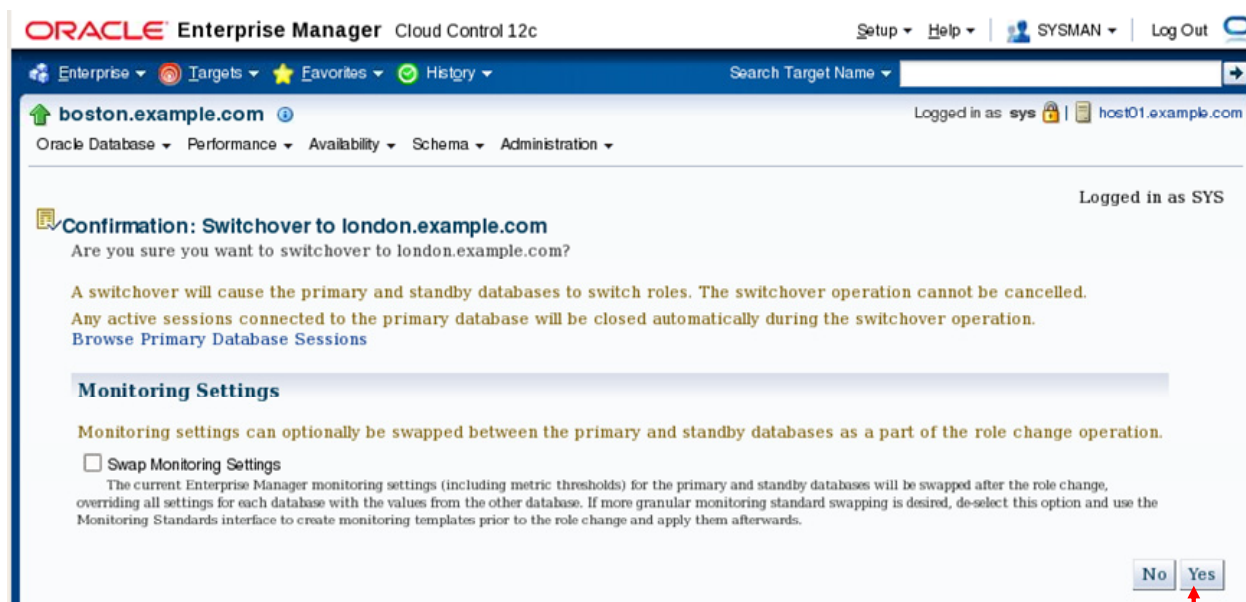
- Checks to ensure that the primary database and standby database are not currently in an error status condition and that broker management of the primary database is enabled and online
- Automatically closes any active sessions connected to the primary database during the switchover
- First changes the primary database to the standby role, and then changes the standby database to the primary role
- Opens the target database if it is a mounted physical standby database and restarts the former primary database

To initiate a switchover by using Enterprise Manager:

- On the Data Guard page, select the standby database that you want to become the primary database.
- Click Switchover. A credentials page may appear if you have not logged in to the standby database.

The Data Guard Switchover Confirmation page is displayed.

Performing a Switchover by Using Enterprise Manager



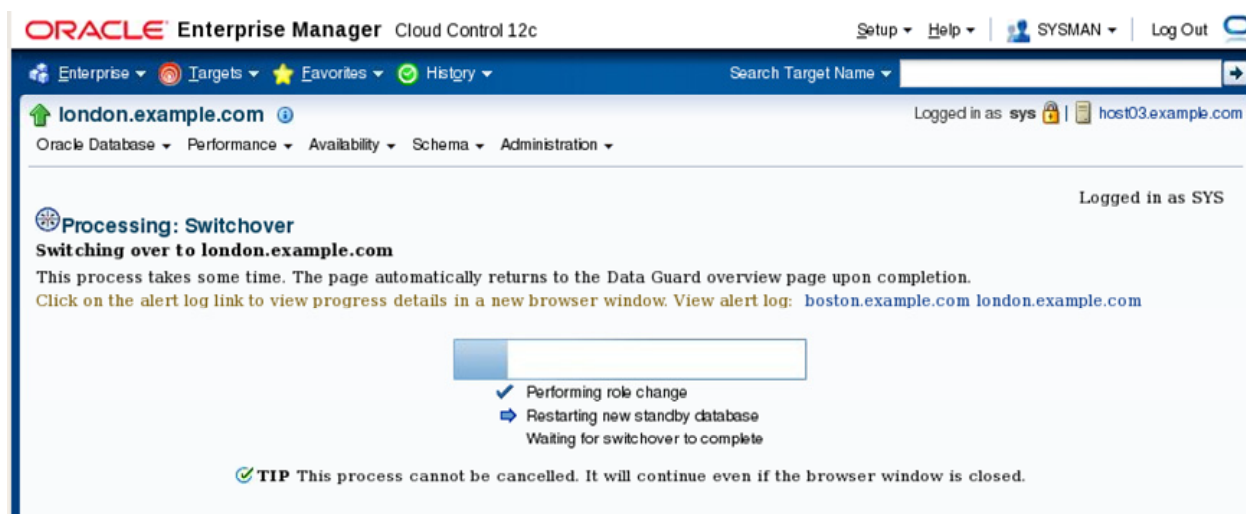
Click Yes to confirm.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

3. Click the Browse Primary Database Sessions link to view active sessions.
4. Click Yes to continue with the switchover, or click No to cancel.
You cannot cancel the switchover operation after it begins.

Performing a Switchover by Using Enterprise Manager



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Data Guard Switchover Processing page displays the progress of the switchover operation as it:

- Switches roles between the primary and standby databases (If the switchover target is a physical standby database and mounted, it is opened without restarting. The former primary database is restarted.)
- Waits for the Data Guard broker to complete the initialization tasks required to switch the database roles

You can view the database alert log of the primary and standby databases by clicking the respective “View alert log” links. A new browser window opens with the content of the alert log.

After the switchover operation is complete, you are returned to the Data Guard page.

Validating Databases for Switchover by Using DGMGRL

The `VALIDATE DATABASE` command performs a comprehensive set of database checks prior to a role change.

```
DGMGRL> validate database london
Database Role:      Physical standby database
Primary Database:   boston
Ready for Switchover: Yes
Ready for Failover: Yes (Primary Running)
Flashback Database Status:
  boston: Off
  london: Off
Current Log File Groups Configuration:
  Thread#   Online Redo Log Groups   Standby Redo Log Groups
           (boston)                (london)
    1         3                     2
Future Log File Groups Configuration:
  Thread #   Online Redo Log Groups   Standby Redo Log Groups
           (london)                (boston)
    1         3                     0
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The `VALIDATE DATABASE` command performs a comprehensive set of database checks prior to a role change. The checks use information available in various Oracle Data Guard views as well as the Automatic Diagnostic Repository. The `VALIDATE DATABASE` command shows a brief summary of the database, and reports any errors or warnings that were detected. `VALIDATE DATABASE VERBOSE` shows everything in the brief summary plus all items that were validated. The `VALIDATE DATABASE` command should be performed against both the primary database and the standby database prior to performing a switchover. An example of verbose output against a physical standby database is as follows:

```
DGMGRL> validate database verbose london
```

```
Database Role:      Physical standby database
Primary Database:   boston

Ready for Switchover: Yes
Ready for Failover: Yes (Primary Running)
```

Capacity Information:

Database	Instances	Threads
boston	1	1
london	1	1

Temporary Tablespace File Information:

boston TEMP Files: 3
london TEMP Files: 3

Flashback Database Status:

boston: Off
london: Off

Data file Online Move in Progress:

boston: No
london: No

Standby Apply-Related Information:

Apply State: Running
Apply Lag: 0 seconds
Apply Delay: 0 minutes

Transport-Related Information:

Transport On: Yes
Gap Status: No Gap
Transport Lag: 0 seconds
Transport Status: Success

Log Files Cleared:

boston Standby Redo Log Files: Cleared
london Online Redo Log Files: Cleared

Current Log File Groups Configuration:

Thread #	Online Redo Log Groups (boston)	Standby Redo Log Groups (london)
1	3	2

Future Log File Groups Configuration:

Thread #	Online Redo Log Groups (london)	Standby Redo Log Groups (boston)
1	3	0

Current Configuration Log File Sizes:

Thread #	Smallest Online Redo Log File Size (boston)	Smallest Standby Redo Log File Size (london)
1	50 MBytes	50 MBytes

Apply-Related Property Settings:

Property	boston Value	london Value
DelayMins	0	0
ApplyParallel	AUTO	AUTO

Transport-Related Property Settings:

Property	boston Value	london Value
LogXptMode	ASync	ASync
Dependency	<empty>	<empty>
DelayMins	0	0
Binding	optional	optional
MaxFailure	0	0
MaxConnections	1	1
ReopenSecs	300	300
NetTimeout	30	30
RedoCompression	DISABLE	DISABLE
LogShipping	ON	ON

Automatic Diagnostic Repository Errors:

Error	boston	london
No logging operation	NO	NO
Control file corruptions	NO	NO
SRL Group Unavailable	NO	NO
System data file missing	NO	NO
System data file corrupted	NO	NO
System data file offline	NO	NO
User data file missing	NO	NO
User data file corrupted	NO	NO
User data file offline	NO	NO
Block Corruptions found	NO	NO

Performing a Switchover by Using DGMGRL

After verifying the conditions required for a switchover, execute the SWITCHOVER command:

```
DGMGRL> SWITCHOVER TO 'london';
Performing switchover NOW, please wait...
Operation requires a connection to instance "london" on
database "london"
Connecting to instance "london"...
Connected as SYSDBA.
New primary database "london" is opening...
Operation requires startup of instance "boston" on
database "boston"
Starting instance "boston"...
ORACLE instance started.
Database mounted.
Switchover succeeded, new primary is "london"
```



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

After verifying the conditions required for executing a switchover, execute the SWITCHOVER command to perform the switchover operation. The Data Guard broker performs the following operations:

1. Verifies that the primary database and target standby database are in the correct states
2. Shuts down any instances as necessary
3. Switches roles between the primary and standby databases
4. Updates the broker configuration file with the new role information
5. Restarts the new standby database (former primary) with Redo Apply running
6. Opens the new primary database in read/write mode and starts redo transport services

Note: For detailed information about each step, see *Oracle Data Guard Broker*.

Preparing for a Switchover Using SQL

To prepare for the SQL switchover operation, verify that:

- Each database is properly configured for the role it is about to assume.
- There are no redo transport errors or gaps at the standby database by querying `V$ARCHIVE_DEST_STATUS`
- Temporary files exist on the standby database to match the temporary files on the primary database
- Delays in applying redo on the standby database that will become the new primary are removed
- Active Data Guard is stopped for the fastest possible role transition
- Any user sessions connect to the physical standby are cleanly terminated prior to switchover

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Before you execute the `SWITCHOVER` command, verify that standby redo log files are configured on the primary database. Standby redo log configuration should be identical on the primary database and on any physical standby databases. Even though the standby redo logs are not used when the database is in the primary role, configuring the standby redo logs on the primary database is recommended in preparation for an eventual switchover operation.

For switchovers involving a physical standby database, verify that the primary database is open and that Redo Apply is active on the standby database. For switchovers involving a logical standby database, verify both the primary and standby database instances are open and that SQL Apply is active.

The `VALID_FOR` clause of the `LOG_ARCHIVE_DEST_n` parameters can be configured for both the primary database role and the standby database role. By creating separate destinations in advance for each role, the parameters would not need to be adjusted at the time of switchover.

Performing a Switchover by Using SQL

1. Verify that the target standby database is ready:

```
SQL> ALTER DATABASE SWITCHOVER TO 'london' VERIFY;  
Database altered.
```

2. Initiate the switchover on the primary database:

```
SQL> ALTER DATABASE SWITCHOVER TO 'london';  
Database altered.
```

3. Open the new primary database.
4. Mount the new physical standby database.
5. Start Redo Apply on the new physical standby database.

```
SQL> ALTER DATABASE RECOVER MANAGED STANDBY DATABASE  
DISCONNECT;  
Database altered.
```



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When not using the Data Guard broker, a switchover will require multiple commands to be issued. Each database involved in the switchover will have to be started appropriately (`MOUNT` for standby database, `OPEN READ WRITE` for primary databases) by making connections to each instance separately. If the parameter file or server parameter file was not created considering that a role change would take place, the parameters may need to be adjusted accordingly.

Considerations When Performing a Switchover to a Logical Standby Database

- The switchover operation does not cause a shutdown of the primary database instance.
- Although there is no need to terminate user sessions, termination is recommended.
- The logical standby database may not have all data.
- Switchover to a logical standby database invalidates and disables all physical and snapshot standby databases in the broker-managed Data Guard configuration.
- Other logical standbys in the configuration will become standbys of the new primary.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

- Unlike a switchover to a physical standby database, a switchover to a logical standby database does not require a shutdown of the primary database.
- If you are switching over to a logical standby database, you do not need to terminate applications that are connected to the current primary database or to the logical standby database, because neither database is shut down during the switchover operation. However, because sessions on the old primary database may fail after the switchover operation is completed and the database guard is turned on, you should terminate such open sessions now. The database guard prevents users from making changes in the logical standby database.
- If you switch over to a logical standby database, be aware that you may not have all of the data that was in your primary database if the logical standby database is only a subset of the primary database.
- A switchover to a logical standby database invalidates and disables all physical and snapshot standby databases in the configuration. You must re-create the physical standby databases from a backup of the new primary database before you can reenableView them.

Situations That Prevent a Switchover

You cannot perform a switchover if:

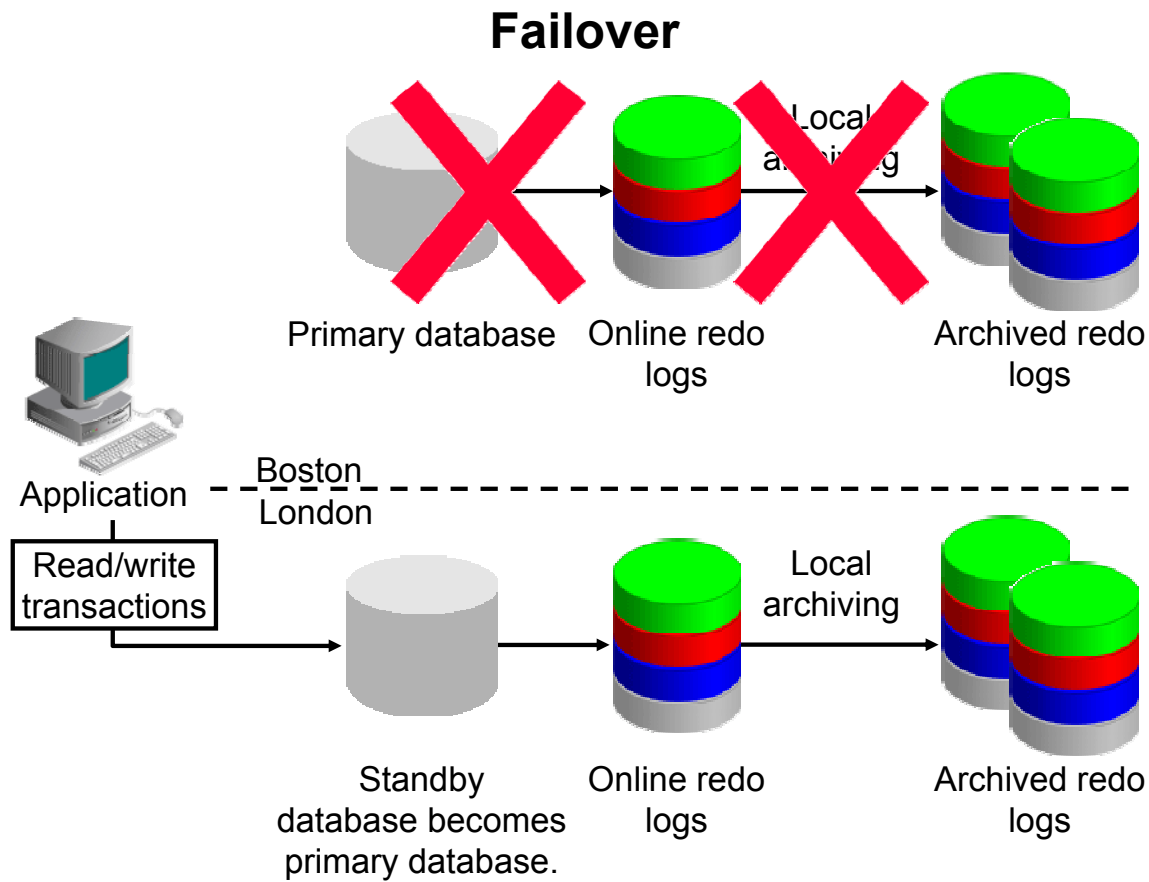
- Archived redo log files are unavailable
- Point-in-time recovery is required
- The primary database is not open and cannot be opened

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You cannot perform a switchover if:

- **Archived redo log files are unavailable:** If there is a gap in the archived redo log files on the standby database, you cannot switch over to that standby database.
- **Point-in-time recovery is required:** When you perform a switchover to a standby database, you always switch over to the current state of the primary database. You cannot switch over to a time in the past.
- **The primary database is not open and cannot be opened:** A switchover is initiated on the primary database while it is in the open state. If you cannot open the primary database, a switchover is not possible.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You invoke a failover operation when a failure occurs on the primary database and there is no possibility of recovering the primary database in a timely manner. During a failover operation, the primary database is disabled from the Data Guard environment and a standby database assumes the primary database role. Failing over to a standby database is a one-way operation. You cannot “fallback” and return the database to its former role as a standby database. Because of this, you should invoke a failover operation only in an emergency.

It is not always necessary to fail over to the standby database. In some cases, recovery of the primary database may be faster. Most failures can be resolved at a primary database in a reasonable amount of time.

In a failover operation:

- The original primary database is presumed to be lost. (If you want, you can reinstate or re-create the original primary database as a new standby database. You may then initiate a switchover operation compared to a failover operation to return the databases to their original roles.)
- Standby databases that are online at the time of the failover operation, but are not involved in the role transition, do not need to be shut down and restarted unless they were ahead of the failover target due to differences in apply lag situations. In that case, they may need to be flashed back to the point the standby became a primary or re-created.

Types of Failovers

- Manual failover: Invoked by the DBA
 - Complete: Attempts to minimize data loss by applying all available redo on the standby database
 - Immediate: No additional data is applied on the standby database
- Fast-start failover: Invoked automatically by the Data Guard broker

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A manual failover can be invoked through Enterprise Manager, DGMGRL, or SQL. There are two types of manual failover:

- **Complete:** The maximum amount of redo data for the protection mode is recovered. In this type of failover, the broker avoids disabling any standby databases that are not the failover target. Complete failover is the default and recommended type of failover.
- **Immediate:** No additional redo data is applied to the standby database after the failover is invoked. This is the fastest type of failover. After an immediate failover, you must re-create or reinstate the original primary database and all standby databases that were not a target of the failover.

Note: You should always try to perform a complete failover. Perform an immediate failover only when a complete failover is unsuccessful. Depending on the destination attributes of redo transport services, a complete failover can take place without incurring any data loss, while an immediate failover usually results in the loss of data.

You can configure fast-start failover so that the broker automatically fails over to a chosen standby database in the event of the loss of the primary database. For details, see the lesson titled “Enabling Fast-Start Failover.”

Failover Considerations

- The old primary database is disabled from the Data Guard configuration.
- Data loss is possible.
- Failover should be used only in an emergency.
- When choosing a standby database to fail over to, you should:
 - Choose a physical standby database when possible
 - Choose the standby database that is most current

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

During a failover operation, a standby database transitions to the primary role and the old primary database is rendered unable to participate in the configuration. Depending on the protection mode under which the old primary database was operating before the failover, there may be some or no data loss during a failover.

A failover is typically used only when a primary database becomes incapacitated and there is no possibility of performing a switchover or successfully repairing the primary database in a reasonable amount of time.

The specific actions that are performed during a failover vary, depending on:

- Whether a logical or physical standby database is involved in the failover operation
- The state of the configuration at the time of the failover
- The specific SQL commands that are used to initiate the failover

Performing a Failover by Using Enterprise Manager

Data Guard
Page Refreshed August 29, 2013 6:41:25 AM UTC
View Data Real Time: 30 Second Refresh

Overview
Data Guard Status: Normal
Protection Mode: Maximum Performance
Fast-Start Failover: Disabled

Primary Database
Name: london.example.com
Host: host03
Data Guard Status: Normal
Current Log: 43
Properties: Edit

Standby Database Progress Summary
Transport lag is the time difference between the last update on the primary database and the last received redo on the standby database. Apply lag is the time difference between the last update on the primary database and the last applied redo on the standby database.

Standby Databases
Add Standby Database

Select	Name	Host	Data Guard Status	Role	Real-time Query	Last Received Log	Last Applied Log	Estimated Failover Time
☒	boston.example.com	host01	Normal	Physical Standby	Disabled	42	42	Not available
☐	london2.example.com	host03	Normal	Logical Standby	N/A	42	42	1.4 seconds

Failover

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Perform a failover only in the event of a software or system failure that results in the loss of the primary database. The failed primary database is disabled by the broker and must be reinstated or re-created. The standby database then assumes the primary database role.

To initiate a failover by using Enterprise Manager:

1. On the Data Guard page, select the standby database that you want to become the primary database.
2. Click Failover.

Performing a Failover by Using Enterprise Manager

Confirmation: Failover to boston.example.com
Are you sure you want to failover to boston.example.com?

A failover will cause the standby database to become the primary database. The failover operation cannot be cancelled.

Select Failover Option

☒ **Complete**
All available redo data will be applied on the standby database, thereby minimizing data loss. Oracle recommends this type of failover.

☐ **Immediate**
No additional redo data will be applied on the standby database; data may be lost. This is the fastest type of failover.

Monitoring Settings

Monitoring settings can optionally be swapped between the primary and standby databases as a part of the role change operation.

☐ **Swap Monitoring Settings**
The current Enterprise Manager monitoring settings (including metric thresholds) for the primary and standby databases will be swapped after the role change, overriding all settings for each database with the values from the other database. If more granular monitoring standard swapping is desired, de-select this option and use the Monitoring Standards interface to create monitoring templates prior to the role change and apply them afterwards.

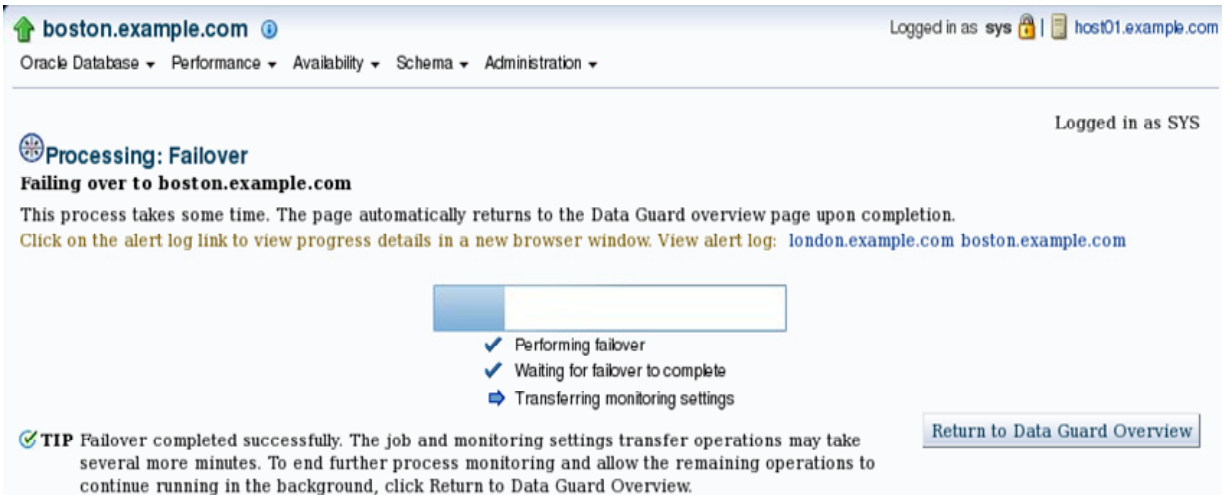
No Yes

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

3. On the Data Guard Failover Confirmation page, specify the type of failover that you want to perform:
 - **Complete:** All available redo is applied on the standby database. Oracle Corporation recommends that you specify this type of failover.
 - **Immediate:** No additional data is applied on the standby database, resulting in a data-loss failover. An immediate failover should be used only when a complete failover is not possible.
4. Select the failover option, and click Yes to confirm the failover operation.

Performing a Failover by Using Enterprise Manager

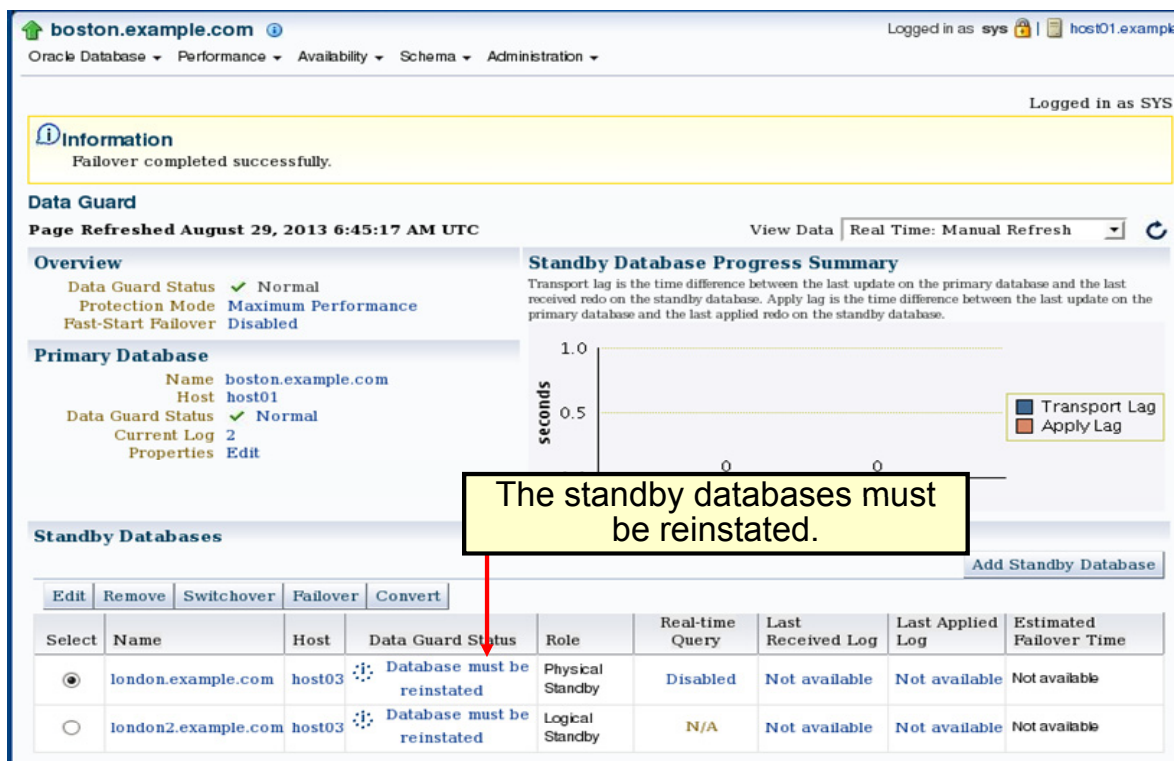
**ORACLE**

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

After you click Yes, the Failover Processing page displays the progress of the failover operation.

You cannot cancel this operation after it begins.

Performing a Failover to a Physical Standby Database



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

During the failover operation, the selected standby database transitions into the primary role. The failover target (a physical standby database) is restarted. When the operation finishes, the Data Guard Overview page reflects the updated configuration.

After a complete or immediate failover, the primary database and some standby databases may need to be reinstated or re-created. To reinstate the database, click the "Database must be reinstated" link. Then click Reinstall on the General Properties page. Enterprise Manager reinstates the database as a standby database to the new primary database.

Note: Flashback Database must have been enabled on the database prior to the failover, and the required flashback logs must be available on that database for reinstatement to succeed. In addition, the database to be reinstated and the new primary database must have network connectivity. Flashback database will be covered later in the course.

Performing a Failover to a Logical Standby Database

Oracle Enterprise Manager Cloud Control 12c

Setup Help SYSMAN Log Out

london2.example.com

Oracle Database Performance Availability Schema Administration

Logged in as SYS

Information
Failover completed successfully.

Data Guard
Page Refreshed October 8, 2013 8:29:12 PM UTC

View Data Real Time: Manual Refresh

Overview

Data Guard Status ✓ Normal
Protection Mode ✓ Maximum Performance
Fast-Start Failover Disabled

Primary Database

Name london2.example.com
Host host03
Data Guard Status ✓ Normal
Current Log 7
Properties Edit

Standby Database Progress Summary

Transport lag is the time difference between the last update on the primary database and the last received redo on the standby database. Apply lag is the time difference between the last update on the primary database and the last applied redo on the standby database.

The logical standby database is now the primary database. The former physical standby database is disabled.

Standby Databases

Select	Name	Host	Data Guard Status	Role	Real-time Query	Last Received Log	Last Applied Log	Estimated Failover Time
☒	boston.example.com	host01	Database must be reinstated	Logical Standby	N/A	Not available	Not available	Not available
☐	london	host03	Disabled	Physical Standby	Disabled	Not available	Not available	< 1 second

Performance
Data Guard Performance
Log File Details

Additional Administration
Verify Configuration
Remove Data Guard Configuration

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When you perform a failover to a logical standby database, any physical standby databases in the configuration are permanently disabled after the failover and are no longer usable. These databases must be re-created from the new primary database.

Performing a Manual Failover by Using DGMGRL

1. Execute the `FAILOVER` command to initiate the failover operation to the standby database:

```
DGMGRL> FAILOVER TO 'london' [IMMEDIATE] ;
```

2. Reset the protection mode (if necessary).
3. Reinststate or re-create the former primary database to serve as a standby database in the configuration.
4. Reinststate or re-create other disabled standby databases in the configuration.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

After determining that the primary database cannot be recovered in a timely manner, execute the `FAILOVER` command to perform the failover operation.

If you specify the `IMMEDIATE` option, no attempt is made to apply any unapplied redo that has been received.

The Data Guard broker performs the following operations for a complete failover:

1. Verifies that the target standby database is enabled
2. Stops Redo Apply after all unapplied redo data is applied to the target standby database
3. Transitions the target standby database to the primary database role by:
 - Opening the new primary database in read/write mode
 - Determining whether any standby databases need to be reinstated or recreated
 - Starting redo transport services

For an immediate failover, the broker does not wait for all available redo data to be applied (as described in step 2). For details about each step, see *Oracle Data Guard Broker*.

Reenabling Disabled Databases by Using DGMGRL

- Disabled databases must be reinstated or re-created to reenable broker management.
- Reinstall a database using REINSTATE DATABASE:

```
DGMGRL> REINSTATE DATABASE boston;
```

- If you cannot reinstall a database, re-create it from a copy of the primary database and then reenable the database by using ENABLE DATABASE:

```
DGMGRL> ENABLE DATABASE boston;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

After a failover, you may need to reinstall or re-create databases in your configuration. If a database can be reinstated, the database has the following status after a complete failover:

ORA-16661: the standby database needs to be reinstated

Note: To reinstall a database, Flashback Database must have been enabled on the database prior to the failover and flashback logs must be available.

To reinstall the database:

1. Start the database instance and mount the database.
2. Invoke DGMGRL and connect to the new primary database.
3. Use the REINSTATE DATABASE DGMGRL command to reinstall the database.

If a database must be re-created, it has the following status after the failover:

ORA-16795: the standby database needs to be re-created

You must re-create the standby database from a copy of the primary database and then reenable it by using the ENABLE DATABASE DGMGRL command.

Quiz

The target for a failover operation can be a snapshot standby database.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: a

Quiz

After a failover operation, the old primary database is removed from the Data Guard configuration and must be re-created from a backup.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: b

Summary

In this lesson, you should have learned how to:

- Use DGMGRL to perform switchover and failover operations
- Use Enterprise Manager to perform switchover and failover operations

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 12: Overview

This practice covers performing a switchover using DGMGRL.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

13

Using Flashback Database in a Data Guard Configuration

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Objectives

After completing this lesson, you should be able to:

- Explain the advantages of using Flashback Database in a Data Guard configuration
- Configure Flashback Database

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Using Flashback Database in a Data Guard Configuration

- Flashback Database provides the following in a Data Guard configuration:
 - An alternative to restoring and recovering the primary database
 - A way to reinstate the primary database that was disabled as part of a failover to any standby database operation
 - An alternative to delaying the application of redo to protect against user errors or logical corruptions
- Flashback Database is used by the following features in a Data Guard configuration:
 - Fast-start failover
 - Snapshot standby

The Oracle logo, consisting of the word "ORACLE" in white, uppercase letters on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Flashback Database provides the following advantages in a Data Guard configuration:

- Provides an alternative to delaying the application of redo to protect against user errors or logical corruptions. By using Flashback Database in this context, standby databases are more closely synchronized with the primary database, thereby reducing failover and switchover times.
- Eliminates the need to completely re-create the original primary database after a failover. The failed primary database can be flashed back to a point in time before the failover and converted to be a standby database for the new primary database.

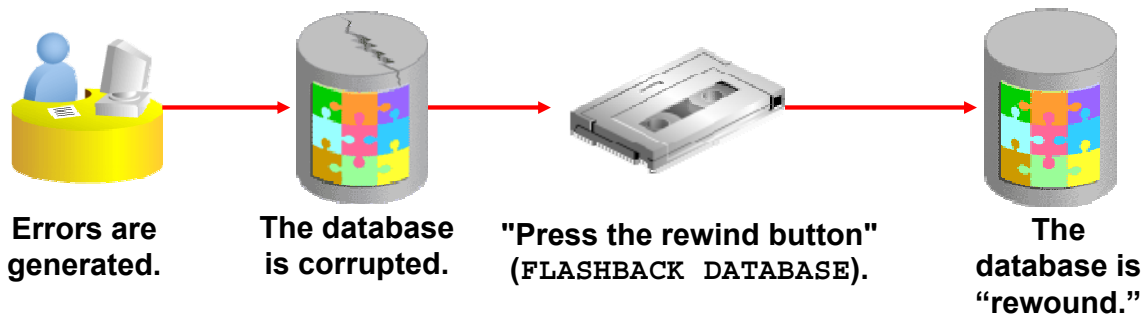
Flashback Database is used in a Data Guard configuration for the following features:

- **Fast-start failover:** You must enable Flashback Database and set up a fast recovery area on the primary database and the target standby database before enabling fast-start failover. (See the lesson entitled “Enabling Fast-Start Failover” for additional information.)
- **Snapshot standby:** To convert a physical standby database to a snapshot standby database, you must configure the Fast Recovery Area and size. If Flashback Database is not enabled, it will be enabled when the physical standby database is converted to a snapshot standby database. (See the lesson titled “Creating and Managing a Snapshot Standby Database” for additional information.)

Overview of Flashback Database

The Flashback Database operation:

- Works like a “rewind” button for the database
- Can be used when users make logical data corruptions



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

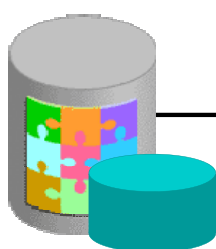
With Flashback Database, you can quickly bring your database to an earlier point in time by undoing all changes that took place since that time. This operation is fast because you do not need to restore backups. You can use this feature to undo changes that resulted in logical data corruptions.

When you use Flashback Database, Oracle Database uses past block images to back out changes to the database. During normal database operation, Oracle Database occasionally logs these block images in flashback logs, which are written sequentially and are not archived. Oracle Database automatically creates, deletes, and resizes flashback logs in the fast recovery area. You need to be aware of flashback logs only for monitoring performance and deciding how much disk space to allocate to the fast recovery area for flashback logs.

The time it takes to rewind a database with Flashback Database is proportional to how far back in time you need to go and the amount of database activity after the target time. The time it would take to restore and recover the whole database could be much longer. The before images in the flashback logs are used only to restore the database to a point in the past, and forward recovery is used to bring the database to a consistent state at some time in the past. Oracle Database returns data files to the previous point in time—but not auxiliary files such as initialization parameter files.

Flashback Database can also be used to complement Data Guard and Recovery Advisor and to synchronize cloned databases.

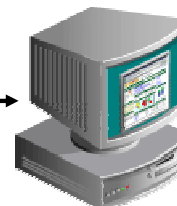
Configuring Flashback Database



1. Configure the fast recovery area.



2. Set the retention target.



3. Enable Flashback Database.

```
SQL> ALTER SYSTEM SET DB_FLASHBACK_RETENTION_TARGET=2880
SCOPE=BOTH;

SQL> ALTER DATABASE FLASHBACK ON;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The database can be open when you enable flashback. To enable flashback:

1. Configure the fast recovery area.
2. Set the retention target with the `DB_FLASHBACK_RETENTION_TARGET` initialization parameter.
You can specify an upper limit, in minutes, on how far back you want to be able to flashback the database. The example uses 2,880 minutes, which is equivalent to two days. This parameter is only a target and does not provide any guarantee. Your flashback time interval depends on how much flashback data was kept in the fast recovery area.
3. Enable Flashback Database with the following command:

```
ALTER DATABASE FLASHBACK ON;
```

Before you can issue the command to enable Flashback Database, the database must be configured for archiving.

Determine whether Flashback Database is enabled with the following query:

```
SELECT flashback_on FROM v$database;
```

Disable Flashback Database with the `ALTER DATABASE FLASHBACK OFF` command. As a result, all existing Flashback Database logs are deleted automatically.

Configuring Flashback Database by Using Enterprise Manager

Verify that the database is in ARCHIVELOG mode:

Media Recovery

The database is currently in ARCHIVELOG mode. In ARCHIVELOG mode, hot backups and recovery to the latest time are possible, but you must provide space for archived redo log files. If you change the database to ARCHIVELOG mode, you should perform a backup immediately. In NOARCHIVELOG mode, only cold backups are possible and data may be lost in the event of database corruption.

☒ ARCHIVELOG Mode*

Log Archive Filename Format*

Number	Archived Redo Log Destination	Status	Type
1	USE_DB_RECOVERY_FILE_DEST	VALID	Local
2	(DESCRIPTION=(ADDRESS=(PROTOCOL=tcp)(HOST=host01.example.co	VALID	Remote
3	(DESCRIPTION=(ADDRESS=(PROTOCOL=tcp)(HOST=host03.example.co	VALID	Remote

[Add Another Row](#)

TIP It is recommended that archived redo log files be written to multiple locations spread across the different disks.

TIP You can specify up to 10 archived redo log destinations.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

1. From the database home page, select Availability > Backup and Recovery > Recovery Settings from the menu.
2. Ensure that your database is in ARCHIVELOG mode. If not, select ARCHIVELOG mode.

Configuring Flashback Database by Using Enterprise Manager

Set the flash recovery area and enable Flashback Database:

Fast Recovery

This database is using a fast recovery area. The chart shows space used by each file type that is not reclaimable by Oracle. Performing backups to tertiary storage is one way to make space reclaimable. Usable Fast Recovery Area includes free and reclaimable space.

Fast Recovery Area Location

Fast Recovery Area Size GB

Fast Recovery Area Size must be set when the location is set.

Non-reclaimable Fast Recovery Area (GB) 4.97

Reclaimable Fast Recovery Area (GB) 0

Free Fast Recovery Area (GB) 5.03

☒ **Enable Flashback Database**

Flashback database can be used for fast database point-in-time recovery, as it returns the database to a prior point-in-time without restoring files. Flashback is the preferred point-in-time recovery method in the recovery wizard when appropriate. The fast recovery area must be set to enable flashback database.

Flashback Retention Time Hours

Current size of the flashback logs(MB) 100

Lowest SCN in the flashback data 6992026

Flashback Time Oct 17, 2013 8:13:56 PM

☐ Apply initialization parameter changes to SPFILE only. If not checked, parameter changes will be made to both the SPFILE and the running instance.

* Changes to this setting or parameter require a database restart.

Fast Recovery Area Usage

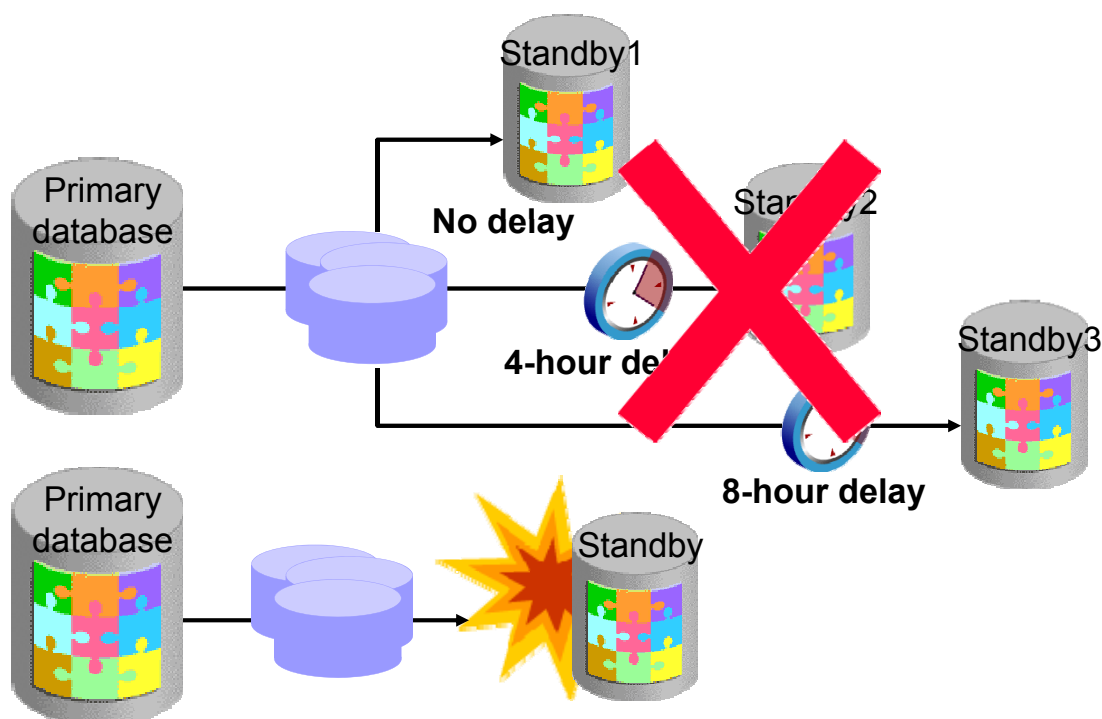
File Type	Size (GB)	Percentage (%)
Archived Redo Log	4.77	47.7%
Backup Piece	0.1	1%
Flashback Log	0.1	1%
Control File	0	0%
Redo Log	0	0%
Image Copy	0	0%
Auxiliary Datafile Copy	0	0%
Usable	5.03	50.3%

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

- Review the fast recovery area location.
The fast recovery area is a unified storage location for all recovery-related files and activities in an Oracle database. All files that are needed to completely recover a database from a media failure are part of the fast recovery area. The recovery-related files that can be created in the fast recovery area include archived redo log files, control files, backups created by Recovery Manager (RMAN), flashback logs, and the change tracking file. By allocating a storage location and unifying recovery-related files in a specific area, the Oracle database server relieves the database administrator from having to manage the disk files created by these components. If you want it in a different location, specify the location in the Fast Recovery Area Location field.
- Set the fast recovery size in the Fast Recovery Area Size field.
- Select Enable Flashback Database. You can also set the flashback retention time, and you can view important information regarding your flashback database window.
- Scroll down to the bottom of the Recovery Settings page, and click Apply.

Using Flashback Database Instead of Apply Delay



ORACLE

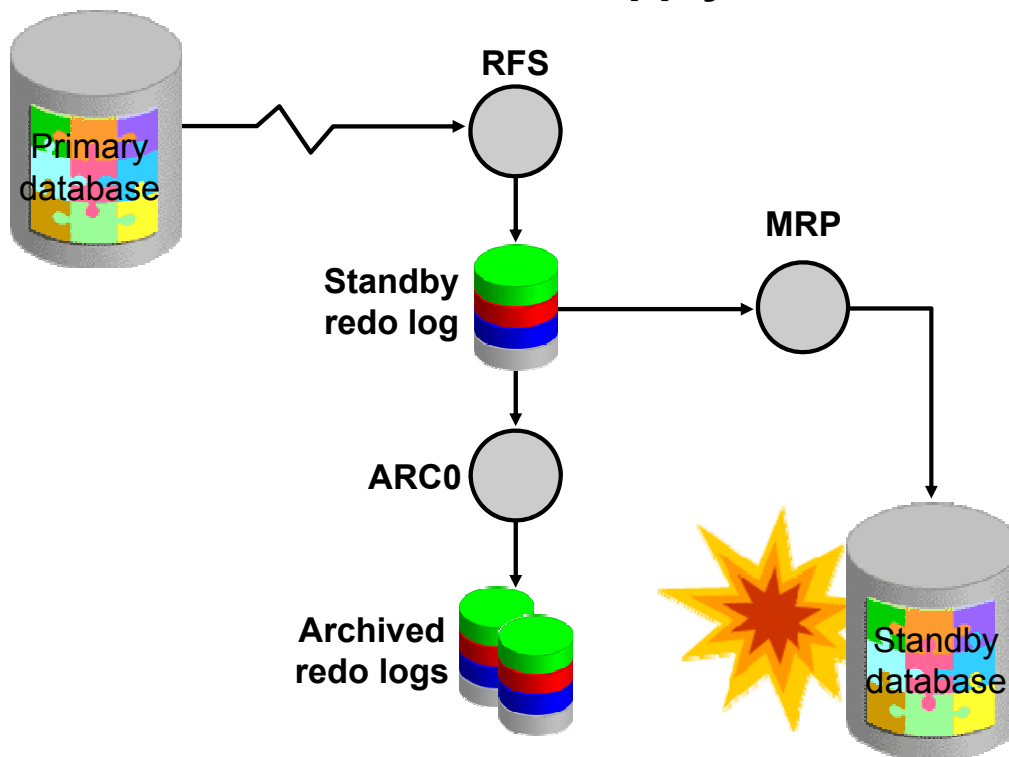
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

As an alternative to the Apply Delay configuration option, you can use Flashback Database to protect against the application of corrupted or erroneous data to the standby database. The Apply Delay configuration uses the `DELAY` attribute of the `LOG_ARCHIVE_DEST_n` parameter so that redo data from the primary database is received on the standby database, but redo apply delays its application. This provides a window of time so that logical errors, such as an accidental row deletion that is committed on the primary database, can have the data exported from the standby database before the SQL command executes on the standby database.

Flashback Database can quickly and easily flash back a standby database to an arbitrary time in the past. You can configure one standby database with Flashback Database to achieve the same benefit as multiple standby databases with different delays.

Note: Apply Delay will be discussed in the lesson titled "Optimizing a Data Guard Configuration".

Using Flashback Database and Real-Time Apply



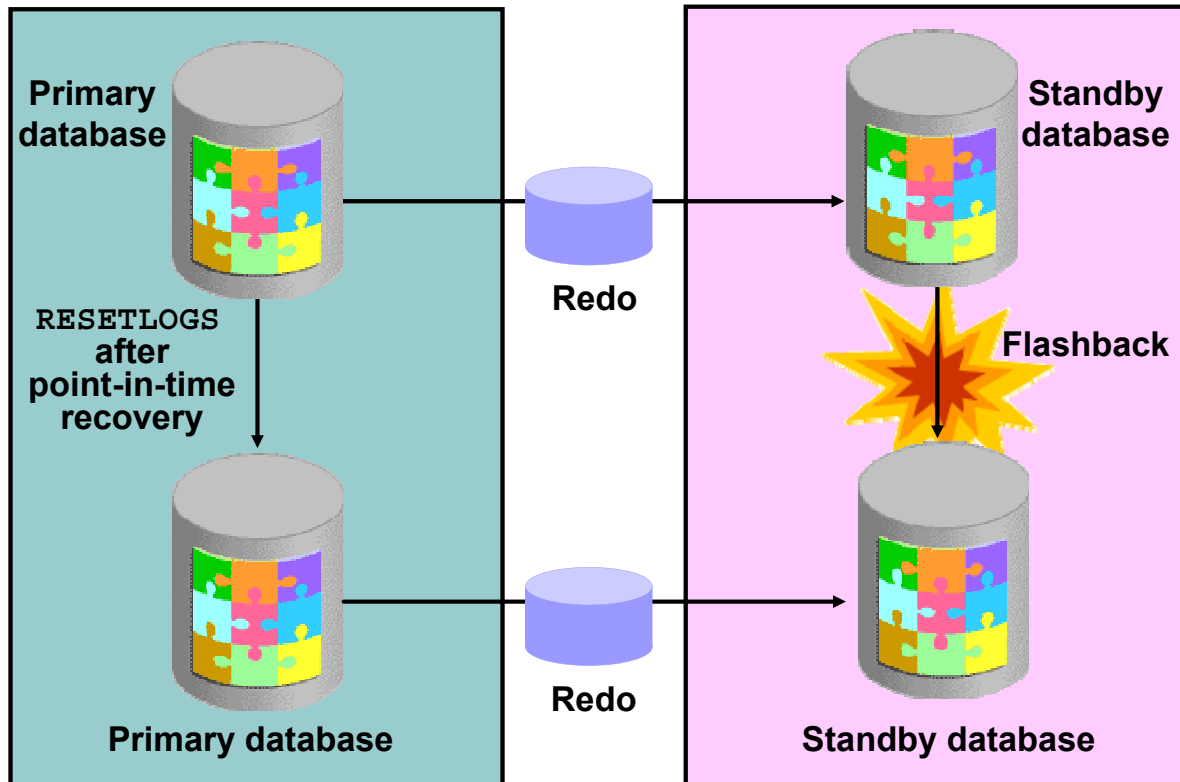
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Oracle Data Guard real-time apply feature reduces failover time. Flashback Database protects a physical standby database from logical data corruption and user error.

The Data Guard broker automatically enables real-time apply.

Using Flashback Database After RESETLOGS



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Physical Standby Configuration

For a physical standby database, it is possible that the Redo Apply service might not halt when it encounters the `OPEN RESETLOGS` command in the redo information. If the physical standby database's SCN (system change number) is far enough behind the primary database's SCN, the Redo Apply service can interpret the `OPEN RESETLOGS` command without stopping. This feature is known as "recovery through `RESETLOGS`."

Use the following procedure to avoid re-creating a standby database after you issue an `OPEN RESETLOGS` command on the primary database and the managed recovery process is halted on the physical standby database:

1. On the primary database, determine an SCN that is at least two SCNs prior to the SCN when the `OPEN RESETLOGS` command was issued. This is necessary to enable the standby to recover properly through `OPEN RESETLOGS`. Use the following query to find the "before `RESETLOGS`" SCN:

```
SELECT TO_CHAR(resetlogs_change# - 2) FROM v$database;
```

2. On the standby database, obtain the current SCN by using the following query:

```
SELECT TO_CHAR(current_scn) FROM v$database;
```
3. Flash back the standby database to the “before RESETLOGS” SCN that you queried in step 1:

```
FLASHBACK STANDBY DATABASE TO SCN <before RESETLOGS SCN>;
```
4. Restart managed recovery on the standby database. The standby database will be ready to receive and apply logs from the primary database.

```
ALTER DATABASE RECOVER MANAGED STANDBY DATABASE DISCONNECT;
```

Logical Standby Configuration

For a logical standby database, it is possible that the SQL Apply service might not halt when it encounters the `OPEN RESETLOGS` command in the redo information. If the logical standby database’s SCN is far enough behind the primary database’s SCN, the SQL Apply service will be able to interpret the `OPEN RESETLOGS` command without stopping.

Use the following procedure to avoid re-creating a standby database after you perform an `OPEN RESETLOGS` on the primary database and the SQL Apply process has halted on the logical standby database:

1. On the primary database, determine an SCN that is at least two SCNs prior to the SCN when the `OPEN RESETLOGS` command was issued. This is necessary to enable the standby to recover properly through `OPEN RESETLOGS`. Use the following query to find the “before RESETLOGS” SCN:

```
SELECT TO_CHAR(resetlogs_change# - 2) FROM v$database;
```
2. On the standby database, obtain the current SCN by using the following query:

```
SELECT TO_CHAR(current_scn) FROM v$database;
```
3. Flash back the standby database to the “before RESETLOGS” SCN that you queried in step 1:

```
FLASHBACK STANDBY DATABASE TO SCN <before RESETLOGS SCN>;
```
4. Restart SQL Apply on the standby database. The standby database will be ready to receive and apply logs from the primary database.

```
ALTER DATABASE START LOGICAL STANDBY APPLY IMMEDIATE;
```

Flashback Through Standby Database Role Transitions

- Use Flashback Database to flash back a database to a point in time before a switchover or failover.
- Primary and standby databases retain their current roles when you flash back through physical standby switchovers or failovers.
- Database roles are flashed back when you flash back through logical standby switchovers or failovers.
- Flashback Database can be used to undo a physical database activation.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

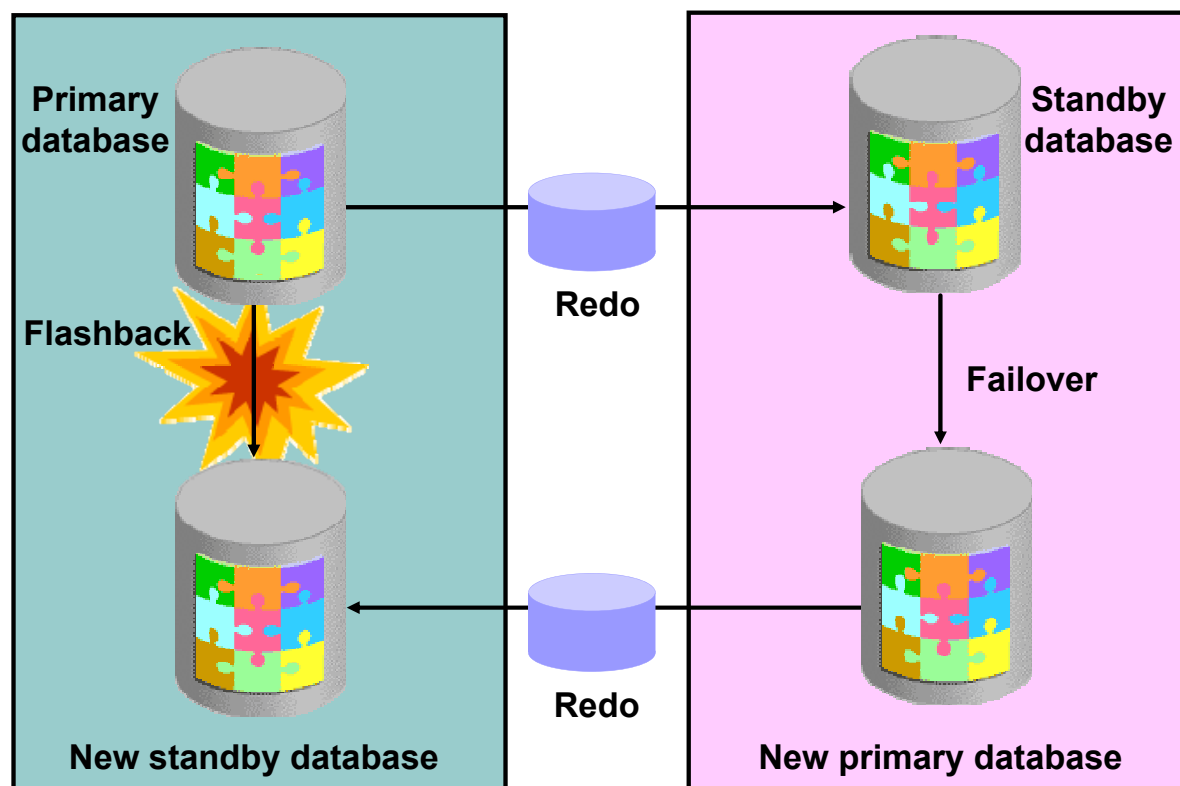
Use Flashback Database to revert a database to a point in time before a physical standby database switchover or failover. The database role does not change when flashing back through a physical standby switchover, failover, or activation.

In a logical standby database, the `FLASHBACK DATABASE` command reverts the database to a system change number (SCN) or time stamp before a switchover or failover. For a logical standby database, the database reverts to its original role at the time of the flashback SCN or flashback time.

You can also use Flashback Database to undo a physical standby activation.

Note: For detailed information about switchover and failover, see the lesson titled “Performing Role Transitions.”

Using Flashback Database After Failover



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You invoke a failover operation when a catastrophic failure occurs on the primary database and there is no possibility of recovering the primary database in a timely manner. After a failover operation, the old standby database becomes the new primary database, and the old primary database is placed in the “needs reinstatement” state.

Note: Detail on reinstating the database is provided in the lesson titled “Performing Role Transitions.”

Use the Flashback Database feature to convert the old primary database into a new standby database without re-creating the database. You can flash back the old primary database so that it contains only those changes that are already applied to the old standby database. This enables you to convert the old primary database into a new standby database without restoring the old primary database.

The `STANDBY_BECAME_PRIMARY_SCN` column in `V$DATABASE` can provide the SCN number corresponding to the failover. The old primary database should use the `FLASHBACK DATABASE TO SCN ...` command to rewind to this point. It can then be converted into a standby database with the SQL command `ALTER DATABASE CONVERT TO PHYSICAL STANDBY`.

Note: You must enable the Flashback Database feature on the old primary database prior to the failover.

Quiz

Flashback Database can be performed only on databases that are in the primary role.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: b

Summary

In this lesson, you should have learned how to:

- Enable Flashback Database
- Use Flashback Database effectively in a Data Guard configuration

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 13: Overview

This practice covers enabling Flashback Database:

- On the primary database
- On the standby database

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

14

Enabling Fast-Start Failover

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Objectives

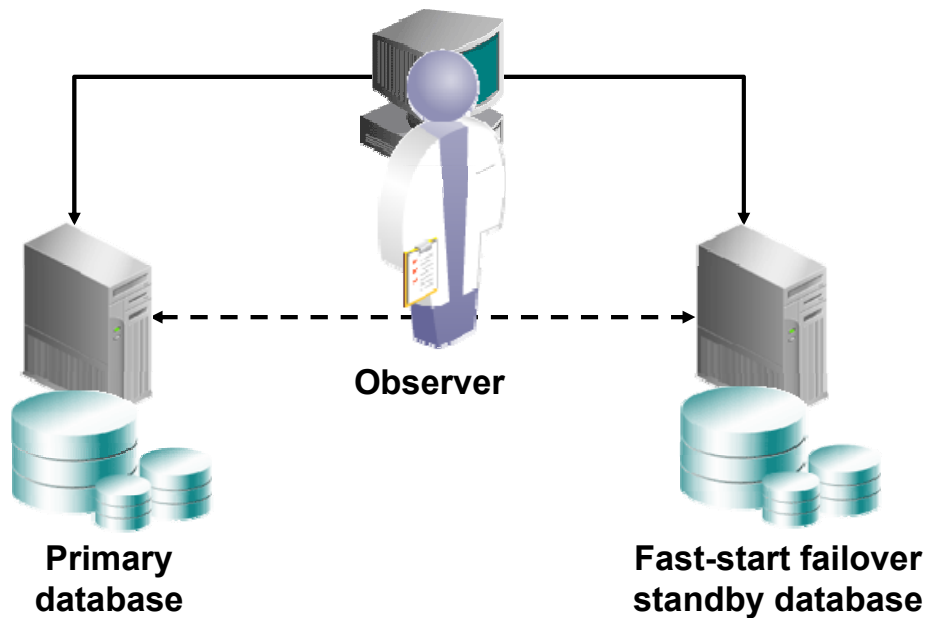
After completing this lesson, you should be able to:

- Configure fast-start failover
- View information about the fast-start failover configuration
- Manage the observer
- Perform role changes in a fast-start failover configuration
- Manually reinstate the primary database

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Fast-Start Failover: Overview



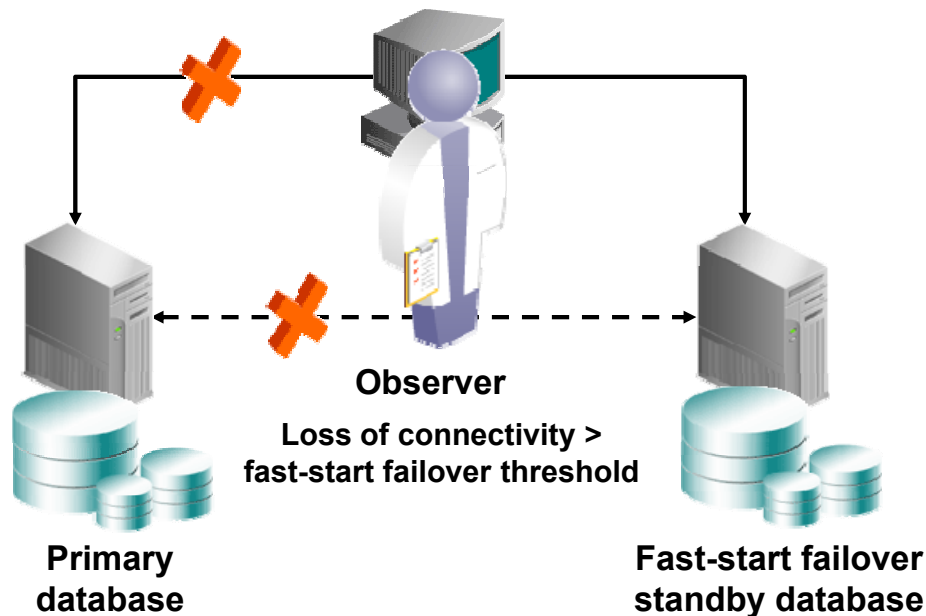
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Fast-start failover enables the Data Guard broker to rapidly and automatically fail over to a previously chosen standby database without requiring manual intervention. This feature increases the availability of your database in the event of a disaster by reducing the need for you to perform a failover operation manually. Fast-start failover can only be used in a broker configuration and can only be configured through DGMGRL or Enterprise Manager.

The observer is an Oracle Call Interface (OCI) client-side component that typically runs on a separate computer and monitors the availability of the primary database. It is integrated into the DGMGRL client-side component. You must use DGMGRL release 12.1 to observe Oracle Databases running release 12.1. There can be only one observer monitoring the broker configuration.

When Does Fast-Start Failover Occur?



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Fast-start failover occurs when any of the following conditions are met:

- Loss of connectivity between both the primary database and the observer—and between the primary database and the fast-start failover target standby database—exceeds the fast-start failover threshold.
- The database health-check mechanism determines any of the following (as optionally configured):
 - A data file is offline because of a write error.
 - Dictionary corruption of a critical database object occurs.
 - Control file is permanently damaged because of a disk failure.
 - LGWR is unable to write to any member of the log group because of an I/O error.
 - Archiver is unable to archive a redo log because the device is full or unavailable.
- An instance crash occurs for a single-instance database.
- All instances of a Real Application Clusters (RAC) primary database crash.
- Shutdown abort of the primary database occurs.
- An application initiates a fast-start failover by calling the `DBMS_DG.INITIATE_FS_FAILOVER` function.

Installing the Observer Software

- The observer is a separate OCI client-side component that monitors the availability of the primary database.
- Install observer software on a different computer from the primary and standby databases.
- Manage the observer by using Oracle Enterprise Manager or DGMGRL commands.



Observer

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To use fast-start failover, you should install the DGMGRL program (which contains the observer software) on a computer system that is separate from the primary database and standby database systems.

To install DGMGRL on the observer computer, use one of the following methods:

- Install the complete Oracle Client Administrator by choosing the Administrator option in the Oracle Universal Installer. This installation includes DGMGRL but it does not include the Oracle Enterprise Manager agent. This type of installation enables you to manage the observer by using DGMGRL commands but not Oracle Enterprise Manager.
- Install the full Oracle Database software kit. This installation includes DGMGRL.

Note: To manage the observer from Oracle Enterprise Manager Cloud Control, the Enterprise Manager Cloud Control agent will need to be installed on the observer machine in addition to DGMGRL.

The observer host should be located on the client network or the location that contains the most client activity or highest priority activity. This method could introduce a false failover, that is a fast-start failover occurs even though the primary is running and accessible locally; however, if the clients cannot get to the primary it may as well be down.

Fast-Start Failover Prerequisites

The following prerequisites must be met to enable fast-start failover:

- Configuration must be in maximum availability or maximum performance mode.
- LogXptMode property of target must be set as follows:
 - SYNC or FASTSYNC in maximum availability mode
 - ASYNC in maximum performance mode
- Flashback Database must be enabled on the primary database and the pre-selected target standby database.
- Configure `tnsnames.ora` entries for the observer.
- Create a static service name so that the observer can automatically restart databases.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

- The broker configuration must be operating in maximum availability or maximum performance mode.
- The primary database must be configured with standby redo log files.
- The standby database that is the target of fast-start failover must have its LogXptMode property set to SYNC or FASTSYNC to enable fast-start failover in maximum availability mode or to ASYNC to enable fast-start failover in maximum performance mode. To use a Far Sync instance with fast-start failover, the primary database must use the SYNC or FASTSYNC to the Far Sync instance. The Far Sync instance must use ASYNC to the target standby database. RedoRoutes property must be set if Far Sync is shipping the redo information to the fast-start failover target.
- Flashback Database must be enabled on the primary database and target standby database.
- The primary database and target standby database must have connectivity.
- Configure the `tnsnames.ora` file on the observer system so that the observer is able to connect to the primary database and the preselected target standby database.
- Create a static service name so that the observer can automatically restart a database as part of reinstatement.

Configuring Fast-Start Failover

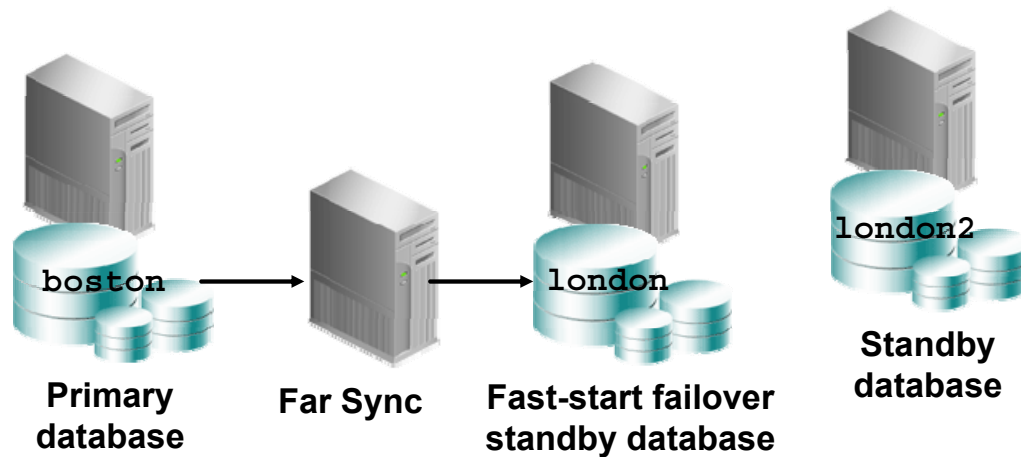
1. Specify the target standby database.
2. Set the protection mode.
3. Set the `FastStartFailoverThreshold` property.
4. Set additional database properties.
5. Set additional fast-start failover conditions.
6. Enable fast-start failover.
7. Start the observer.
8. Verify the configuration.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is centered within a solid red rectangular bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The manual steps to configure fast-start failover are described in detail on the next few pages. Configuring fast-start failover by using Enterprise Manager Cloud Control is described later in the lesson.

Step 1: Specify the Target Standby Database



```
DGMGRL> EDIT DATABASE boston
SET PROPERTY FastStartFailoverTarget = london;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You need not set the `FastStartFailoverTarget` property in a configuration with only one standby database: The Data Guard broker automatically sets it appropriately for the primary database and the standby database when fast-start failover is enabled.

In a Data Guard configuration with multiple standby databases, you must set the `FastStartFailoverTarget` database property on the current primary before enabling fast-start failover. The broker will reciprocally set the `FastStartFailoverTarget` database property on the standby database automatically when fast-start failover is enabled.

Execute the command when connected to the primary database or to any standby database in the configuration that has connectivity to the primary database. The command syntax is:

```
EDIT DATABASE [primary-database-name]
```

```
SET PROPERTY FastStartFailoverTarget = [standby-database-name];
```

To change the `FastStartFailoverTarget` property to point to a different standby database, disable fast-start failover, set the `FastStartFailoverTarget` property, and reenables fast-start failover.

Step 2: Set the Protection Mode

```
DGMGRL> EDIT DATABASE boston SET PROPERTY
  LogXptMode=SYNC;
DGMGRL> EDIT DATABASE london SET PROPERTY
  LogXptMode=SYNC;
DGMGRL> EDIT CONFIGURATION SET PROTECTION MODE AS
  MaxAvailability;
```

```
DGMGRL> EDIT DATABASE boston SET PROPERTY
  LogXptMode=ASync;
DGMGRL> EDIT DATABASE london SET PROPERTY
  LogXptMode=ASync;
DGMGRL> EDIT CONFIGURATION SET PROTECTION MODE AS
  MaxPerformance;
DGMGRL> EDIT CONFIGURATION SET PROPERTY
  FastStartFailoverLagLimit=45;
```



ORACLE

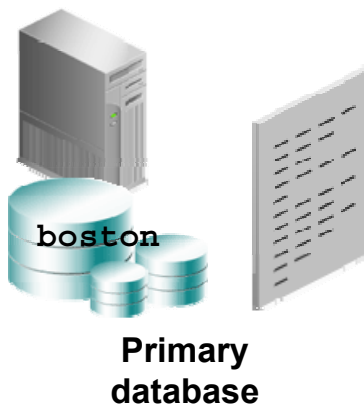
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can enable fast-start failover when the configuration is in maximum availability mode or maximum performance mode. For details about each mode, see the lesson titled “Configuring Data Protection Modes.”

In maximum performance mode, set the `FastStartFailoverLagLimit` configuration property to an acceptable limit (in seconds) by which the standby database is permitted to fall behind the primary database in terms of redo applied with automatic fast-start failover still being permitted. This is to limit the amount of data loss.

Note: The `RedoRoutes` property must be used instead of the `LogXptMode` property when Far Sync is shipping the redo to the fast-start failover target. Fast-start failover cannot be enabled when using maximum performance mode with a Far Sync.

Step 3: Set the Fast-Start Failover Threshold



```
DGMGRL> EDIT CONFIGURATION SET PROPERTY  
FastStartFailoverThreshold = 15;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The fast-start failover threshold specifies how long the observer and target standby database should simultaneously wait to hear from the primary database before initiating a fast-start failover. The threshold value is specified as a positive, nonzero number of seconds. The default value for the `FastStartFailoverThreshold` property is 30 seconds. When choosing a value for this property, you must balance the increased risk of an unnecessary failover (for example, if the network connection was temporarily broken for a few seconds) against the benefits of faster failover and less down time during a critical outage.

Recommended settings for the `FastStartFailoverThreshold` property:

- Single-instance primary, low-latency reliable network = 10–15 seconds
- Single-instance primary, high-latency network over WAN = 30–45 seconds
- RAC primary = (CSS miss count + reconfiguration time) + (24–40 seconds)

Execute the `EDIT CONFIGURATION SET PROPERTY FastStartFailoverThreshold = threshold-val` command when connected to the primary database or to any standby database in the Data Guard broker configuration with connectivity to the primary database.

Note: You can modify this property whether fast-start failover is enabled or disabled.

Step 4: (Optional) Set Additional Fast-Start Failover Properties

Additional fast-start failover properties can be set if desired. The optional properties include the following:

- `FastStartFailoverLagLimit`
- `FastStartFailoverPmyShutdown`
- `FastStartFailoverAutoReinstate`
- `ObserverConnectIdentifier`
- `ObserverOverride`
- `ObserverReconnect`

These parameters are covered in the following slides.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can set additional properties to control fast-start failover operation. If not explicitly specified, the optional failover properties will use default values. The default values are as follows:

Property	Default Value
<code>FastStartFailoverLagLimit</code>	30 seconds
<code>FastStartFailoverPmyShutdown</code>	True
<code>FastStartFailoverAutoReinstate</code>	True
<code>ObserverConnectIdentifier</code>	DGConnectIdentifier value
<code>ObserverOverride</code>	False
<code>ObserverReconnect</code>	0 (zero)

Setting the Lag-Time Limit

- Specifies a data loss threshold representing the maximum amount of data that can be lost with fast-start failover still being possible.
- Set the `FastStartFailoverLagLimit` property to configure a lag-time limit for a configuration in maximum performance mode:

```
DGMGRL> EDIT CONFIGURATION SET PROPERTY  
FastStartFailoverLagLimit = 30;
```

- Destinations that ship redo in `ASYNC` mode are acceptable fast-start failover target standby databases.
- Real-time apply must be enabled on the standby database.
- Fast-start failover cannot be enabled in maximum performance mode with a Far Sync.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In older version of the Oracle Database, fast-start failover could only be used with maximum availability configurations using `SYNC AFFIRM` redo transport settings. When Data Guard operates in the maximum performance mode, redo transport is done with `ASYNC` communication to the standby site. The primary database is allowed to advance transactions forward even if those transactions have not been received or written into the standby redo logs on the standby host. Therefore, a standby database could be missing transactions that have been performed on the primary database. A failover to the standby database at such a moment in time would result in a loss of data. The ability of fast-start failover to automatically fail over to a standby and experience data loss can be controlled by specifying an acceptable amount of data loss.

Use the `FastStartFailoverLagLimit` property to specify an acceptable lag-time limit in seconds. If the standby database's applied redo point is within the specified number of seconds of the primary database's redo generation point, a fast-start failover is allowed. If the standby database's applied point lags beyond this limit, a fast-start failover is not allowed. The lag limit is ignored for `SYNC` destinations. This is applicable only to configurations when fast-start failover is enabled in maximum performance modes.

Real-time apply must be enabled on the target standby database to ensure a valid calculation of the lag time.

Configure the `FastStartFailoverLagLimit` property as follows:

```
EDIT CONFIGURATION
```

```
SET PROPERTY FastStartFailoverLagLimit = {n};
```

The minimum value for n is 10. The default value is 30.

You can perform a manual failover to a maximum performance fast-start failover target. The maximum performance failover target must be within the specified lag limit of the primary database for the failover to be allowed. If the failover target is more than the specified lag limit behind the primary database, an error is returned because the standby database lags behind the primary by more than what is specified for the lag limit.

Configuring the Primary Database to Shut Down Automatically

- Use `FastStartFailoverPmyShutdown` to control whether the primary database shuts down if redo generation stalls and the primary database loses connectivity with the observer and target standby database for a longer time than the value of `FastStartFailoverThreshold`.
- Default value: TRUE

```
DGMGRL> EDIT CONFIGURATION SET PROPERTY
FastStartFailoverPmyShutdown = FALSE;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

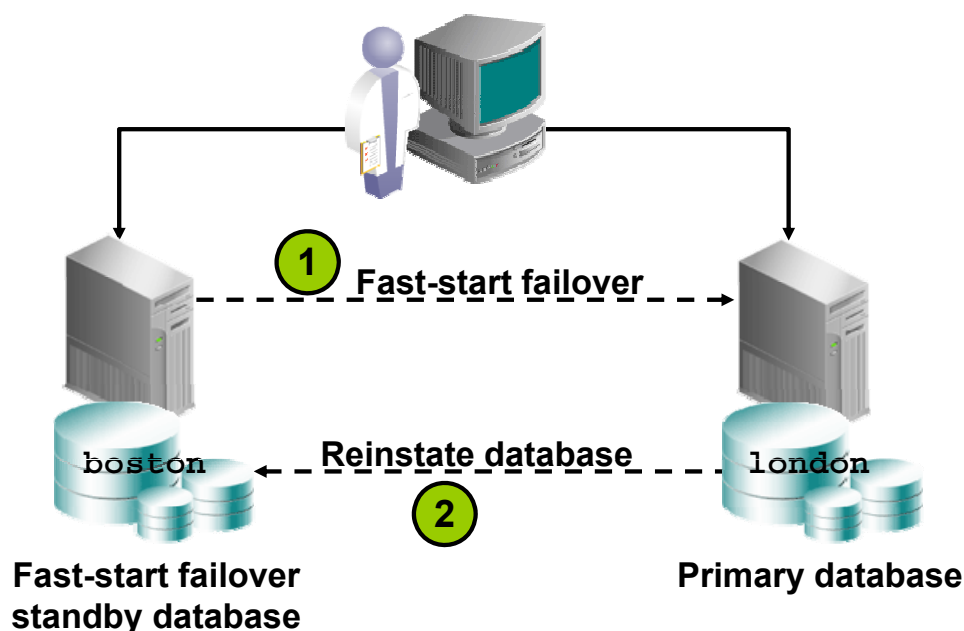
If you do not want the old primary database to shut down automatically after a fast-start failover is triggered due to the primary redo generation being stalled and the primary database losing connectivity with the observer and target standby database for longer than the number of seconds specified by the `FastStartFailoverThreshold` configuration property, set the `FastStartFailoverPmyShutdown` property to `FALSE`. When the property is set to `FALSE`, the old primary database stalls awaiting diagnosis of the condition that caused the fast-start failover.

```
DGMGRL> edit configuration set property
FastStartFailoverPmyShutdown = false;
Property "faststartfailoverpmyshutdown" updated
```

A value of `TRUE` causes the primary database to shut down with the `ABORT` option after the elapse of the number of seconds specified in the `FastStartFailoverThreshold` property and the primary database has no contact from its fast-start failover partners. `TRUE` is the default.

Note: The primary database is always shut down if a user configurable fast-start failover condition is detected or if an application initiated a fast-start failover by calling the `DBMS_DG.INITIATE_FS_FAILOVER` function even if `FastStartFailoverPmyShutdown` is set to `FALSE`.

Automatic Reinstatement After Fast-Start Failover



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Reinstatement of the original primary database is critical for reestablishing high availability after a fast-start failover. The original primary database can act as the fast-start failover target standby database for the new primary database after being reinstated as a standby database. Unless you reconfigure the fast-start failover environment, you cannot perform a subsequent fast-start failover until the original primary database is reinstated.

Automatic reinstatement of the original primary database is triggered when all of the following conditions are met:

- `FastStartFailoverAutoReinstate` is set to `TRUE`.
- The original primary database and the new primary database comprise the same fast-start failover configuration before the failover occurs and after the original primary database restarts.
- In a multi-standby database configuration, you have not performed a subsequent failover or switchover before the original primary database restarting.
- The observer can connect to the original primary database.
- The original primary database must be able to connect to the new primary database to complete the reinstatement operation.

If all conditions are not met, automatic reinstatement of the original primary database does not take place and appropriate errors are logged. You can then request manual reinstatement of the original primary database by using DGMGRL or Enterprise Manager (as described later in this lesson).

Configuring Automatic Reinstatement of the Primary Database

- Use the `FastStartFailoverAutoReinstat` property to determine whether the old primary database should be automatically reinstated if a fast-start failover was initiated because of a loss of connectivity.
- Default value: `TRUE`

```
DGMGRL> EDIT CONFIGURATION SET PROPERTY  
FastStartFailoverAutoReinstat = FALSE;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The `FastStartFailoverAutoReinstat` property enables you to determine whether the observer should automatically reinstate the old primary database after a fast-start failover. In some fast-start failover situations, you may want to diagnose the cause of the fast-start failover before reinstating the primary database; you should set the property to `FALSE`. The default value is `TRUE`.

```
DGMGRL> edit configuration set property  
FastStartFailoverAutoReinstat = false;  
Property "faststartfailoverautoreinstat" updated  
DGMGRL> show fast_start failover  
Fast-Start Failover: ENABLED  
Threshold:          90 seconds  
Target:             pc01sby1  
Observer:           host04  
Lag Limit:          60 seconds  
Shutdown Primary:  TRUE  
Auto-reinstat:      FALSE
```

Setting a Connect Identifier for the Observer

- Set the `ObserverConnectIdentifier` database property to specify how the observer should connect to and monitor the primary and standby database:

```
DGMGRL> EDIT DATABASE boston SET PROPERTY  
ObserverConnectIdentifier = 'boston.example.com';
```

- The default connect identifier is the value of the `DGConnectIdentifier` property.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered on a solid red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Use the `ObserverConnectIdentifier` configurable database property to specify how the observer should connect to and monitor the primary and standby database. Set this property for the primary and target standby database if you want the observer to use a different connect identifier than the one that is used to ship redo data (that is, the connect identifier specified by the `DGConnectIdentifier` property).

Setting an Observer Override

- Set the `ObserverOverride` property to allow an automatic failover to occur when the observer has lost connectivity to the primary, even if the standby can connect to the primary:

```
DGMGRL> EDIT CONFIGURATION SET PROPERTY  
ObserverOverride = TRUE;
```

- The default value of the `ObserverOverride` property is false.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is centered on a solid red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The `ObserverOverride` configuration property, when set to `TRUE`, allows an automatic failover to occur when the observer has lost connectivity to the primary, even if the standby has a healthy connection to the primary.

Setting Observer Reconnection Frequency

- Set the `ObserverReconnect` property to specify how often the observer establishes a new connection to the primary database:

```
DGMGRL> EDIT CONFIGURATION SET PROPERTY  
ObserverReconnect = 15;
```

- The default value of the `ObserverReconnect` property is zero, preventing periodic new connection attempts.
- New connection attempts are necessary to discover if the primary database is not reachable.
- The unit of measurement is seconds.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is centered on a solid red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The `ObserverReconnect` configuration property specifies how often the observer establishes a new connection to the primary database. When this property is set to the default value of 0, it prevents the observer from periodically establishing a new connection with the primary database. While this eliminates the processing overhead associated with periodically establishing a new observer connection to the primary database, it also prevents the observer from detecting that it is not possible to create new connections to the primary database. Oracle recommends that this property be set to a value that is small enough to allow timely detection of faults at the primary database, but large enough to limit the overhead associated with periodic observer connections to an acceptable level.

Step 5: Configure Additional Fast-Start Failover Conditions

- Use `ENABLE/DISABLE FAST_START FAILOVER` commands to specify conditions for fast-start failover:

```
ENABLE FAST_START FAILOVER CONDITION "Stuck Archiver";
```

- Observer initiates a fast-start failover without waiting for `FastStartFailoverThreshold` to expire.
- Configurable conditions:
 - Conditions detectable through database health check
 - Errors raised by the Oracle server
- Use the `SHOW FAST_START FAILOVER` command to obtain a list of valid conditions.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Use the `ENABLE/DISABLE FAST_START FAILOVER` broker commands to specify the conditions under which a fast-start failover should occur. The Oracle Database server detects when a specified condition occurs and informs the observer. The observer initiates a fast-start failover without waiting for `FastStartFailoverThreshold` to expire (if the standby is in a valid fast-start failover state to accept a failover).

Use the `SHOW FAST_START FAILOVER` command to obtain a list of valid conditions and confirm your changes. They include the following:

Configurable Failover Conditions

Health Conditions:

Corrupted Controlfile	YES
Corrupted Dictionary	YES
Inaccessible Logfile	NO
Stuck Archiver	NO
Datafile Offline	YES

Oracle Error Conditions

(none)

For specified Oracle ORA-Error conditions, the primary database will notify the observer if the error is signaled and the observer will immediately initiate a fast-start failover, assuming the standby is in a valid fast-start failover state (observed and either synchronized or within lag limits) to accept a failover. For example, if you experiencing a large amount of block corruptions (ORA-1578), you can specify the following syntax:

```
ENABLE FAST_START FAILOVER CONDITON '1578'
```

Fast-start failover will then occur when the ORA-1578 is encountered. This will not work for ORA-7445 or ORA-600 errors.

Configuring Fast-Start Failover Conditions

Specify additional conditions for a fast-start failover:

Health Condition	Default Value	Description
Datafile Offline	ENABLED	Data file offline due to a write error
Corrupted Controlfile	ENABLED	Corrupted control file
Corrupted Dictionary	ENABLED	Dictionary corruption of a critical database object
Inaccessible Logfile	DISABLED	LGWR unable to write to any member of a log group due to an I/O error
Stuck Archiver	DISABLED	Archiver unable to archive a redo log because the device is full or unavailable

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Additional database health conditions for a fast-start failover are displayed in the table. The conditions **Datafile Offline**, **Corrupted Controlfile**, and **Corrupted Dictionary** are enabled by default. An error is returned if the value you specify is not recognized.

If the condition was already set, no error is displayed.

Example: Enable a database health condition for a fast-start failover:

```
DGMGRL> enable fast_start failover condition "Inaccessible Logfile"
DGMGRL> show fast_start failover
```

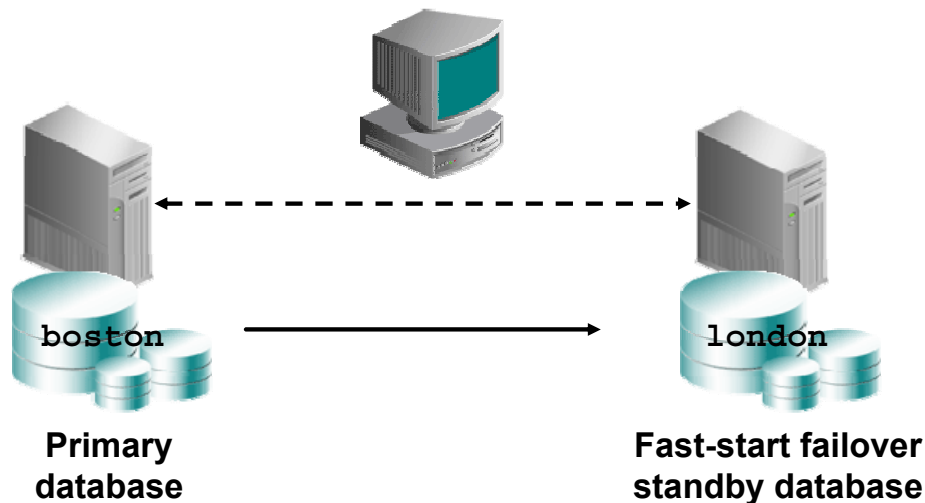
...

Configurable Failover Conditions

Health Conditions:

Corrupted Controlfile	YES
Corrupted Dictionary	YES
Inaccessible Logfile	YES
Stuck Archiver	NO
Datafile Offline	YES

Step 6: Enable Fast-Start Failover



```
DGMGRL> ENABLE FAST_START FAILOVER;
```

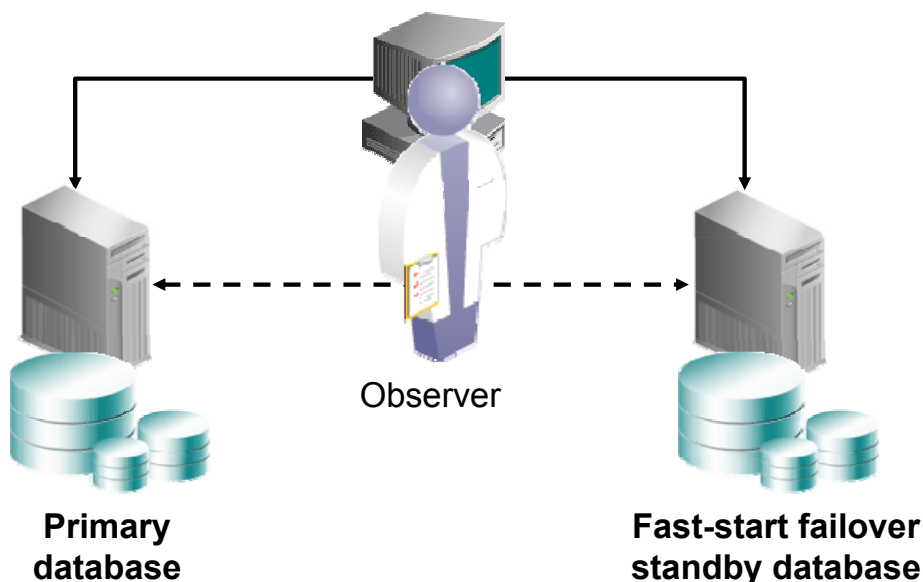
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can enable fast-start failover by using DGMGRL. When you issue the `ENABLE FAST_START FAILOVER` command, the observer is able to observe the primary database and the standby database and can initiate a fast-start failover when necessary.

In addition to enabling fast-start failover, you must start the observer (as described on the next page).

Step 7: Start the Observer



```
DGMGRL> START OBSERVER;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Execute the `START OBSERVER` command, which directs the broker to begin “observing” the Data Guard configuration to which DGMGRL is connected. You must execute this command on the computer where the observer will reside.

When you execute the `START OBSERVER` command, the observer retrieves the primary database and the fast-start failover target standby database connect identifiers and begins observation. It provides the name of its host computer to the Data Guard broker.

The primary database must be available for the `START OBSERVER` command to succeed. There can be only one observer for a Data Guard configuration at any one time. If you attempt to start a second observer for the configuration, you receive an error message.

If you disable fast-start failover while the observer is observing the configuration, the observer idles while waiting for fast-start failover to become enabled.

The observer maintains a small configuration file for persistently recording key information about the Data Guard configuration that it is observing. The file contains a description of the primary and target standby databases, including connect descriptors. Use the `FILE` qualifier with the `START OBSERVER` command to specify an explicit directory path and name for the configuration file on the observer computer:

```
DGMGRL> START OBSERVER FILE=$ORACLE_HOME/dbs/Boston.dat;
```

If you do not specify the `FILE` qualifier, the current working directory is searched for a file named `FSFO.dat`. If the file does not exist, a new file is created. If the configuration file exists, the observer checks whether the file describes a valid fast-start failover environment for the Data Guard configuration to which the observer is connected.

Note: You can execute the `START OBSERVER` command regardless of whether fast-start failover is enabled.

Control is not returned to you when the observer is successfully started. The observer is a continuous foreground process; as a result, the command-line prompt on the observer computer does not return until you issue the `STOP OBSERVER` command from another DGMGRL session. To issue commands and interact with the broker configuration, you must connect through another DGMGRL client session. If you stop the observer with the `STOP OBSERVER` command, the observer's DGMGRL process is terminated.

Step 8: Verify the Configuration

```
DGMGRL> show fast_start failover;  
Fast-Start Failover: ENABLED
```

```
Threshold:          30 seconds  
Target:             london  
Observer:           host04  
Lag Limit:          30 seconds  
Shutdown Primary:   TRUE  
Auto-reinstate:     TRUE  
Observer Reconnect: (none)  
Observer Override:  FALSE
```

Configurable Failover Conditions

Health Conditions:

Corrupted Controlfile	YES
Corrupted Dictionary	YES
Inaccessible Logfile	NO
Stuck Archiver	NO
Datafile Offline	YES

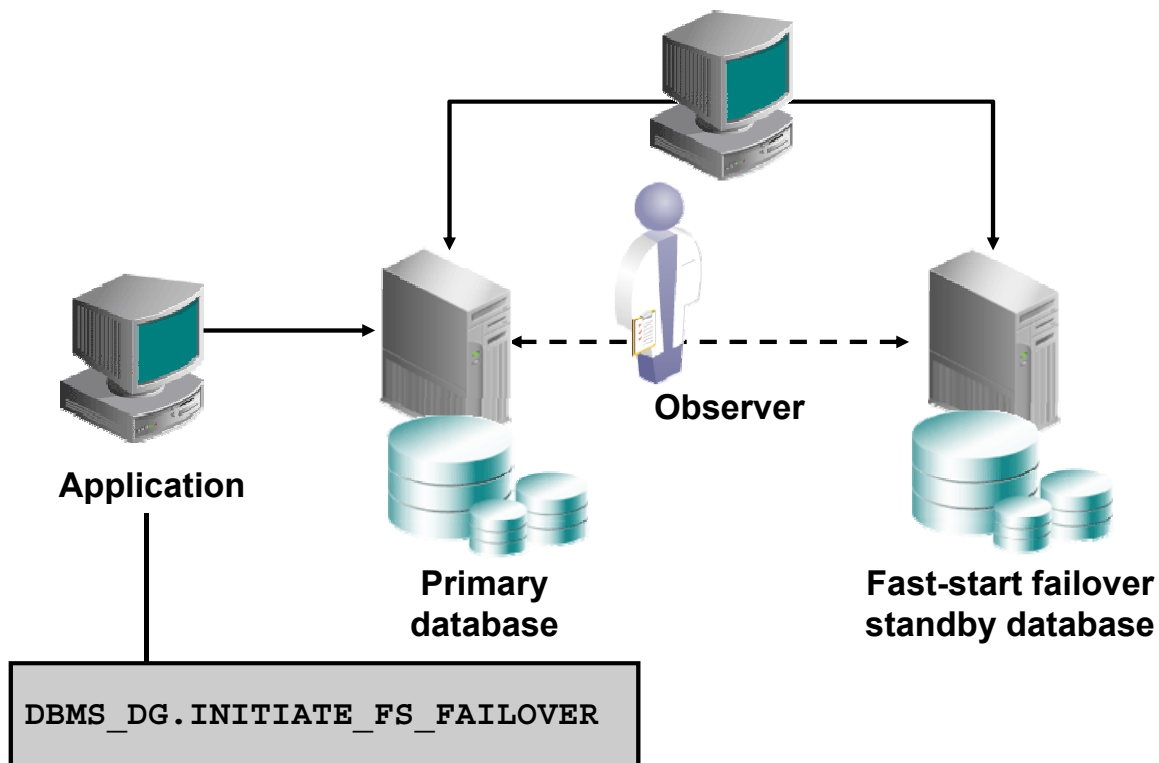
Oracle Error Conditions:
(none)



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can also use the `SHOW FAST_START FAILOVER` command to display all fast-start failover information.

Initiating Fast-Start Failover from an Application



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

An application can initiate a fast-start failover by invoking the `DBMS_DG.INITIALIZE_FS_FAILOVER` function. The function is used to alert the primary database server that the application wants a fast-start failover to occur immediately. The primary database server notifies the observer of this request, and the observer immediately initiates a fast-start failover. The standby database must be in a valid fast-start failover state—observed and synchronized or within the lag limit of the primary database—to accept a failover.

Initiating Fast-Start Failover from an Application

- DBMS_DG package contains the `INITIATE_FS_FAILOVER` function that is used to initiate a fast-start failover from an application:

```
FUNCTION dbms_dg.initiate_fs_failover  
  (condstr IN VARCHAR2) RETURN BINARY_INTEGER;
```

- The DBMS_DG package is defined as an invoker's rights package to address privilege concerns.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The DBMS_DG PL/SQL package contains the `INITIATE_FS_FAILOVER` function, which enables an application to request a fast-start failover.

The application-initiated failover is an invocation of the `FAILOVER` command and requires the `SYSDBA` privilege. The DBMS_DG package is defined as an invoker's rights package to address privilege concerns.

If the configuration is not in a valid fast-start failover state, the `INITIATE_FS_FAILOVER` function returns an `ORA` error, informing the caller that a fast-start failover operation cannot be performed.

Viewing Fast-Start Failover Information

```
SELECT fs_failover_status as STATUS,
       fs_failover_current_target as CURR_TGET,
       fs_failover_threshold as THRESHOLD,
       fs_failover_observer_present as OBS_PRES,
       fs_failover_observer_host as OBS_HOST
FROM v$database;
```

STATUS	CURR_TGET	THRESHOLD	OBS_PRES
TARGET UNDER LAG LIMIT	london	30	YES


```
OBS_HOST
-----
Host04.example.com
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

V\$DATABASE contains the following columns that provide detailed information about fast-start failover. They include:

- **FS_FAILOVER_STATUS:**
 - **BYSTANDER:** Fast-start failover is enabled, but this standby database is not the target of the fast-start failover.
 - **DISABLED:** Fast-start failover is disabled.
 - **LOADING DICTIONARY:** This status appears only on a logical standby database that has not completed loading a copy of the primary database data dictionary.
 - **PRIMARY UNOBSERVED:** This status occurs only on the target standby database when it is synchronized with or is TARGET UNDER LAG LIMIT of the primary database and has connectivity with the observer, but the primary database does not have a connection to the observer.
 - **REINSTATE FAILED:** Reinstatement of the failed primary database as a new standby database failed. See the Data Guard broker drc* log files for additional information.

- **REINSTATE REQUIRED:** The failed primary database requires reinstatement as a new standby database to the new primary database. The observer automatically starts the reinstatement process.
- **STALLED:** This status appears on the primary database if redo generation is stalled because the primary database cannot continue without violating the data loss guarantee.
- **SUSPENDED:** This status appears on the target standby database when either the primary database or the target standby database is shut down in a controlled fashion.
- **SYNCHRONIZED:** The primary database and the fast-start failover target standby database are synchronized and the configuration is operating in maximum availability mode.
- **TARGET OVER LAG LIMIT:** (Only for maximum performance mode) The standby database redo applied point lags behind the primary database redo generation point by more than the number of seconds specified by the `FastStartFailoverLagLimit` configuration property.
- **TARGET UNDER LAG LIMIT:** (Only for maximum performance mode) The standby database redo applied point does not lag the primary database redo generation point by more than the number of seconds specified by the `FastStartFailoverLagLimit` configuration property.
- **UNSYNCHRONIZED:** The target standby database does not have all the redo from the primary database and the configuration is operating in maximum availability mode. Fast-start failover is not possible.

Note: If the value for `FS_FAILOVER_STATUS` is `DISABLED`, the values for `FS_FAILOVER_CURRENT_TARGET`, `FS_FAILOVER_THRESHOLD`, `FS_FAILOVER_OBSERVER_PRESENT`, and `FS_FAILOVER_OBSERVER_HOST` columns in this table are not meaningful.

- **FS_FAILOVER_CURRENT_TARGET:** `DB_UNIQUE_NAME` of the standby database that is the current fast-start failover target standby database for the Data Guard configuration
- **FS_FAILOVER_THRESHOLD:** Time (in seconds) during which the observer attempts to reconnect with a disconnected primary database before attempting fast-start failover with the target standby database
- **FS_FAILOVER_OBSERVER_PRESENT:**
 - **YES:** Observer is connected to the local database.
 - **NO:** Observer is not connected to the local database.
- **FS_FAILOVER_OBSERVER_HOST:** Name of the machine that hosts the observer process

Determining the Reason for a Fast-Start Failover

Determine the reason for fast-start failover by querying the V\$FS_FAILOVER_STATS view:

```
SQL> SELECT last_failover_time, last_failover_reason  
        FROM v$fs_failover_stats;
```

```
LAST_FAILOVER_TIME    LAST_FAILOVER_REASON
```

```
-----
```

```
10/17/2013 22:30:12    Primary Disconnected
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can query the V\$FS_FAILOVER_STATS view on the primary database to learn why a fast-start failover operation took place. The view shows the time of the last fast-start failover and the reason for the operation.

Prohibited Operations After Enabling Fast-Start Failover

Many configuration changes are not allowed with fast-start failover enabled.



**Configuration protection
mode changes**



**Disable or delete
the Far Sync**



**Disable or delete
the fast-start failover
standby database**

```
DGMGRL> EDIT DATABASE SET PROPERTY LogXptMode;  
DGMGRL> EDIT DATABASE SET PROPERTY  
FastStartFailoverTarget;
```



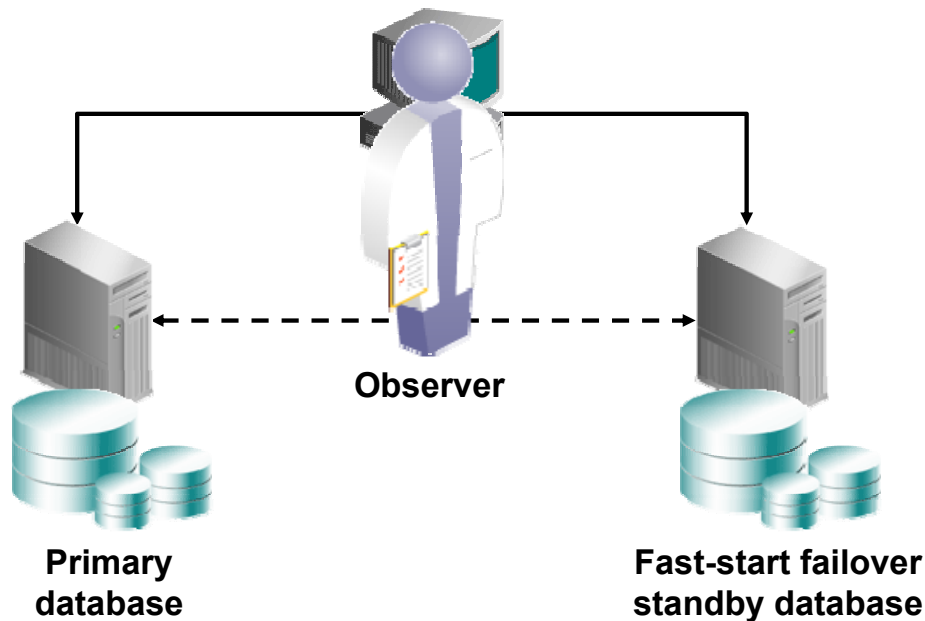
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

After enabling fast-start failover, you cannot perform any of the following operations (which undermine the fast-start failover environment):

- Change the configuration protection mode.
- Change the `LogXptMode` property for the primary database or target standby database.
- Change the `RedoRoutes` property on the primary, Far Sync, or target standby database.
- Perform a failover or switchover to a standby database that is not the fast-start failover target.
- Disable the Data Guard broker configuration.
- Delete the Data Guard broker configuration.
- Disable or delete the standby database that is the target of fast-start failover.
- Disable or delete the Far Sync instance if it is shipping redo to the target of fast-start failover.
- Alter the `FastStartFailoverTarget` database-level property of either the primary database or the standby database.
- Fail over to an unsynchronized fast-start failover target.

Disabling Fast-Start Failover



```
DGMGRL> DISABLE FAST_START FAILOVER [FORCE];
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can disable fast-start failover to prevent the observer from initiating a failover to the target standby database.

When you execute the `DISABLE FAST_START FAILOVER` command, the Data Guard broker disables fast-start failover on the target standby database and then disables fast-start failover on the primary database. The Data Guard broker records this change persistently in the Data Guard broker metadata and propagates the change to all standby databases in the Data Guard broker configuration.

If the primary database and the fast-start failover target standby database do not have connectivity with each other, you can use the `FORCE` option to disable fast-start failover. If this command is executed on the primary database or on a bystander (non-fast-start failover target) standby database that has connectivity with the primary database, the Data Guard broker disables fast-start failover locally. The Data Guard broker then records this change persistently in the Data Guard broker metadata and propagates the change to all databases in the Data Guard broker configuration to which the primary database has connectivity.

If the standby database does not have connectivity with the primary database when you execute the `DISABLE FAST_START FAILOVER` command with the `FORCE` option on the fast-start failover target standby database, fast-start failover is disabled only on that target standby database. If the target standby database regains connectivity with the primary database, the primary database disables fast-start failover (as previously described).

If the standby database does not have connectivity with the primary database when you execute the `DISABLE FAST_START FAILOVER` command with the `FORCE` option on a bystander standby database, it is ignored by the primary database. Fast-start failover is reenabled on the bystander standby database automatically when connectivity with the primary database is restored. You must issue the `DISABLE FAST_START FAILOVER` command on the primary database or on a bystander standby database that has connectivity with the primary database (or with the fast-start failover target standby database itself) if you want to permanently disable fast-start failover.

Note: Disabling fast-start failover does not stop the observer. Stopping the observer is discussed later in the lesson.

Disabling Fast-Start Failover Conditions

Remove specific conditions for which a fast-start failover should be performed:

```
DGMGRL> DISABLE FAST_START FAILOVER CONDITION "Datafile  
Offline";
```

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is centered on a solid red horizontal bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Use the `DISABLE FAST_START FAILOVER CONDITION` command to disable fast-start failover for specified conditions. (You specify conditions as described earlier in the lesson.)

Using the FORCE Option

Use the `FORCE` option in the following situations:

- When the fast-start failover environment is synchronized and the primary has lost connectivity to the observer and the target standby database
- To prevent a fast-start failover from occurring on the target standby database
- To conduct a manual failover when the fast-start failover environment is unsynchronized

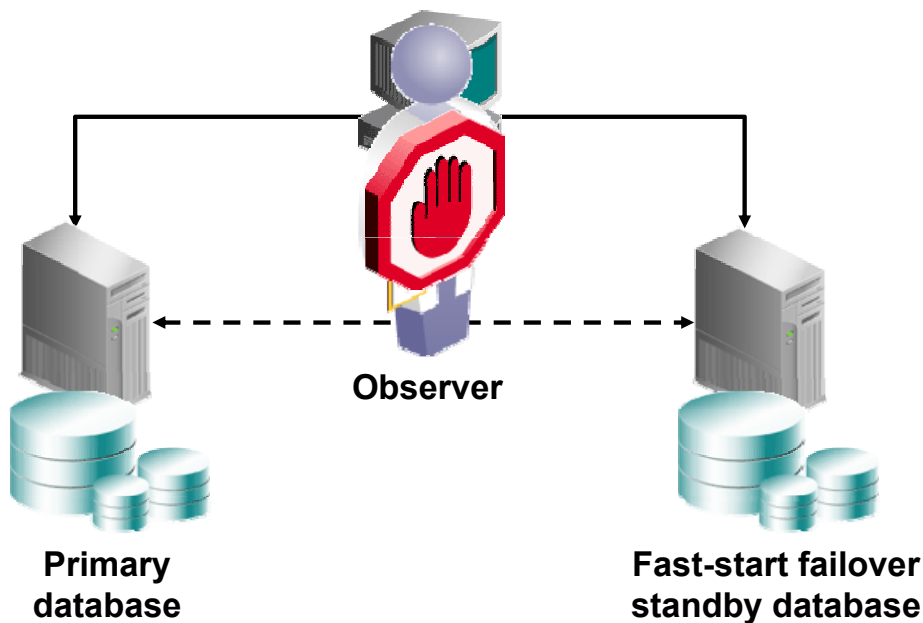
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Use the `FORCE` option when:

- You want to disable fast-start failover when the environment is synchronized and the primary database has lost connectivity with the observer and the target standby database. The `FORCE` option enables you to disable fast-start failover without requiring connectivity with the target standby database or the observer.
- You want to prevent a fast-start failover from occurring on the target standby database because you know that the primary database will resume service before the fast-start failover threshold expires
- You want to conduct a manual failover to either the target standby database or a bystander standby database when the fast-start failover environment is unsynchronized. In this case, you must be willing to accept a data-loss failover.

Stopping the Observer



```
DGMGRL> STOP OBSERVER;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

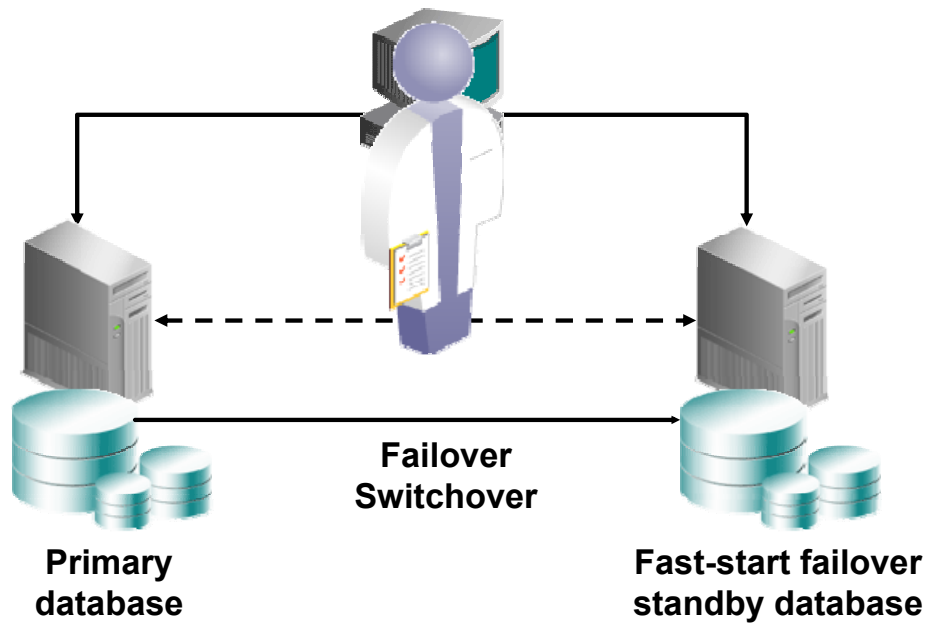
If you no longer want to use fast-start failover, or if you want to move the observer to a different host computer, use the `STOP OBSERVER` command to stop the observer.

Note: The `STOP OBSERVER` command does not disable fast-start failover. You can issue this command whether or not fast-start failover is enabled.

You must issue this command on the primary database or on a standby database in the configuration that has connectivity to the primary database. If you execute this command when fast-start failover is enabled, the fast-start failover target standby database must have connectivity to the primary database.

The observer does not immediately stop when you execute the `STOP OBSERVER` command. The broker informs the observer the next time it is contacted by the observer. After you execute the `STOP OBSERVER` command, the Data Guard broker can accept a new observer whether or not the stopped observer is terminated.

Performing Manual Role Changes



ORACLE

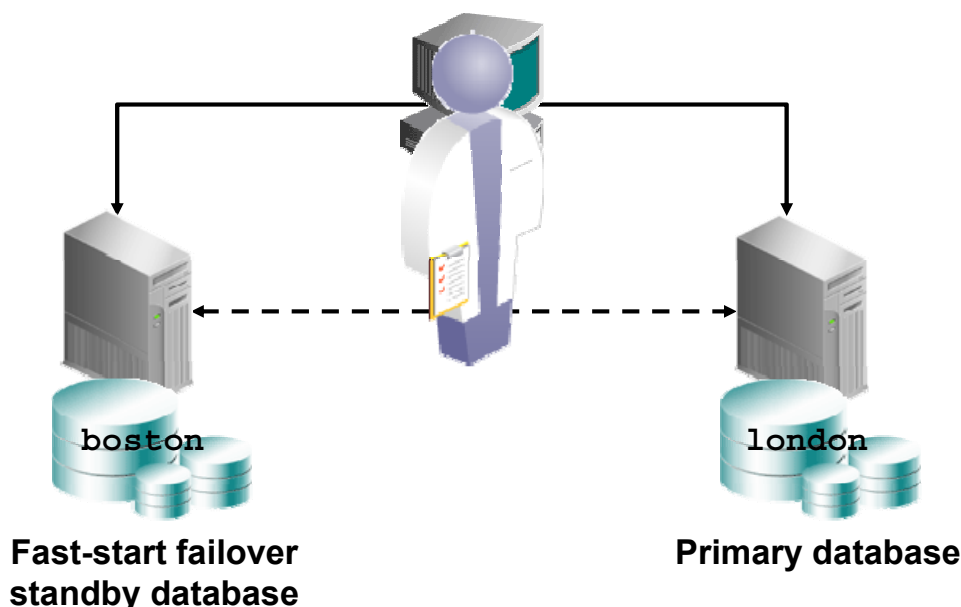
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can perform manual role changes in a fast-start failover configuration if the role change is directed to the fast-start failover target standby database and if the configuration is synchronized.

The following conditions must be met:

- If the configuration is operating in maximum availability mode, the target standby database must be synchronized with the primary database.
- If the configuration is operating in maximum performance mode, the target standby database must be within the specified lag limit of the primary database.
- For manual failover, the observer must be started and must be communicating with the target standby database.

Manually Reinstating the Database



```
DGMGRL> REINSTATE DATABASE boston;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Use the `REINSTATE DATABASE` command to reinstate the database. If the conditions for reinstatement are not satisfied, the request fails and the specified database remains disabled.

If the database name specified is the original primary database and fast-start failover is enabled, the original primary database is reinstated as a standby database to the new primary database. The fast-start failover configuration is updated to reflect the availability of the new standby database. It accepts archived redo log files from the new primary database and serves as the fast-start failover target if the new primary database fails.

Issue this command while connected to either the primary database or to a standby database in the configuration that has connectivity to the primary database other than the original primary database to be reinstated.

Note: The `REINSTATE DATABASE` command does not require fast-start failover to be enabled. It can also be used to reinstate an original primary database after a conventional no-data-loss failover.

Using Enterprise Manager to Enable Fast-Start Failover

Oracle Enterprise Manager Cloud Control 12c

Enterprise Targets Favorites History Search Target Name

boston.example.com Logged in as: sys | host01.example.com

Oracle Database Performance Availability Schema Administration

Data Guard Page Refreshed August 29, 2013 5:53:30 AM UTC View Data Real Time: Manual Refresh

Overview

- Data Guard Status: ✓ Normal
- Protection Mode: Maximum Performance
- Fast-Start Failover: Disabled

Primary Database

Name: boston.example.com
Host: host01
Data Guard Status: ✓ Normal
Current Log: 33
Properties: [Edit](#)

Standby Database Progress Summary

Transport lag is the time difference between the last update on the primary database and the last update on the standby database.

seconds

0.5
0.0

london.example.com london2.example.com

Legend: Transport Lag (blue square), Apply Lag (orange square)

Standby Databases [Add Standby Database](#)

Select	Name	Host	Data Guard Status	Role	Real-time Query	Last Received Log	Last Applied Log	Estimated Failover Time
☒	london.example.com	host03	✓ Normal	Physical Standby	Disabled	32	32	< 1 second
☐	london2.example.com	host03	✓ Normal	Logical Standby	N/A	32	32	< 1 second

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

On the Data Guard page, click the Disabled link (next to Fast-Start Failover) to invoke the fast-start failover wizard.

Using Enterprise Manager to Enable Fast-Start Failover

Data Guard > Fast-Start Failover: Configure Logged in as SYS

Information

There is currently no observer for this configuration, nor has one been specified.

Fast-Start Failover: Configure Cancel Continue

Target Database Selection

Select a standby database to be the fast-start failover target. The redo transport mode for the selected database will be set to ASYNC (if not currently set to ASYNC).

Select	Name	Role	Redo Transport Mode
☒	boston.example.com	Physical Standby	ASYNC
☐	london2.example.com	Logical Standby	ASYNC

Observer

Fast-start failover requires a Data Guard observer process. For highest availability, Oracle recommends that the observer be on a separate host from the primary and standby databases.

TIP Specify an alternate observer host to maximize observer availability.

Observer Location **Not Set**
Alternate Observer Location **Not Set** Configure Observer

Select the database.

Click Configure Observer.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Select the fast-start failover target database and (optional) the observer host. If the observer was not started, click the Configure Observer button.

Using Enterprise Manager to Enable Fast-Start Failover

Fast-Start Failover: Configure Observer Cancel OK

Observer Location

There is currently no observer for this configuration. Select the discovered host and Oracle Home where Enterprise Manager will start the observer.

 **TIP** Specify an alternate observer location to enhance observer availability. If an unobserved condition is detected, Enterprise Manager will attempt to restart the observer on the original observer host, falling back to the alternate host if necessary.

Observer Host 

Observer Oracle Home

Alternate Observer Host 

Alternate Observer Oracle Home

Failover Properties

 **Failover Threshold** Amount of time the primary database must be out of contact with the observer and the standby database before a fast-start failover is initiated.

Lag Limit Amount of time the standby database is allowed to fall behind the primary database, beyond which a fast-start failover will not be allowed.

User Configurable Failover Conditions [Edit](#)

Primary Database Properties

Automatically Reinstall Primary ☒ Controls whether the observer will automatically reinstall the former primary database once contact is re-established after the former primary database is restarted. Does not control reinstall behavior for failovers caused by an error condition.

Automatically Shutdown Primary ☒ Controls whether the primary database will shut itself down if it independently discerns that a fast-start failover may have occurred, but cannot verify it due to network isolation from the observer and the standby database. Does not control shutdown behavior for failovers caused by an error condition.

Cancel Continue

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Specify the observer host and observer Oracle home to indicate where the observer will be started.

You can also specify an alternate observer host and alternate observer Oracle home. Enterprise Manager will provide high availability for the observer with this setup. If the original observer or observer node dies, Enterprise Manager will automatically start the observer on the alternate node.

Failover properties and primary database properties related to fast-start failover can also be defined on this page.

Note: Enterprise Manager ensures that each observer has uniquely named data and DGMGRL log files. As a result, there is no problem if observers for different Data Guard configurations are started in the same Oracle Home.

The observer data files, named `afXXXXXX.dat`, are stored in the `$ORACLE_HOME/dbs` directory. The DGMGRL log file, named `dgmgrlXXXXXX.log`, is stored in the `$ORACLE_HOME/rdbms/log` directory.

XXXXXX is a unique number generated by Enterprise Manager and is the same for the observer data file and the DGMGRL log file for a given primary standby database configuration. XXXXX changes each time an observer is restarted.

Using Enterprise Manager to Enable Fast-Start Failover

Data Guard > Fast-Start Failover: Enable Flashback Logging Logged in as SYS

Fast-Start Failover: Enable Flashback Logging

Flashback logging is required to support fast-start failover, and will be enabled on the primary and standby databases.

✓ **TIP** Flashback logging requires a fast recovery area.

Primary Database
Flashback logging is already enabled on database london.example.com.

Standby Database
The following parameters must be set to enable flashback logging on database boston.example.com.

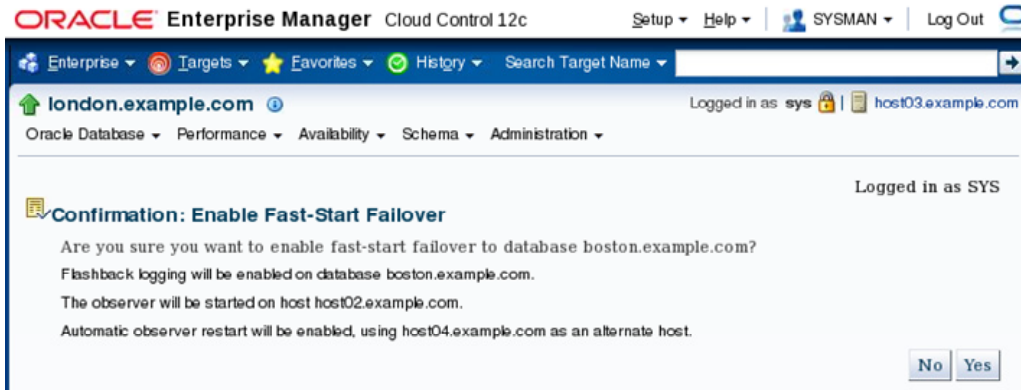
i Fast Recovery Area	/u01/app/oracle/fast_recovery_area
	Specifies the default storage area where archived redo log files (and other recovery-related files) reside.
i Fast Recovery Area Size (MB)	8000
	Limit on the total space used by files created in the fast recovery area. The default value is twice the database size.
i Flashback Retention Time	24 hours
	The upper limit on how far back in time the database may be flashed back.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Flashback logging must be enabled on both the primary database and the standby database to enable fast-start failover. If either database does not have flashback logging enabled, the Enable Flashback Logging page is displayed. Specify the fast recovery area location, fast recovery area size, and flashback retention time for each database.

Using Enterprise Manager to Enable Fast-Start Failover

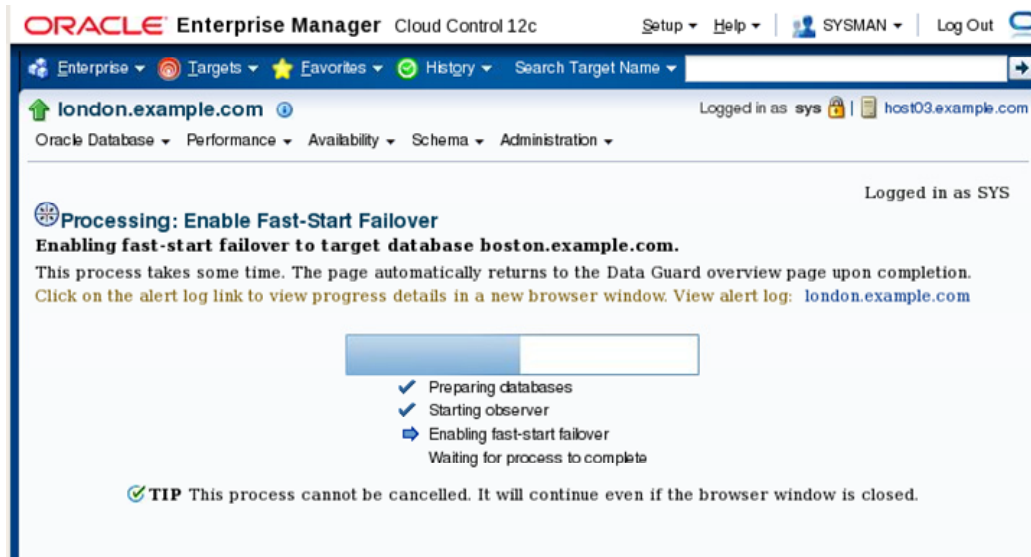


ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

On the Confirmation: Enable Fast-Start Failover page, confirm that you want to enable fast-start failover to the named database.

Using Enterprise Manager to Enable Fast-Start Failover



ORACLE

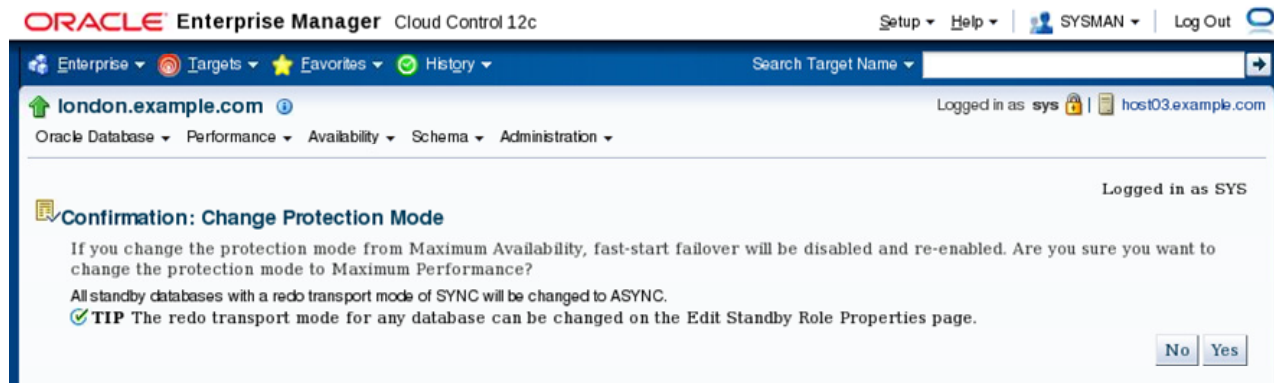
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

After you confirm that you want to enable fast-start failover to a specific database, the fast-start enabling process begins. After the processing steps are complete, you are returned to the Data Guard page.

On the Data Guard Fast-Start Failover Processing page, you can observe the progress of the fast-start failover enabling operation as the following actions are performed:

- Standby redo log files are created on the primary and standby databases.
- The protection mode is upgraded to Maximum Availability (if required).
- The primary database and standby database are restarted (if required).
- Fast-start failover is enabled.
- The observer process is started.

Changing the Protection Mode and Disabling Fast-Start Failover



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If you change the protection mode to a mode that does not support fast-start failover, you implicitly disable fast-start failover.

Using Enterprise Manager to Disable Fast-Start Failover

Data Guard

Page Refreshed August 29, 2013 6:35:31 AM UTC

Overview

Data Guard Status	Warning
Protection Mode	Maximum Performance
Fast-Start Failover	Enabled to boston.example.com
Observer Location	host02

Primary Database

Name	london.example.com
Host	host03
Data Guard Status	ORA-16829: fast-start failover configuration is lagging
Current Log	42
Properties	Edit

Click the Enabled link to access the Change Mode page.

Data Guard > Fast-Start Failover: Change Mode

Fast-Start Failover: Change Mode

Logged in as SYS

Fast-start failover is currently enabled. Choose whether to edit fast-start failover properties or disable fast-start failover, then click Continue.

☐ Edit properties
 Change the fast-start failover target database and threshold, and start or restart the observer process.

☒ Disable
 Disable fast-start failover. Does not stop the observer unless the Stop Observer option is selected.

☒ Stop observer
 To temporarily suspend fast-start failover, select Disable but do not select the Stop Observer option.

Cancel Continue

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Click the Enabled link to access the Fast-Start Failover: Change Mode page, where you select the Disable option and the “Stop observer” check box to disable fast-start failover.

Using Enterprise Manager to Suspend Fast-Start Failover

Database Instance: [pc01prmv.us.oracle.com](#) > [Data Guard](#) >

Fast-Start Failover: Change Mode

Fast-start failover is currently enabled. Choose whether to edit fast-start failover properties or disable fast-start failover, then click Continue.

☐ Edit properties
Change the fast-start failover target database and threshold, and start or restart the observer process.

☒ Disable
Disable fast-start failover. Does not stop the observer unless the Stop Observer option is selected.

☐ Stop observer
To temporarily suspend fast-start failover, select Disable but do not select the Stop Observer option.

Do not stop observer for suspension only.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To suspend fast-start failover, select Disable on the Fast-Start Failover: Change Mode page. Do not select the “Stop observer” check box.

Moving the Observer to a New Host

To move the fast-start failover observer to a new host:

1. Execute the `STOP OBSERVER` command to sever the link between the original observer and the broker configuration.
2. Execute the `SHOW CONFIGURATION VERBOSE` and `SHOW DATABASE` commands to verify that the observer is stopped.
3. Start the observer on your new host.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a solid red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If the observer host machine fails, you may need to move the observer to a new host.

To stop the original observer and start the observer on a new host:

1. Execute the `STOP OBSERVER` command to sever the link between the original observer and the broker configuration:

```
DGMGRL> STOP OBSERVER;
```
2. Execute the `SHOW CONFIGURATION VERBOSE` and `SHOW DATABASE` commands to verify that the observer is stopped and the configuration is no longer being observed.
3. Invoke `DGMGRL`, connect to a database in your configuration, and start the observer on your new host.

Quiz

If the observer is on a different machine than the primary and standby databases, and that machine is lost, a failover will be triggered.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: b

Quiz

If an independent host is not available for the observer, which one of the following hosts is best suited to start the observer on?

- a. Host with the standby database
- b. Host with the primary database

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: a

Summary

In this lesson, you should have learned how to:

- Configure fast-start failover
- View information about the fast-start failover configuration
- Manage the observer
- Perform role changes in a fast-start failover configuration
- Manually reinstate the primary database

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 14: Overview

This practice covers the following topics:

- Enabling fast-start failover
- Starting the observer
- Simulating a failure of the primary database and observing the fast-start failover operation
- Observing the automatic reinstatement of the new physical standby into the Data Guard configuration
- Switching back to your primary database
- Stopping the observer and disabling fast-start failover

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

15

Backup and Recovery Considerations in an Oracle Data Guard Configuration

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Objectives

After completing this lesson, you should be able to:

- Use Recovery Manager (RMAN) to back up and restore files in a Data Guard configuration
- Offload backups to a physical standby database
- Recover your primary database by using a file from the physical standby database

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Using RMAN to Back Up and Restore Files in a Data Guard Configuration

- RMAN can be used to back up and recover databases in a Data Guard configuration.
- Standby database backups are usable on the primary database.
- Backups of logical standby databases are not usable at the primary database.
- The recovery catalog is required when using RMAN in a Data Guard configuration for physical standby databases.
- RMAN and Data Guard best practices:
 - Perform backups at the primary and standby databases to reduce mean time to recover (MTTR).
 - Take daily incremental backups.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is centered on a solid red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Recovery Manager (RMAN) can be used to back up and recover a standby database. Backups that you make on a physical standby database are usable at a primary database or another standby database.

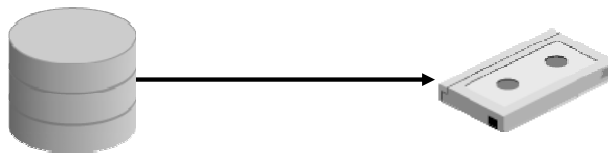
A recovery catalog is required when you use RMAN in a Data Guard configuration containing physical standby databases. Metadata for the primary database and all standby databases is stored in the recovery catalog.

Oracle Maximum Availability Architecture (MAA) best practices recommend that backups be taken at both the primary and the standby databases to:

- Reduce mean time to recover (MTTR)
- Handle outages at multiple sites
- Avoid introducing new site practices during switchover and failover

Offloading Backups to a Physical Standby

- Backups of data files and archived redo logs are fully interchangeable.
- Backups of standby control files and nonstandby control files are interchangeable.
- SPFILE backups are not interchangeable.
- Primary and standby databases must use the same recovery catalog.
- It is not necessary to register the standby database.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

RMAN can back up the standby database and its associated archived redo logs. Standby backups of data files, control files, and archived redo logs are fully interchangeable with primary database backups. You can execute the `RESTORE` command to restore a backup of a standby data file to the primary database, and you can restore a backup of a primary data file to the standby database.

Both the primary database and standby database must use the same recovery catalog. Note that you do not need to register the standby database in the catalog if the primary is already registered; simply connect to the standby database and execute the `BACKUP` command.

Restrictions and Usage Notes

- You must back up a physical standby database.
- You must be connected to the recovery catalog when backing up.
- You must connect to the physical standby database with the `TARGET` keyword.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If physical standby database backups are to be usable for restore jobs at the primary site, you must be connected to the recovery catalog when backing up the standby database or you must resynchronize the physical standby database shortly after the backup. This step is necessary because there is no way for the primary database to know about the standby backups unless the backup records are stored in the recovery catalog.

When you back up the standby database, you must connect to the standby database with the `TARGET` keyword (not the `AUXILIARY` keyword). The standby database essentially “substitutes” for the primary database during the backup.

Association and Accessibility of RMAN Backups

- Every database file or backup file is associated with a `DB_UNIQUE_NAME`.
- The database that creates a file is associated with the file.
- Disk backups are accessible only to the database with which it is associated.
- Tape backups created on one database are accessible to all databases.
- Association of an existing backup file can be changed with the `CHANGE ... RESET DB_UNIQUE_NAME` command.
- A copied backup file from one host to another can be associated with another database with the `CATALOG` command.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The recovery catalog tracks the files in the Oracle Data Guard environment by associating every database file or backup file with a `DB_UNIQUE_NAME`. The database that creates a file is associated with the file. For example, if RMAN backs up the database with the unique name “standby1,” then standby1 is associated with this backup. A backup remains associated with the database that created it unless you use the `CHANGE ... RESET DB_UNIQUE_NAME` to associate the backup with a different database.

The accessibility of a backup is different from its association. In an Oracle Data Guard environment, the recovery catalog considers disk backups as accessible only to the database with which it is associated, whereas tape backups created on one database are accessible to all databases. If a backup file is not associated with any database, the row describing it in the recovery catalog view shows null for the `SITE_KEY` column. By default, RMAN associates files whose `SITE_KEY` is null with the target database.

RMAN commands such as `BACKUP`, `RESTORE`, and `CROSSCHECK` work on any accessible backups. For example, for a `RECOVER COPY` operation, RMAN considers only image copies that are associated with the database as eligible to be recovered. RMAN considers the incremental backups on disk and tape as eligible to recover the image copies. In a database recovery, RMAN considers only the disk backups associated with the database and all files on tape as eligible to be restored.

Backup and Recovery of a Logical Standby Database

- Backup considerations:
 - Use the same backup method that you use for your primary database.
 - Files are not interchangeable with the primary database.
- Recovery considerations:
 - If you lose individual files, perform recovery as you would for any other database.
 - Re-create the logical standby database if you lose the entire database.
 - Use a binary copy of the control file for control-file recovery.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can back up your logical standby database by using the same method that you use for your primary database. The backup files are not interchangeable with the primary database. Remember that this is a different database.

You can use the same recovery techniques as you use with any other database for loss of data files or online log files. You must use the backups of the logical (not the primary) database.

If the entire logical standby database is lost, you must re-create the logical standby.

If you lose all copies of your control file, you must use a binary copy of the control file when performing the recovery. (Using a trace file or the `CREATE CONTROLFILE` command for control file recovery does not create a logical standby control file.) Make a binary copy of the control file by doing either of the following:

- Shut down the logical standby and copy the control file to a backup.
- Issue the following command while the logical standby database is open:

```
ALTER DATABASE BACKUP CONTROLFILE TO '<filename>';
```

This command creates a binary copy of the control file with the name that you supply.

Using the RMAN Recovery Catalog in a Data Guard Configuration

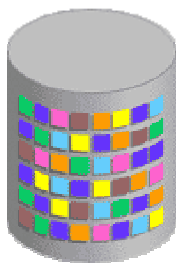
- Use of the RMAN recovery catalog permits backups taken on one database server to be restored to another database server.
- RMAN uses the recovery catalog metadata to behave transparently across different physical databases in the Data Guard configuration.
- Configure a separate server for the recovery catalog so that a disaster at any site in the Data Guard configuration does not affect the ability to recover from the latest backups.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is centered on a solid red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In a Data Guard configuration, you should use an RMAN recovery catalog. This permits backups taken on one database server to be restored to another database server. If you use only the control file as the RMAN repository, the primary database will have no knowledge of backups taken on the standby database.

Creating the Recovery Catalog



1. Configure the recovery catalog database.



2. Create the recovery catalog owner.



3. Create the recovery catalog.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

1. Configure the database in which you want to store the recovery catalog.
 - a. Determine the database in which you will install the recovery catalog schema. Be sure to consider your backup and recovery procedures for this database.
 - b. Create a tablespace for the recovery catalog, which becomes the default tablespace for the recovery catalog owner. The amount of space required by the recovery catalog schema depends on the number of databases monitored by the catalog. The typical space requirement is 15 MB for each database registered in the recovery catalog.

```
SQL> CREATE TABLESPACE rc_ts DATAFILE SIZE 30M;
```

2. Create a user to serve as the recovery catalog owner. Set the default tablespace for this user to the tablespace that you created for the recovery catalog. Be sure to provide UNLIMITED quota on this tablespace for the user. After creating the user, grant the RECOVERY_CATALOG_OWNER role to the user.

```
SQL> CREATE USER rcowner IDENTIFIED BY rcpass
2 TEMPORARY TABLESPACE temp
3 DEFAULT TABLESPACE rc_ts
4 QUOTA UNLIMITED ON rc_ts;
SQL> GRANT recovery_catalog_owner TO rcowner;
```

3. Use the `CREATE CATALOG RMAN` command to create the catalog tables in the default tablespace of the catalog owner.

```
$ rman
```

```
RMAN> CONNECT CATALOG rcowner/rcpass
```

```
RMAN> CREATE CATALOG;
```

Registering a Database in the Recovery Catalog

- Only the primary database is explicitly registered:

```
$ rman TARGET / CATALOG
   username/password@net_service_name
RMAN> REGISTER DATABASE;
```

- A standby database is automatically registered when you connect to it or when the `CONFIGURE DB_UNIQUE_NAME` command is executed.
- RMAN performs the following actions:
 - Creates rows in the recovery catalog tables for the target database
 - Copies data from the target database control file to the recovery catalog tables

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

After creating the recovery catalog, you must register the target databases in the recovery catalog. Only the primary database is registered by using the `REGISTER DATABASE` command. A standby database is automatically registered in the recovery catalog when you connect to it or when the `CONFIGURE DB_UNIQUE_NAME` command is executed to specify the net service name for the physical standby database.

To register your primary database:

1. Invoke RMAN and connect to the recovery catalog database and to the target database, as shown in the following example:

```
rman TARGET / CATALOG rcowner/rcpass@reccatdb
```
2. Ensure that the target database is mounted or open.
3. Issue the `REGISTER` command to register the target database in the recovery catalog:

```
RMAN> REGISTER DATABASE;
```

Setting Persistent Configuration Settings

- Use the `CONFIGURE` command with the `FOR DB_UNIQUE_NAME` clause to create a persistent configuration for a database in a Data Guard configuration.
- You do not have to connect to the standby database or primary database as `TARGET`.
- RMAN must be connected to a recovery catalog when you create or alter a configuration for a database in the Data Guard configuration.

```
RMAN> configure controlfile autobackup on for db_unique_name  
      boston;  
new RMAN configuration parameters:  
CONFIGURE CONTROLFILE AUTOBACKUP ON;  
new RMAN configuration parameters are successfully stored
```



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Use the `CONFIGURE RMAN` command to register and configure settings for databases in your Data Guard configuration.

RMAN uses the `DB_UNIQUE_NAME` initialization parameter to distinguish the databases in the Data Guard configuration. You can use the `CONFIGURE` command with the `FOR DB_UNIQUE_NAME` clause to create a persistent configuration for a database in a Data Guard configuration without connecting to the database as `TARGET`. Use the `SET DBID` command to set the `DBID` in the RMAN session so that you can configure a standby database when RMAN is not connected as `TARGET` to a database in the Data Guard environment. You can also use this technique to create a configuration for a standby database that is not yet created.

RMAN must be connected to a recovery catalog when you create or alter a configuration for a database in the Data Guard environment.

Configuring a Persistent Setting: Example

```
[oracle@host01 ~]$ rman target sys/oracle_4U@boston catalog
rcowner/rcpass@emrep
```

```
Recovery Manager: Release 12.1.0.1.0 - Production on Wed Dec 18 16:04:49
2013
```

```
Copyright (c) 1982, 2013, Oracle and/or its affiliates. All rights
reserved.
```

```
connected to target database: BOSTON (DBID=2524687871, not open)
```

```
connected to recovery catalog database
```

```
RMAN> configure controlfile autobackup on;
```

```
new RMAN configuration parameters:
```

```
CONFIGURE CONTROLFILE AUTOBACKUP ON;
```

```
new RMAN configuration parameters are successfully stored
```

```
RMAN> show all for db_unique_name boston;
```

```
RMAN configuration parameters for database with db_unique_name BOSTON are:
```

```
CONFIGURE RETENTION POLICY TO REDUNDANCY 1; # default
```

```
CONFIGURE BACKUP OPTIMIZATION OFF; # default
```

```
CONFIGURE DEFAULT DEVICE TYPE TO DISK; # default
```

```
CONFIGURE CONTROLFILE AUTOBACKUP ON; # default
```

```
CONFIGURE CONTROLFILE AUTOBACKUP FORMAT FOR DEVICE TYPE DISK TO '%F'; #
default
```

```
CONFIGURE DEVICE TYPE DISK PARALLELISM 1 BACKUP TYPE TO BACKUPSET; # default
```

```
CONFIGURE DATAFILE BACKUP COPIES FOR DEVICE TYPE DISK TO 1; # default
```

```
CONFIGURE ARCHIVELOG BACKUP COPIES FOR DEVICE TYPE DISK TO 1; # default
```

```
CONFIGURE MAXSETSIZE TO UNLIMITED; # default
```

```
CONFIGURE ENCRYPTION FOR DATABASE OFF; # default
```

```
CONFIGURE ENCRYPTION ALGORITHM 'AES128'; # default
```

```
CONFIGURE COMPRESSION ALGORITHM 'BASIC' AS OF RELEASE 'DEFAULT' OPTIMIZE FOR
LOAD TRUE ; # default
```

```
CONFIGURE RMAN OUTPUT TO KEEP FOR 7 DAYS; # default
```

```
CONFIGURE ARCHIVELOG DELETION POLICY TO NONE; # default
```

```
CONFIGURE SNAPSHOT CONTROLFILE NAME TO
```

```
'/u01/app/oracle/product/12.1.0/dbhome_1/dbs/snapcf_boston.f'; # default
```

Setting RMAN Persistent Configuration Parameters on the Primary Database

- Configure the backup retention policy:

```
RMAN> CONFIGURE RETENTION POLICY TO RECOVERY WINDOW OF
      <n> DAYS;
```

- Specify deletion of archived redo logs:

```
RMAN> CONFIGURE ARCHIVELOG DELETION POLICY TO SHIPPED TO
      ALL STANDBY;
RMAN> CONFIGURE ARCHIVELOG DELETION POLICY TO APPLIED ON
      ALL STANDBY;
```

- Configure the connect identifier for the primary database and the standby databases:

```
RMAN> CONFIGURE DB_UNIQUE_NAME boston CONNECT IDENTIFIER
      'boston';
RMAN> CONFIGURE DB_UNIQUE_NAME london CONNECT IDENTIFIER
      'london'
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

- Configure the retention policy to determine the backups that need to be kept to recover to the specified point in time. This setting specifies that necessary backups should be kept to perform database recovery to any point in time within the specified number of days.
CONFIGURE RETENTION POLICY TO RECOVERY WINDOW OF <n> DAYS;
Execute the DELETE OBSOLETE command to delete any backups that are not required by the retention policy.
- Use the CONFIGURE ARCHIVELOG DELETION POLICY command to specify when archived logs can be deleted.
 - To delete logs after ensuring that they shipped to all destinations:
CONFIGURE ARCHIVELOG DELETION POLICY TO SHIPPED TO ALL STANDBY;
 - To delete logs after ensuring that they were applied on all standby destinations:
CONFIGURE ARCHIVELOG DELETION POLICY TO APPLIED ON ALL STANDBY;

- Configure the connect identifier for the primary database and all standby databases so that RMAN can connect remotely and perform resynchronization when the `RESYNC CATALOG FROM DB_UNIQUE_NAME` command is executed. The connect identifier is the Oracle Net service name that can be used from any database site to connect to the specified database site. `CONFIGURE DB_UNIQUE_NAME` implicitly registers the database.

Setting RMAN Persistent Configuration Parameters on the Physical Standby Database

Set the following parameters on the physical standby database where you will perform backups:

- Enable automatic backup of the control file and the server parameter file:

```
RMAN> CONFIGURE CONTROLFILE AUTOBACKUP ON;
```

- Skip backing up data files for which there already exists a valid backup with the same checkpoint:

```
RMAN> CONFIGURE BACKUP OPTIMIZATION ON;
```

- Specify when the archived logs can be deleted:

```
RMAN> CONFIGURE ARCHIVELOG DELETION POLICY TO BACKED UP  
1 TIMES TO DEVICE TYPE DISK;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

On the physical standby database that you will be using for backups, set RMAN persistent configuration parameters as follows:

- Enable automatic backup of the control file and the server parameter file.
CONFIGURE CONTROLFILE AUTOBACKUP ON;
- Set backup optimization to ON so that RMAN does not back up data files for which there is valid backup with the same checkpoint.
CONFIGURE BACKUP OPTIMIZATION ON;
- Specify the archived log file deletion policy to indicate when archived redo log files are deleted.
CONFIGURE ARCHIVELOG DELETION POLICY TO
BACKED UP 1 TIMES TO DEVICE TYPE DISK;

When backing up to tape, configure tape channels as required by the media management software. For details about backups to tape and the use of media management software, see *Oracle Data Guard Concepts and Administration*.

Setting RMAN Persistent Configuration Parameters on the Other Standby Databases

On the standby databases where you will *not* be performing backups, specify that the archived logs can be deleted after they are applied at the standby database:

```
RMAN> CONFIGURE ARCHIVELOG DELETION POLICY TO APPLIED ON  
ALL STANDBY;
```

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is centered on a solid red horizontal bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You specify the archived log file deletion policy to indicate that archived logs are deleted after they are applied at the standby database.

Configuring Daily Incremental Backups

- Configure a daily backup strategy:
 - Day 1: Create a full database backup.
 - Day 2: Create an incremental backup.
 - Day 3: Create an incremental backup.
The previous day's incremental backup is merged with the data file copy, and a current incremental backup is taken.
- A single RMAN script can be created to implement this strategy:

```
RESYNC CATALOG FROM DB_UNIQUE_NAME ALL;
RECOVER COPY OF DATABASE WITH TAG 'dgbkup';
BACKUP DEVICE TYPE DISK INCREMENTAL LEVEL 1 FOR RECOVER
OF COPY WITH TAG 'dgbkup' DATABASE;
BACKUP DEVICE TYPE SBT ARCHIVELOG ALL;
BACKUP BACKUPSET ALL;
DELETE ARCHIVELOG ALL;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Corporation recommends that you implement a daily incremental backup strategy. In this backup strategy, data file image copies are rolled forward with the latest incremental backups. The resulting up-to-date data file image copy is used by RMAN for media recovery.

Perform a full database backup on the first day, followed by an incremental backup on day two. Archived redo logs can be used to recover the database to any point during either day. Beginning on day three, the previous day's incremental backup is merged with the data file copy and a current incremental backup is taken, allowing fast recovery to any point within the last day. Redo logs can be used to recover the database to any point during the current day.

You can create an RMAN script to take all of the backups (as shown in the slide):

- **RESYNC CATALOG FROM DB_UNIQUE_NAME ALL:** Resynchronizes the information from all database sites (primary and other standby databases) in the Data Guard configuration that are known to the recovery catalog.
- **RECOVER COPY OF DATABASE WITH TAG 'dgbkup':** Rolls forward the level 0 copy of the database by applying the level 1 incremental backup taken the day before.
 - On day 1, there is no roll forward because there is no incremental level 1 backup.
 - On day 2, there is no roll forward because there is only a level 0 incremental backup at this time.
 - On day 3 and the following days, the roll forward is performed using the level 1 incremental backup created on the previous day.

- **BACKUP DEVICE TYPE DISK INCREMENTAL LEVEL 1 FOR RECOVER OF COPY WITH TAG 'dgbkup' DATABASE:** Creates a new level 1 incremental backup.
 - On day 1, a level 0 incremental backup is created.
 - On day 2 and the following days, a level 1 incremental backup is created.
- **BACKUP DEVICE TYPE DISK ARCHIVELOG ALL:** Backs up archived logs to disk.
- **BACKUP BACKUPSET ALL:** Backs up any backup sets that were created as a result of incremental backup.
- **DELETE ARCHIVELOG ALL:** Deletes archived logs according to the log deletion policy set by the `CONFIGURE ARCHIVELOG DELETION POLICY` command. If the archived logs are in a flash recovery area, they are automatically deleted when more open disk space is required. Therefore, you need to use this command only if you explicitly want to delete logs each day.

Recovering from the Loss of a Data File on the Primary Database

- Using backups
- Using a file on the standby database

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

These two methods are covered on the following pages.

Using a Backup to Recover a Data File on the Primary Database

- Using backups to restore and recover data files:

```
RMAN> CONNECT TARGET sys@boston  
RMAN> RESTORE DATAFILE 5,6;  
RMAN> RECOVER DATAFILE 5,6;
```

- Using backups to restore and recover tablespaces:

```
RMAN> CONNECT TARGET sys@boston  
RMAN> RESTORE TABLESPACE tbs_name1,tbs_name2;  
RMAN> RECOVER TABLESPACE tbs_name1,tbs_name2;
```



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Invoke RMAN and connect to the primary database and the recovery catalog.

- Execute the `RESTORE DATAFILE` command to restore the data file.
- Execute the `RECOVER DATAFILE` command to recover the data file.

For detailed information about the `RESTORE` and `RECOVER` commands, see the *Oracle Database Backup and Recovery Reference*.

Using a Physical Standby Database Data File to Recover a Data File on the Primary Database

1. Connect to the standby database as the target database and connect to the primary database as the auxiliary database:

```
RMAN> CONNECT TARGET sys/oracle_4U@london  
RMAN> CONNECT AUXILIARY sys/oracle_4U@boston
```

2. Back up the data file on the standby database host to a location on the primary database host:

```
RMAN> BACKUP AS COPY DATAFILE 5 AUXILIARY FORMAT  
      '/u02/oracle/restore/df5copy.dbf';
```

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To use a current data file from the physical standby database to recover a data file on the primary database:

1. Connect to the standby database as the target database:
`CONNECT TARGET sys/oracle@london`
Connect to the primary database as the auxiliary database:
`CONNECT AUXILIARY sys/oracle@boston`
2. Back up the data file on the standby host across the network to a location on the primary host, as in the following example:
`BACKUP AS COPY DATAFILE 5`
`AUXILIARY FORMAT '/u02/oracle/restore/df5copy.dbf';`

Using a Physical Standby Database Data File to Recover a Data File on the Primary Database

3. Connect to the primary database as the target database and connect to the recovery catalog:

```
RMAN> CONNECT TARGET sys/oracle_4U@boston CATALOG
rcowner/rcpass@em12;
```

4. Catalog the data file copy for RMAN use:

```
RMAN> CATALOG DATAFILECOPY
'/u02/oracle/restore/df5copy.dbf';
```

5. Switch the data file copy so that it becomes the current data file:

```
RMAN> RUN { SET NEWNAME FOR DATAFILE 5 TO
'/u02/oracle/restore/df5copy.dbf';
SWITCH DATAFILE 5;
RECOVER DATAFILE 5; }
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

3. Exit the RMAN session. Invoke RMAN again and connect to the primary database as the target database and connect to the recovery catalog, as in this example:

```
CONNECT TARGET sys/oracle_4U@boston
CATALOG rcowner/rcpass@em12;
```
4. Use the CATALOG DATAFILECOPY command to catalog this data file copy so that RMAN can use it.

```
CATALOG DATAFILECOPY '/u02/oracle/restore/df5copy.dbf';
```
5. Use the SWITCH DATAFILE command to switch the data file copy so that the cataloged file becomes the current data file.

```
RUN {
  SET NEWNAME FOR DATAFILE 5 TO
  '/u02/oracle/restore/df5copy.dbf';
  SWITCH DATAFILE 5;
  RECOVER DATAFILE 5;
}
```

Recovering a Data File on the Standby Database

- If all archived redo log files needed for recovery are accessible on disk:
 1. Restore the data file by using the `RESTORE DATAFILE` command.
 2. Restart Redo Apply.
- If all archived redo log files needed for recovery are not accessible on disk:
 1. Restore the data file by using the `RESTORE DATAFILE` command.
 2. Use the `RECOVER DATABASE` command to recover the data file to an SCN that is greater than the last log applied to the standby database.
 3. Restart Redo Apply.
- Use a current data file from the primary database.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Using a Backup to Recover a Data File on the Standby Database

To recover a data file on the standby database, you must restore the lost file to the standby database from the backup by using the `RESTORE DATAFILE RMAN` command. If all archived redo log files that are required for recovery of the file are accessible on disk by the standby database, you only need to restart Redo Apply.

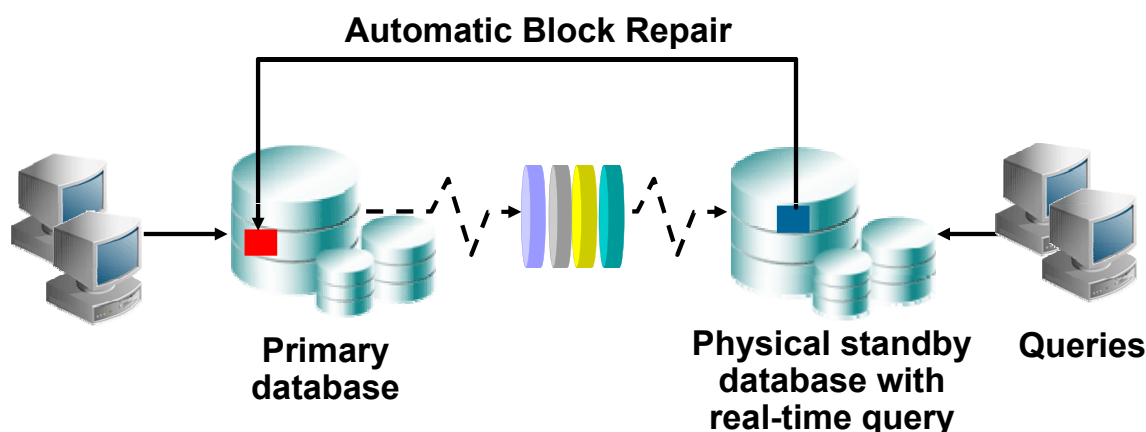
If the archived redo log files that are required for recovery are not accessible on disk, use RMAN to recover the restored data files to an SCN or log sequence that is greater than the last log applied to the standby database. Next, restart Redo Apply to continue the application of redo data.

Using a Data File from the Primary Database

You can use a current data file from the primary database to recover a file on the standby database.

Enhancements to Block Media Recovery

- Corrupted blocks in the primary database are automatically repaired by using blocks from a physical standby database.
- Real-time query and Active Data Guard must be enabled on the physical standby database.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In past releases of Oracle Database, if a block was corrupted, you had to identify the corrupt block and manually perform block media recovery by executing the `RECOVER ... BLOCK` command.

The Automatic Block Repair feature enables block media recovery to use blocks from a physical standby database to perform the recovery without manual intervention. The physical standby database must have real-time query enabled to take advantage of this feature.

When a query is executed on a physical standby database configured with real-time query and a corrupted block is detected, a valid block is retrieved from the primary database.

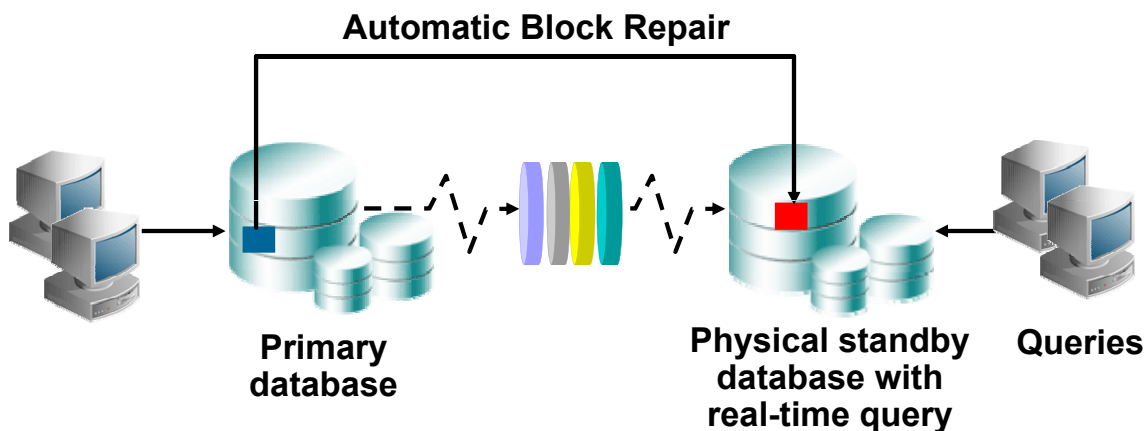
When a corrupted block is detected during a recovery operation on the standby database, the recovery process will retrieve a valid block from the primary database.

This feature reduces the amount of time that production data cannot be accessed due to block corruption by automatically repairing the corruption as soon as it is detected. Block recovery time is reduced because up-to-date good blocks from a real-time, synchronized physical standby database are used rather than blocks from disk or tape backups or flashback logs.

Note: Automatic Block Repair is applicable only for physical block corruption (when the checksum is invalid, the block contains all zeros, or the block header is fractured).

Enhancements to Block Media Recovery

- Corrupted blocks in the physical standby database are automatically repaired by using blocks from the primary database
- Real-time query and Active Data Guard must be enabled on the physical standby database



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Automatic Block Repair also enables the automatic repair of corrupt blocks on the physical standby database. Block media recovery is performed by using blocks retrieved from the primary database. Real-time query must be enabled on the physical standby database to take advantage of this feature.

Executing the RECOVER BLOCK Command

- Block media recovery can also be performed manually by using the RECOVER BLOCK command.

```
RECOVER DATAFILE 6 BLOCK 3; Recover a single block

RECOVER                                Recover multiple blocks
DATAFILE 2 BLOCK 43,79                in multiple data files
DATAFILE 6 BLOCK 183;
```

- Search for usable blocks for the recovery proceeds as follows:
 1. Physical standby database
 2. Flashback logs
 3. Blocks in full or level 0 incremental backup

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In past releases of Oracle Database, when you executed the RECOVER ... BLOCK command to perform block media recovery, RMAN searched the flashback logs for good copies of the blocks, and then searched in full or level 0 incremental backups. Now RMAN first attempts to restore blocks from a physical standby database that is configured with real-time query. If RMAN does not find a valid block in the physical standby database, the flashback logs are searched. Finally, the full or level 0 incremental backups are searched.

For RMAN to be able to search a standby database for good copies of corrupt blocks, the target database must be associated with a physical standby database in a Data Guard environment. In addition, the physical standby database must be open read-only in managed recovery.

Excluding the Standby Database

- By default, block media recovery searches the physical standby database first for blocks to use.
- Exclude the standby database from the search by including the `EXCLUDE STANDBY` clause.

```
RECOVER BLOCK ... EXCLUDE STANDBY
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

By default, the `RECOVER ... BLOCK` command tries to retrieve a good block from a physical standby database. You can disable the search on the physical standby database by specifying the `EXCLUDE STANDBY` clause with the command. You may use the `EXCLUDE STANDBY` clause when you know there is a network issue or other reason that the standby database is not available.

Quiz

You must register the standby database to the RMAN recovery catalog in order to exchange backup files with the primary database.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: b

Quiz

If a logical standby database has applied all data from the primary database, the data files can be exchanged with that of the primary database.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: b

Summary

In this lesson, you should have learned how to:

- Use RMAN to back up and restore files in a Data Guard configuration
- Offload backups to a physical standby database
- Recover your primary database by using a file from the physical standby database

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 15: Overview

This practice covers the following topics:

- Creating a recovery catalog
- Setting RMAN persistent configuration parameters
- Performing a backup of the physical standby database
- Recovering a primary database data file by using a data file from your standby database

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Enhanced Client Connectivity in a Data Guard Environment

16

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Objectives

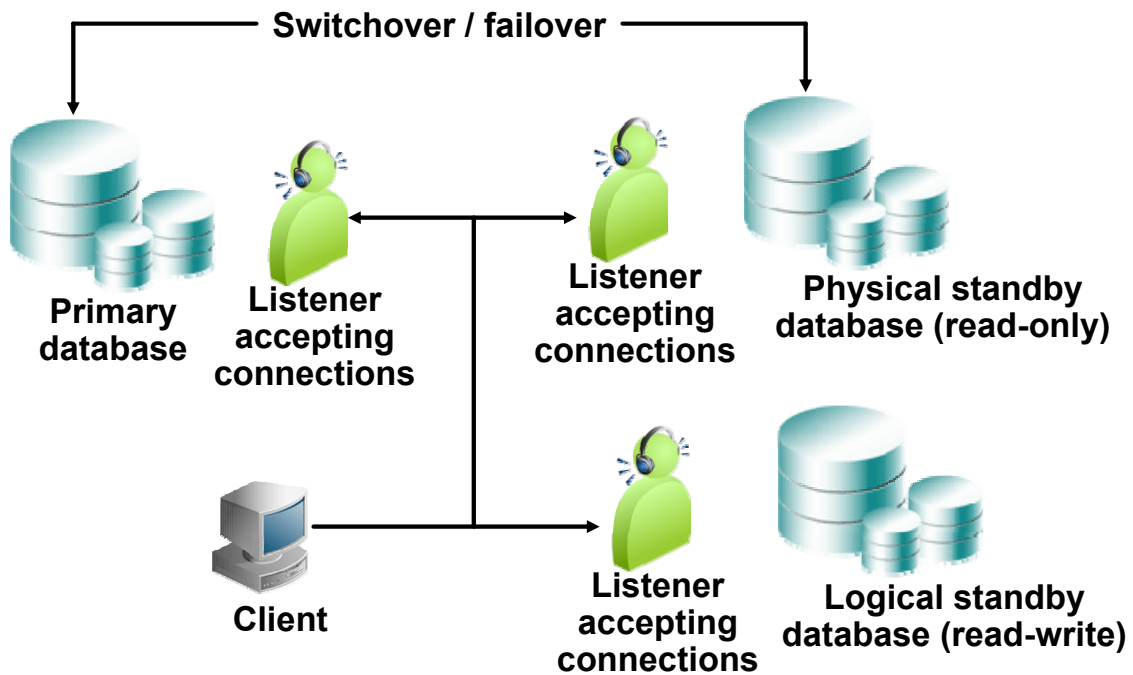
After completing this lesson, you should be able to:

- Configure client connectivity in a Data Guard configuration
- Implement failover procedures to automatically redirect clients to a new primary database

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Connecting to the Appropriate Environment



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A physical standby database not using Active Data Guard maintains the database in a mounted mode and would not normally allow listener connections for normal users. However, with Active Data Guard, the database is open read-only to allow reporting against the physical standby. Also the addition of logical standby databases that are open read write would allow a client to successfully connect using standard database authentication schemes such as username and password. It is important that the client connect to the appropriate environment.

Understanding Client Connectivity in a Data Guard Configuration

Be aware of the following issues when you manage client connectivity in a Data Guard configuration:

- Databases reside on different hosts in a Data Guard configuration.
- Clients must connect to the correct database that provides a specific business role such as primary, logical standby, snapshot standby, or physical standby.
- Primary and standby databases can alternate hosts with switchover and failover operations.
- If clients send connection requests to the wrong host, they may be connected to the wrong database or receive an error.
- Clients must automatically reconnect to the correct database in the event of a failover.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Consider the issues listed in the slide when managing client connectivity in a Data Guard configuration.

Preventing Clients from Connecting to the Wrong Database

- Use database services to prevent clients from connecting to the wrong database in the Data Guard configuration.
- Database services act as an abstraction layer between the client and database instances.
- Database services register with listeners.
- Clients connect to database services instead of database instances.
- Listeners use registration details to determine which instances support a particular service at a particular moment in time.
- Listeners then direct connection requests to the correct instances; otherwise, the appropriate error is returned.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is positioned on the right side of a solid red horizontal bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Clients who send connection requests to the wrong host might be connected to the wrong database instance, or they might receive an error message such as the following:

```
ORA-01033: ORACLE initialization or shutdown in progress
```

You can prevent clients from connecting to the wrong database by using database services.

Managing Services

- Database services can be managed by using the `DBMS_SERVICE` package when Oracle Restart is not used.
- Database services attributes:
 - Service Name: Name of the service for administration
 - Network Name: Network name of the service as used in SQLNet connect descriptors for client connections
 - Parameter Array: Associative array with name-value pairs used to define:
 - Transparent Application Failover (TAF) attributes
 - Workload management goal directives for the service
 - Fast Application Notification (FAN)
 - Distributed Transaction Processing (DTP or XA)

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Database services are implemented with the `DBMS_SERVICE` package. This package provides for the creation, deletion, starting, and stopping of services for a single database instance.

Note: If Oracle Restart is installed and configured by the installation of Grid Infrastructure for a Standalone Server, the `SRVCTL` utility should be used to manage role-based services, instead of `DBMS_SERVICE`.

The `SRVCTL` utility for Oracle Real Application Clusters should always be used to manage role-based services for a cluster environment.

Creating Services for the Data Guard Configuration Databases

```
DBMS_SERVICE.CREATE_SERVICE( -  
  SERVICE_NAME => 'DG_PROD', -  
  NETWORK_NAME => 'DG_PROD', -  
  FAILOVER_METHOD => 'BASIC', -  
  FAILOVER_TYPE => 'SELECT', -  
  FAILOVER_RETRIES => 180, -  
  FAILOVER_DELAY => 1);
```

```
DBMS_SERVICE.CREATE_SERVICE( -  
  SERVICE_NAME => 'DG_RTQ', -  
  NETWORK_NAME => 'DG_RTQ');
```

```
DBMS_SERVICE.CREATE_SERVICE('DG_LSBY', 'DG_LSBY');
```

```
DBMS_SERVICE.CREATE_SERVICE('DG_SNAP', 'DG_SNAP');
```



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

By using the `DBMS_SERVICE.CREATE_SERVICE` procedure, you define a service to represent each role or state in which the databases in your Data Guard configuration can operate. A service created with `DBMS_SERVICE.CREATE_SERVICE` is not aware of the actual database role. This functionality is only available with the `SRVCTL` interface for both Real Application Clusters and Oracle Restart. In the examples in the slide, the service name is being used to imply the database role, which could be inaccurate after role transitions. The `CREATE_SERVICE` procedure creates a service name in the data dictionary.

You should create services for the physical standby database when it is opened in read-only mode (using real-time query) and when it is converted into a snapshot standby database. The `DBMS_SERVICE.CREATE_SERVICE` will fail to execute on a physical standby database even if it is open read-only. The service must be created on the primary and allowed to propagate to the physical standby. In addition, create a service for logical standby databases in your configuration.

Note: Database service names in the slide are examples.

Connecting Clients to the Correct Database

- Use a database event trigger to ensure that clients connect to a database in the Data Guard configuration that is in the correct state and role.
- If no database is in the correct state and role, the trigger ensures that clients do not connect to a database.
- Use the trigger to start database services that have already been created.
 - DG_PROD: Primary database
 - DG_RTQ: Physical standby database opened in READ ONLY mode (real-time query)
 - DG_SNAP: Physical standby database converted to a snapshot standby database
 - DG_LSBY: Logical standby database

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Ensure that clients connect to the database instance that is in the correct state and role in the Data Guard configuration by using a database event trigger. The `AFTER STARTUP` trigger starts the appropriate service on the database.

Note: Database service names in the slide are shown as an example. It is not necessary to employ an `AFTER STARTUP` trigger if the services are managed by Oracle Clusterware.

Creating the AFTER STARTUP Trigger for Role-Based Services

```
CREATE TRIGGER MANAGE_SERVICES AFTER STARTUP ON DATABASE
DECLARE
    ROLE VARCHAR2(30);
    OMODE VARCHAR2(30);
BEGIN
    SELECT DATABASE_ROLE INTO ROLE FROM V$DATABASE;
    SELECT OPEN_MODE INTO OMODE FROM V$DATABASE;
    IF ROLE = 'PRIMARY' THEN
        DBMS_SERVICE.START_SERVICE ('DG_PROD');
    ELSIF ROLE = 'PHYSICAL STANDBY' THEN
        IF OMODE LIKE 'READ ONLY%' THEN
            DBMS_SERVICE.START_SERVICE ('DG_RTQ');
        END IF;
    ELSIF ROLE = 'LOGICAL STANDBY' THEN
        DBMS_SERVICE.START_SERVICE ('DG_LSBY');
    ELSIF ROLE = 'SNAPSHOT STANDBY' THEN
        DBMS_SERVICE.START_SERVICE ('DG_SNAP');
    END IF;
END;
/
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The AFTER STARTUP trigger is invoked when the database is opened. The trigger checks the database role and the open mode of the database and, based on the values, invokes the DBMS_SERVICE.START_SERVICE procedure to start the appropriate service. The CREATE TRIGGER SQL command needs to be run only in the primary database. It will be replicated to all standby databases.

Starting with Oracle Database 12c Release 1 (12.1), there is a new role value for the DATABASE_ROLE column of the V\$DATABASE view. The value FAR SYNC has been added. No additional services should be started for this role type.

Note: The trigger shown in the slide is an example using the service names defined earlier in the lesson. Additional trigger functionality may be necessary based on your circumstances. It is not necessary to employ an AFTER STARTUP trigger if the services are managed by Oracle Clusterware. The trigger listed in the slide does not consider the effect of pluggable databases. In the practice sessions for this course, you will write a startup trigger that is aware of pluggable databases.

Configuring Service Names in the tnsnames.ora File

```
PROD = (DESCRIPTION = (ADDRESS_LIST =
  (ADDRESS=(PROTOCOL = TCP) (HOST = host01) (PORT = 1521))
  (ADDRESS=(PROTOCOL = TCP) (HOST = host03) (PORT = 1521)))
  (CONNECT_DATA = (SERVICE_NAME = DG_PROD)))

RTQ = (DESCRIPTION = (ADDRESS_LIST =
  (ADDRESS=(PROTOCOL = TCP) (HOST = host01) (PORT = 1521))
  (ADDRESS=(PROTOCOL = TCP) (HOST = host03) (PORT = 1521)))
  (CONNECT_DATA = (SERVICE_NAME = DG_RTQ)))

SNAP = (DESCRIPTION = (ADDRESS_LIST =
  (ADDRESS=(PROTOCOL = TCP) (HOST = host01) (PORT = 1521))
  (ADDRESS=(PROTOCOL = TCP) (HOST = host03) (PORT = 1521)))
  (CONNECT_DATA = (SERVICE_NAME = DG_SNAP)))

LSBY = (DESCRIPTION = (ADDRESS_LIST =
  (ADDRESS=(PROTOCOL = TCP) (HOST = host01) (PORT = 1521))
  (ADDRESS=(PROTOCOL = TCP) (HOST = host03) (PORT = 1521)))
  (CONNECT_DATA = (SERVICE_NAME = DG_LSBY)))
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To ensure that clients connect to a database in the correct state and role for a particular service, configure a Net service name for each service in the `tnsnames.ora` file. For each entry, both the listening endpoint on the primary database and the listening endpoint on the standby database are listed. This is needed if role-reversal events such as failover and switchover occur. Remember that the `ADDRESS_LIST` syntax causes the `FAILOVER=on` mode to be enabled even if not specified.

Note: The example in the slide corresponds to the service names and trigger defined earlier in the lesson.

Configuring Role-Based Services Using Oracle Clusterware

- Use `SRVCTL` to configure Oracle Clusterware–managed services on each database in the Data Guard configuration.
- Role changes managed by the Data Guard broker automatically start services appropriate to the database role.
- The service is started when `ROLE` matches the current role of the database and `MANAGEMENT POLICY` is set to `AUTOMATIC`.
- Services can be started manually.

```
srvctl add service -db <db_unique_name> -service
<service_name> -role "[PRIMARY] [,PHYSICAL_STANDBY]
[,LOGICAL_STANDBY] [,SNAPSHOT_STANDBY]" [-policy
{AUTOMATIC | MANUAL}]
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can configure database services with a database role on an Oracle RAC database or a single-instance database registered with Oracle Restart. The Data Guard broker interacts with Oracle Clusterware or Oracle Restart to ensure that the correct database services are active after a role change. This feature is available only if the Data Guard configuration is broker managed.

The `SRVCTL ADD SERVICE` and `MODIFY SERVICE` commands support the following attributes:

- **-role:** `ROLE` attribute with values of `PRIMARY`, `PHYSICAL_STANDBY`, `LOGICAL_STANDBY`, and `SNAPSHOT_STANDBY`. Default is `PRIMARY`. This attribute is used to specify the database role for which a given database service should be active.
- **-policy:** `MANAGEMENT POLICY` attribute with values of `AUTOMATIC` and `MANUAL`. Default is `AUTOMATIC`.

When the database instance starts, the service is started automatically if the `MANAGEMENT POLICY` attribute is set to `AUTOMATIC` and the value of the `ROLE` attribute matches the database role. Users can also start a service manually if the database instance is started.

Note: If the service is created on a database instance that has already been started, then it will be necessary to manually start the service the first time. The following syntax illustrates using `SRVCTL` to manually start a service:

```
srvctl start service -db DG_PRMY -service DG_PRMY
```

For stand-alone servers not using Real Application Clusters (RAC), the following options to the `SRVCTL ADD SERVICE` command are applicable in Oracle Data Guard environments only:

- **Failover type** (`-failovertype`) with values { `NONE` | `SESSION` | `SELECT` | `TRANSACTION` }
- **Failover method** (`-failovermethod`) with values { `NONE` | `BASIC` }
- **Failover delay** (`-failoverdelay`) integer value measured in seconds
- **Failover retries** (`-failoverretry`) integer value indicating a count

Note: Starting with Oracle Database 12c Release 1(12.1), Oracle Restart has been deprecated. It is still currently available to be used, but may be removed in a future version.

Adding Standby Databases to Oracle Restart Configuration

- Standby databases created with Enterprise Manager are not automatically added to the Oracle Restart configuration.
- Use SRVCTL to add the standby databases.

```
srvctl add database -d <db_unique_name> -o <oracle_home>
[-m <domain_name>] [-p <spfile>] [-r {PRIMARY | PHYSICAL_STANDBY
| LOGICAL_STANDBY | SNAPSHOT_STANDBY }] [-s {start_options}]
[-t {stop_options}] [-n <db_name>] [-y {AUTOMATIC | MANUAL}]
[-a "<diskgroup_list>"]
```

- Example:

```
srvctl add database -d london -o
/u01/app/oracle/product/11.2.0/dbhome_1 -m example.com -p
/u01/app/oracle/product/11.2.0/dbhome_1/dbs/spfilelondon.ora
-r PHYSICAL_STANDBY -n boston -a "SBDAT,SBFRA"
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When Enterprise Manager Cloud Control is used to create standby databases with the wizard-driven approach, the newly created standby databases are not automatically registered with the Oracle Restart configuration. Use SRVCTL to create the database object in the Oracle Restart Configuration before database services can be associated with the standby databases. The syntax for SRVCTL is displayed in the slide to create a new database object. The values for `start_options` include `open`, `mount`, or `nomount`. The values for `stop_options` include `normal`, `transactional`, `immediate`, or `abort`.

Note: Starting with Oracle Database 12c Release 1(12.1), Oracle Restart has been deprecated. It is still currently available to be used, but may be removed in a future version. The example in the slide is using a reference to an Oracle Database 11.2 in which Oracle Restart was a suggested best practice.

Example: Configuring Role-Based Services

- PAYROLL: Read-write service that always runs on the database with the primary role
- ORDERSTATUS: Read-only service that always runs on an Active Data Guard standby database

```
srvctl add service -d boston -s payroll -l PRIMARY
                  -m BASIC -e SELECT -w 1 -z 180

srvctl add service -d dallas -s payroll -l PRIMARY
                  -m BASIC -e SELECT -w 1 -z 180

srvctl add service -d dallas -s orderstatus
                  -l PHYSICAL_STANDBY
```



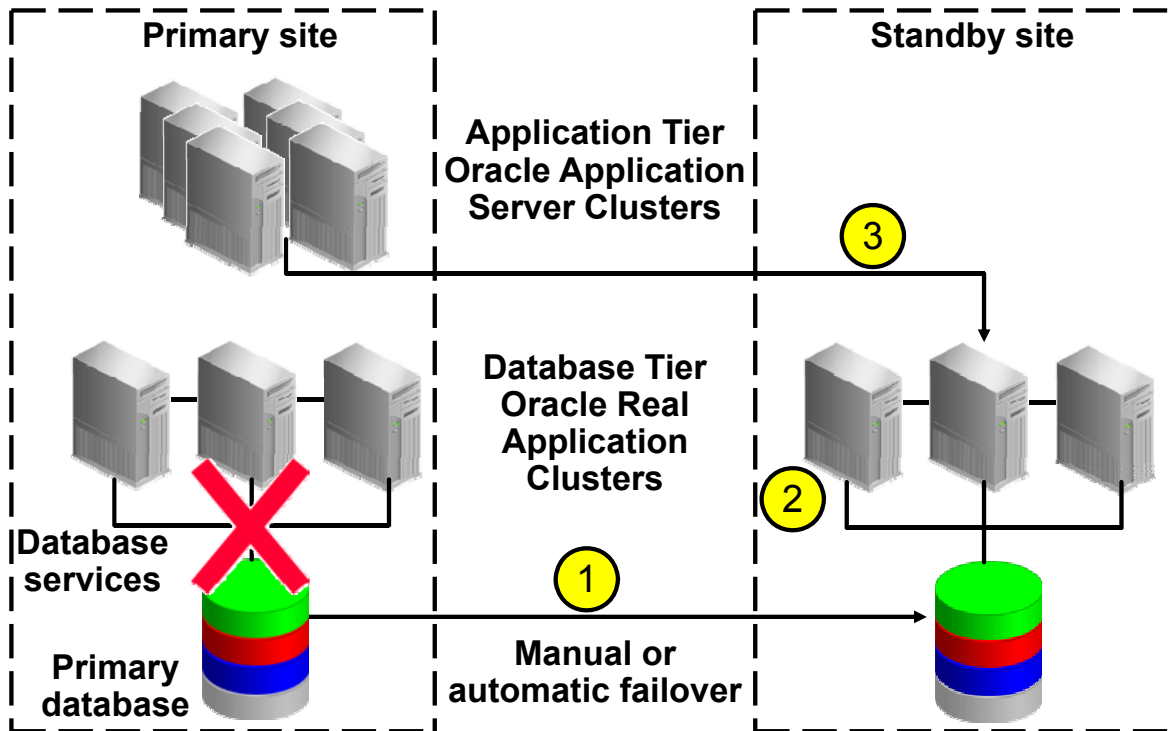
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The example in the slide shows the configuration of two role-specific services:

- PAYROLL is a read-write service that always runs on the database with the primary role.
- ORDERSTATUS is a read-only service that always runs on an Active Data Guard standby database, which is currently defined as the Dallas database while it is in the PHYSICAL_STANDBY role. A role transition can stop this service.

Note: Database services are linked to a specific DB_UNIQUE_NAME in the Oracle Restart Configuration. For the PAYROLL service to be associated with both the primary database and standby database, it will have to be registered to each database individually. The role is set to PRIMARY, so it will start only on one of the two databases.

Automatic Failover of Applications to a New Primary Database



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Without the use of the Oracle Data Guard broker, user-written database triggers are required to implement automatic failover as follows:

- A startup trigger is used to start database services on the new primary database.
- A role-change trigger is used to publish a FAN ONS event to break JDBC clients still connected to the original primary database out of a TCP timeout.

You can automate fast failover of applications to a new primary database without the need for user-written triggers. You must use the Data Guard broker to use this feature.

Automatic fast failover of application clients to a new primary database following failover requires:

1. Fast database failover
2. Restarting database services on the new primary database
3. Notifying clients that a failure has occurred in order to break them out of TCP timeout and redirect them to the new primary database

Data Guard Broker and Fast Application Notification (FAN)

- The Data Guard broker publishes FAN events at failover time.
- Applications respond to FAN events without programmatic changes if using Oracle-integrated database clients:
 - Oracle Database JDBC
 - Oracle Database Oracle Call Interface (OCI)
 - Oracle Database ODP.NET
- Clients that receive FAN events can be configured for Fast Connection Failover (FCF) to automatically connect to a new primary database.
- Clients connect to the new primary database using an Oracle Net connect descriptor configured for connect-time failover.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When a failover operation is complete, the Data Guard broker publishes a Fast Application Notification (FAN) event to notify applications that the old primary database is down, and that services on the old primary database are down. This enables applications to transparently fail over to the new primary database.

FAN notification is sent after failover for databases configured with Cluster Ready Services (CRS) and for single-instance databases registered with Oracle Restart.

Applications using the following Oracle integrated database clients can be configured for Fast Connection Failover (FCF) to automatically connect to a new primary database after a failover:

- Oracle Database JDBC
- Oracle Database Oracle Call Interface (OCI)
- Oracle Database ODP.NET

These clients can use FAN without programmatic changes.

Applications can use FAN programmatically by using the Oracle Notification Service (ONS) Application Programming Interface to subscribe to FAN events and to execute event-handling actions upon the receipt of an event.

Refer to *Oracle Real Application Clusters Administration and Deployment Guide 12c Release 1* for detailed information about configuring FAN, FCF, and ONS in an Oracle RAC environment.

Automating Client Failover in a Data Guard Configuration

- Relocating database services to the new primary database as part of a failover operation
- Notifying clients that the failure has occurred
- Redirecting clients to a new primary database

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Automating client failover in a Data Guard configuration includes:

- Relocating database services to the new primary database as part of a Data Guard failover
- Notifying clients that the failure occurred in order to break them out of TCP timeout
- Redirecting clients to the primary database that is established during the failover operation

Client Failover: Components

The following features are used to implement client failover and minimize the impact of planned and unplanned outages:

- Connect time failover
- Transparent Application Failover (TAF)
- Fast Application Notification (FAN)
- Fast Connection Failover (FCF)
- `DB_ROLE_CHANGE` system event

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

- **Connect time failover:** Redirects failed connection requests to a secondary listener.
- **Transparent Application Failover (TAF):** Enables Oracle Call Interface (OCI) client applications to automatically reconnect to a database if the original connection fails. TAF fails over only the session and `SELECT` statements. `SELECT` statements are automatically restarted in the new session when TAF is configured for `SELECT` failover. `INSERT`, `UPDATE`, and `DELETE` statements must be rolled back by the application. In addition, any session customizations (for example, `ALTER SESSION` statements) must be reexecuted by the application. Process state variables (such as PL/SQL session level variables) are not reestablished but can be reestablished by using a TAF callback.
- **Fast Application Notification (FAN):** Provides quick notification when a resource (such as an instance, service, node, or database) fails. FAN is available to all applications by using either Fast Connection Failover with a FAN-integrated Oracle client (clients using JDBC, OCI, or OLE DB) or by using the FAN API to read FAN events directly.
- **Fast Connection Failover:** Provides fast failover of database connections by enabling you to configure FAN-integrated JDBC clients to automatically subscribe to FAN high-availability events and react to service, instance, and database `UP` and `DOWN` events.
- **`DB_ROLE_CHANGE` system event:** Is fired when any database is first opened after a Data Guard role transition occurs. Using this system event, you can write a trigger to perform post-role change actions.

Client Failover: Best Practices

- Configure OCI clients for FAN OCI:
 - AQ_HA_NOTIFICATIONS
- Configure DBC clients for Fast Connection Failover and FAN ONS:
 - ONS daemons on primary and standby clusters
 - JDBC client uses remote subscription to all daemons
- Implement fast ADDRESS_LIST transversal:
 - OCI: OUTBOUND_CONNECT_TIMEOUT
 - JDBC: SQLnetDef.TCP_CONNTIMEOUT_STR

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is centered on a solid red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Additional information about each type of client follows in this lesson. For detailed examples, see the white paper titled “Client Failover Best Practices for Highly Available Oracle Databases: Oracle Database 11g Release 2.” This white paper is valid for Oracle Database 12c Release 1 (12.1) until a new white paper is developed to replace it.

Automating Failover for OCI Clients

- On the database instance:
 - Use `DBMS_SERVICE.CREATE_SERVICE` or `SRVCTL` to create a database service, enable high-availability notification, and configure server-side TAF settings.
 - Create a trigger on the system startup event to relocate the database service after a role transition if needed.
- To configure your OCI clients:
 - Use the `OCI_EVENTS` parameter to initialize the environment.
 - Link the OCI client applications with the thread library.
 - Set the `SQLNET.OUTBOUND_CONNECT_TIMEOUT` parameter in the `sqlnet.ora` file to a value of 3 seconds.
 - Register an event callback.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To automate failover for OCI clients, perform the following steps to configure your database:

1. Ensure that your configuration is managed by the Data Guard broker.
2. Execute the `DBMS_SERVICE.CREATE_SERVICE` procedure to create the database service, enable high-availability notification, and configure server-side TAF settings.
3. Create a trigger that fires on the system startup event.
This trigger relocates the database to the service (created in step 2) to a Data Guard standby database after a role transition.

To automate failover for OCI clients, perform the following steps to configure your OCI clients to receive notification of FAN high-availability events and avoid reconnecting to a failed instance:

1. Create an Oracle Net service name that includes an `ADDRESS` entry for the primary database host and all standby databases hosts.
2. Use the `OCI_EVENTS` parameter to initialize the environment so that OCI clients receive FAN notifications:

```
OCIEnvCreate(...OCI_EVENTS...)
```

Note: See the *Oracle Call Interface Programmer's Guide* for additional information.

3. Link the OCI client application to the `libthread` or `libpthread` thread library.

4. Set the `SQLNET.OUTBOUND_CONNECT_TIMEOUT` parameter in the `sqlnet.ora` file to a value of 3 seconds.

This parameter enables clients to quickly traverse an address list in the event of a failure. If a client attempts to connect to a host that is unavailable, the connection attempt is bounded to the time specified by the `SQLNET.OUTBOUND_CONNECT_TIMEOUT` parameter, after which the client attempts to connect to the next host in the address list. This behavior continues for each host in the address list until a successful connection is made.

5. Register a callback that is invoked when a high-availability event occurs.

Note: For details, see the *Oracle Call Interface Programmer's Guide*.

Automating Failover for OLE DB Clients

1. On the database instance, ensure Data Guard broker management of the configuration.
2. Execute the `DBMS_SERVICE.CREATE_SERVICE` procedure or `SRVCTL` to create the database service, enable high availability notification, and configure server-side TAF settings.
3. Create a trigger on the system startup event to relocate the database service after a role transition if needed.

The Oracle logo, consisting of the word "ORACLE" in white, uppercase letters on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To automate failover for OLE DB clients, perform the following steps to configure your database:

1. Ensure that your configuration is managed by the Data Guard broker.
2. Execute the `DBMS_SERVICE.CREATE_SERVICE` procedure or `SRVCTL` to create the database service, enable high-availability notification, and configure server-side TAF settings.
3. Create a trigger that fires on the system startup event if needed.
This trigger relocates the database after a role transition to the service that was created in step 2.

Configuring OLE DB Clients for Failover

1. Set the `DBNotifications` and `DBNotificationPort` `OraOLEDB` connection string attributes.
2. Set the `SQLNET.OUTBOUND_CONNECT_TIMEOUT` parameter in the client `sqlnet.ora` file to a value of 3 seconds.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a solid red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To configure OLE DB clients to receive notification of FAN high-availability events:

1. Set the following `OraOLEDB` connection string attributes:
 - a. `DBNotifications = true`
 - b. `DBNotificationPort = [unsigned integer]`
Setting the `DBNotificationPort` attribute allows the port to be specified. If this attribute is not set, the port is randomly selected.
2. Set the `SQLNET.OUTBOUND_CONNECT_TIMEOUT` parameter in the `sqlnet.ora` file to a value of 3 seconds.

This parameter enables clients to quickly traverse an address list if a failure occurs. If a client attempts to connect to a host that is unavailable, the connection attempt is bounded to the time specified by the `SQLNET.OUTBOUND_CONNECT_TIMEOUT` parameter, after which the client attempts to connect to the next host in the address list. This behavior continues for each host in the address list until a successful connection is made.

Automating Failover for JDBC Clients

1. On the database instance, execute the `DBMS_SERVICE.CREATE_SERVICE` procedure or `SRVCTL` to create the database service.
2. Configure ONS in the `$ORACLE_HOME/opmn/conf` directory.
3. Start the ONS daemon.
4. Create a trigger on the system startup event to relocate the database service after a role transition if needed.
5. Create a trigger on the `DB_ROLE_CHANGE` system event if needed.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To automate failover for JDBC clients, perform the following steps to configure your database:

1. Execute the `DBMS_SERVICE.CREATE_SERVICE` procedure or `SRVCTL` to create the database service for JDBC clients.
Because JDBC clients use FCF rather than TAF, database services for JDBC clients are not configured for AQ HA events. Instead, a trigger is required to notify JDBC clients when a Data Guard failover occurs.
2. Configure and start ONS daemons on all hosts that may contain a primary database. Configure ONS in the `$ORACLE_HOME/opmn/conf` directory (similar to the example in the slide). See the *Oracle Database JDBC Developer's Guide and Reference* for details.
3. Start the ONS daemon.
4. Create a trigger on the system startup event to relocate the database service after a role transition if needed. The trigger is not needed when using Oracle Restart.
5. Create a trigger enabled for the `DB_ROLE_CHANGE` system event that calls a C program named the FAN ONS Publisher.
This trigger is required because the primary host where the ONS daemons reside are no longer available. By calling the FAN ONS Publisher program based on a trigger enabled on the `DB_ROLE_CHANGE` system event, JDBC clients are notified of the primary site failure and instructed to reconnect to the new primary database. See the white paper titled "Client Failover Best Practices for Highly Available Oracle Databases: Oracle Database 11g Release 2" for information about the FAN ONS Publisher.

Configuring JDBC Clients for Failover

1. Set the `FastConnectionFailoverEnabled` `DataSource` property to `True`.
2. Set the `oracle.net.ns.SQLnetDef.TCP_CONNTIMEOUT_STR` property on the data source to a value of 3 seconds.
3. Create an Oracle Net service name that includes the primary database host and all standby database hosts in the address list.
4. Configure a remote ONS subscription on the JDBC client.
5. Enable SSL for communications.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To configure JDBC clients for failover:

1. Set the `FastConnectionFailoverEnabled` `DataSource` property to `True` so that the client application uses implicit JDBC connection cache on its data source.
2. Set the `oracle.net.ns.SQLnetDef.TCP_CONNTIMEOUT_STR` property to a value of 3 seconds on the data source.
This property enables the JDBC client to quickly traverse an address list in the event of a failure. If the client attempts to connect to a host that is unavailable, the connection attempt is bounded to the time specified by the `SQLnetDef.TCP_CONNTIMEOUT_STR` property, after which the client attempts to connect to the next host in the address list. The behavior continues for each host in the address list until a successful connection is made.
3. Create an Oracle Net service name that includes an `ADDRESS` entry for the primary database host and all standby database hosts.
4. Configure a remote ONS subscription on the JDBC client so that an ONS daemon is not required on the client. The remote ONS subscription should contain all hosts that have the potential to become a primary database.
5. Enable SSL for communications.
SSL should be used for all ONS communications.

Quiz

In a Real Application Cluster (RAC) environment, which method should you generally use to manage role-based services?

- a. DBMS_SERVICE
- b. SRVCTL

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: b

Quiz

FAN events are not enabled or started during the installation of Grid Infrastructure for Standalone Servers for Data Guard.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: a

Summary

In this lesson, you should have learned how to:

- Configure client connectivity in a Data Guard configuration
- Implement failover procedures to automatically redirect clients to a new primary database

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 16: Overview

This practice covers the following topics:

- Creating a service
- Creating Oracle Net service names
- Testing your implementation

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Patching and Upgrading Databases in a Data Guard Configuration

17

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Objectives

After completing this lesson, you should be able to patch and upgrade databases in your Data Guard configuration:

- By using Standby-First Patch Apply technique
- By using traditional upgrade methods
- By performing rolling upgrades
- By using the `DBMS_ROLLING` package with Active Data Guard

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The methods outlined in this lesson are applicable for a patch, a critical patch update (CPU), a patch set, or a major release.

Data Guard Standby-First Patch Apply

Provides support for different software releases between a primary database and physical standby database for the purpose of applying and validating patches in a rolling fashion

- The `COMPATIBLE` RDBMS parameter must remain the same on all systems.
- The database patch release dates must be no more than one year apart.
- The patches must be within six versions of each other.
- Data Guard role transitions are allowed between different releases on the primary and physical standby databases for Standby-First Patch.
- The patch must be certified as being a “Standby-First” patch

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A typical database patch consists of two main steps: step 1) patch installation consisting of patching the Oracle Database Home or Grid Infrastructure Home or both, and step 2) patch post-installation consisting of running SQL scripts or SQL commands. A Standby-First Patch only executes step 1 to a physical standby environment. To finalize the patch, both step 1 and step 2 need to be executed on the primary database, because step 2 requires a read-write database. All changes from step 2 are replicated via the redo stream to the physical standby database.

Oracle Data Guard Standby-First Patch Apply provides a supported method to apply a patch initially to a physical standby database while the primary database remains at the previous software release.

Oracle Data Guard Standby-First Patch Apply is supported between database patch releases between the primary and standby that are a maximum of one year apart from the patch release dates or the patches are within 6 versions of each other. For example (using 11.2.0.2 versions, because there are no valid Oracle Database 12c examples currently), the primary database at 11.2.0.2 Bundle Patch 8 (updated 25-Apr-2011) and standby at 11.2.0.2 Bundle Patch 16 (update 17-Apr-2012) barely qualifies. The Data Guard software combination is a viable candidate because the bundle patches are less than 1 year apart and also within "n-6" and 11.2.0.2 BP16 patch readme states “This patch is Data Guard Standby-First Installable.”

Data Guard Standby-First Patch Apply

The following types of patches are supported:

- Oracle Exadata Database Machine bundled patch or Quarter Database Patch for Exadata
- Oracle Exadata Storage Server Software Update
- Oracle Exadata Database Machine hardware or network changes
- Oracle Grid Infrastructure Patches or Software Updates
- Patch Set Update (PSU)
- Critical Patch Update (CPU)
- Patch Set Exception (PSE)
- Operating System software changes that do not have any dependencies on Oracle database software

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is positioned on the right side of a solid red horizontal bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle patch sets and major release upgrades do not apply. For example, upgrades from 11.2.0.2 to 11.2.0.3 or 11.2 to 12.1 do not qualify for standby-first patch apply. Use the Data Guard transient logical standby method for patch sets and major releases.

Refer to the README file for the patch to determine whether a target patch is certified as being a “Standby-First” or “DG Rolling Installable” patch.

Data Guard Standby-First Patch Apply

Steps to apply the patch to the standby database first:

1. Shut down all standby instances on the standby database (if patch is not RAC Rolling).
2. On the standby site, apply the patch as described in the patch README.
3. Restart the standby instances.
4. Restart media recovery on the physical standby database.
5. Evaluate the patch on the standby by monitoring alert and log files, using snapshot standby, or Active Data Guard.
6. Apply the patch to the primary database site.
 - Standby can be read-only if the patch is RAC Rolling capable.
 - Standby must be mounted if the patch is not RAC Rolling capable.
 - Alternatively, switch over to standby and apply the patch to the former primary.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Data Guard can be managed by SQL*Plus or the broker when using the Standby-First Patch Apply technique. The patch evaluation can be performed by any or all of the following methods:

- Leave the standby database in managed recovery mode at the mount state, and monitor for any issues in the standby alert log and trace files.
- If using the Active Data Guard option, open the standby database in read-only mode and stress the standby database by running your read-only workload.
- Convert the standby database into a snapshot standby, as described in the *Oracle Data Guard Concepts and Administration Guide*, execute patch post-installation step 2, and test the application against the snapshot standby. Optionally, use Oracle Real Application Testing to perform a full application evaluation. This evaluation method is the most comprehensive and is the Oracle recommended best practice. Once testing is complete, convert the snapshot standby back to a physical standby.

Patches that are listed as RAC Rolling Installable in the patch README can be applied on the primary with the standby performing recovery in read-only mode. However, for patches that are not RAC Rolling Installable, you must stop read-only recovery on the standby, bring the standby database to the mount state, and restart recovery prior to applying the patch to the primary database.

Upgrading an Oracle Data Guard Broker Configuration

- If you are using the Data Guard broker, you must perform the following steps before upgrading your databases:

1. Disable broker management of the configuration.

```
DGMGRL> DISABLE CONFIGURATION;
```

2. Stop the Data Guard broker.

```
SQL> ALTER SYSTEM SET DG_BROKER_START=FALSE;
```

- After completing the upgrade:

1. Start the Data Guard broker.

```
SQL> ALTER SYSTEM SET DG_BROKER_START=TRUE;
```

2. Enable broker management of the configuration by connecting to the primary database.

```
DGMGRL> ENABLE CONFIGURATION;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle patch sets and major releases require an upgrade method. To upgrade the databases in your Data Guard configuration:

1. Disable broker management of the databases in the Data Guard configuration by executing the following DGMGRL command:

```
DGMGRL> DISABLE CONFIGURATION;
```

2. Execute the following SQL*Plus statement to stop the broker:

```
SQL> ALTER SYSTEM SET DG_BROKER_START=FALSE;
```

After completing the upgrade:

1. Start the Data Guard broker by executing the following command in SQL*Plus:

```
SQL> ALTER SYSTEM SET DG_BROKER_START=TRUE;
```

2. Enable broker management of the configuration by connecting to the primary database and executing the following DGMGRL command:

```
DGMGRL> ENABLE CONFIGURATION
```

Upgrading Oracle Database in a Data Guard Configuration with a Physical Standby Database

To perform a traditional upgrade:

1. Perform the preparation steps for an upgrade.
2. Install the Oracle Database software on physical standby database systems and the primary database system.
3. Shut down the primary database.
4. Shut down the physical standby database(s).
5. Stop all listeners, agents, and other processes running in the Oracle homes that are to be upgraded.
6. If Oracle Automatic Storage Management (ASM) is in use, it should be upgraded before the databases.
7. In the new Oracle home, restart all listeners, agents, and other processes stopped in step 5.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is centered within a solid red rectangular bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Perform the steps in the slide to upgrade to Oracle Database 12c Release 1 (12.1) when your Data Guard configuration contains one or more physical standby databases.

Note: For detailed information about these steps, see the *Oracle Database Upgrade Guide* (including the “Preparing to Upgrade” chapter).

Upgrading Oracle Database in a Data Guard Configuration with a Physical Standby Database

8. Mount the physical standby database(s) on the new Oracle home.
9. Start Redo Apply on the physical standby database(s).
10. Upgrade the primary database as described in the *Oracle Database Upgrade Guide*. Redo Transport will propagate this to the standby database(s).
11. Open the upgraded primary database.
12. If Active Data Guard was being used prior to the upgrade, it will need to be reenabled.
13. Optionally modify the `COMPATIBLE` initialization parameter.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a solid red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Perform the steps in the slide to upgrade to Oracle Database 12c Release 1 (12.1) when your Data Guard configuration contains one or more physical standby databases.

Note: For detailed information about these steps, see the *Oracle Database Upgrade Guide* (including the “Preparing to Upgrade” chapter).

Upgrading Oracle Database in a Data Guard Configuration with a Logical Standby Database

To perform a traditional upgrade:

1. Perform the preparation steps for an upgrade.
2. Set the data protection mode to `MAXIMUM PERFORMANCE`.
3. Stop all user activity on the primary database and defer the logical standby database remote archival destination.
4. Stop SQL Apply on the standby database.
5. Install Oracle Database software on the primary database system and upgrade the primary database.
6. Install Oracle Database software on the logical standby database system and upgrade the standby database.
7. Restart SQL Apply on the standby database.
8. Open the upgraded primary database.
9. Reset the data protection mode, if necessary.

ORACLE

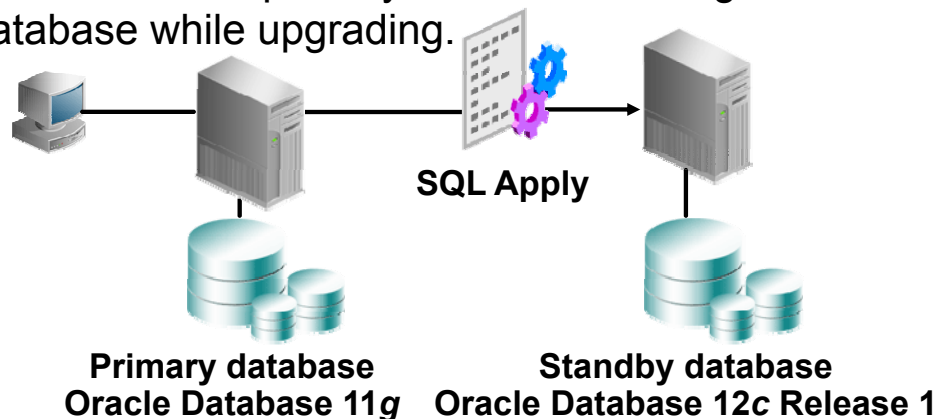
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Perform the steps in the slide to upgrade to Oracle Database 12c Release 1 (12.1) when your Data Guard configuration contains a logical standby database.

Note: For detailed information about these steps, see the *Oracle Database Upgrade Guide* (including the “Preparing to Upgrade” chapter).

Using SQL Apply to Upgrade the Oracle Database

- Use a logical standby database to perform a rolling upgrade of Oracle Database 11g software from patch set release n to any higher-versioned patch set or major version release.
- Incur minimal down time by using different releases of Oracle Database on the primary database and logical standby database while upgrading.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can use a logical standby database to perform a rolling upgrade of Oracle Database 11g software to Oracle Database 12c. During a rolling upgrade, different releases of Oracle Database can be on the primary database and logical standby databases while you upgrade them one at a time, incurring minimal down time on the primary database.

Using Data Guard SQL Apply, you can perform a rolling upgrade of the Oracle Database software from patch set release n to any higher-versioned patch set or major version release.

Note: When upgrading from Oracle Database 12c Release 1 to Oracle Database 12c Release 2, the preferred method is to use the `DBMS_ROLLING` package.

Requirements for Using SQL Apply to Perform a Rolling Upgrade

- The broker configuration should be disabled before a rolling upgrade.
- Data Guard protection mode must be maximum availability or maximum performance.
- The `LOG_ARCHIVE_DEST_n` initialization parameter for the logical standby destination must *not* be set to `MANDATORY`.
- The `COMPATIBLE` initialization parameter value must match the software release prior to the upgrade.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Prior to performing the rolling upgrade, complete the following requirements:

- Disable the broker configuration.
- Set the Data Guard protection mode to either maximum availability or maximum performance.
- The `LOG_ARCHIVE_DEST_n` initialization parameter for the logical standby destination must *not* be set to `MANDATORY`.
- Set the `COMPATIBLE` initialization parameter so that it matches the software release prior to the upgrade. A rolling upgrade from release x to release y requires that the `COMPATIBLE` initialization parameter be set to release x on the primary database and the standby database.

Performing a Rolling Upgrade by Using SQL Apply

- Use an existing logical standby database to perform the rolling upgrade.
- Create a new logical standby database to perform the rolling upgrade.
- Use an existing physical standby database to perform the rolling upgrade.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can use SQL Apply to perform a rolling upgrade in several different configurations. Each of the configurations is outlined in detail in this lesson.

Identifying Unsupported Data Types

Identify data types and storage attributes on the primary database that are not supported in a logical standby database:

- Query `DBA_LOGSTDBY_UNSUPPORTED` and `DBA_LOGSTDBY_SKIP`.
 - Use the `CDB_*` views with multitenant architecture.
- Determine how to handle unsupported objects during the rolling upgrade.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You identify data types and storage attributes on the primary database that are not supported in a logical standby database by querying the `DBA_LOGSTDBY_UNSUPPORTED` and `DBA_LOGSTDBY_SKIP` views on the primary database. If your primary database contains unsupported objects, you may be able to perform the upgrade by temporarily suspending changes to the unsupported tables for the period of time it takes to perform the upgrade procedure.

If you cannot prevent changes to unsupported tables during the upgrade, you may be able to use Oracle Data Pump or the Import utility to import the changed tables to the upgraded databases. Unsupported transactions that occur are recorded in the `DBA_LOGSTDBY_EVENTS` table on the logical standby database.

Logical Standby: New Data Type Support for Oracle Database 12c

Support for more data types helps eliminate the barriers to utilizing database rolling upgrades with transient logical standbys. New types include:

- Abstract data types (ADTs) and ADT tables
- Database file system (DBFS)
- LOBs stored as SecureFiles
- Objects stored as VARRAYs (except for Collections)
- Oracle SecureFiles (deduplication)
- Oracle Text
- Spatial and multimedia (with exceptions)
- User-defined types
- XDB

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

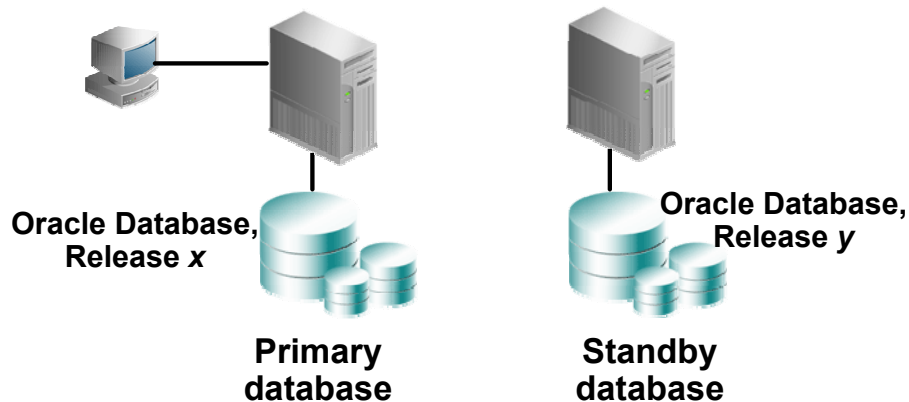
The rolling upgrade process converts a physical standby database to a transient logical standby database. While physical standby databases support all Oracle Database data types, a logical standby database has restrictions on supported data types. If the primary database uses the restricted data types, the Data Guard rolling upgrade process may not be feasible.

To help eliminate this barrier for using Data Guard to perform rolling upgrades with minimal down time, Data Guard supports the following data types in a logical standby database:

- Abstract data types (ADTs) and ADT tables
- Database file system (DBFS)
- LOBs stored as SecureFiles
- Objects stored as VARRAYs (except for Collections)
- Oracle SecureFiles (deduplication)
- Oracle Text
- Spatial (except `MDSYS.SDO_GEORASTER` and `MDSYS.SDO_TOPO_GEOMETRY`) and multimedia (Opaque type restrictions and `REFs` are not supported)
- User-defined types
- XDB

Performing a Rolling Upgrade by Using an Existing Logical Standby Database

1. Prepare for the rolling upgrade by stopping SQL Apply and setting the `COMPATIBLE` initialization parameter.
2. Upgrade the Oracle Database software on the logical standby database and upgrade the logical standby database.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

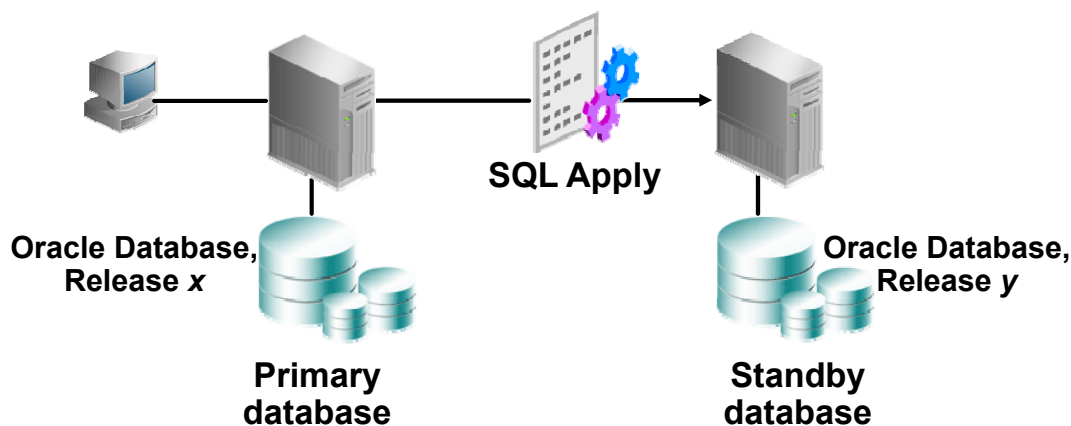
To perform a rolling upgrade by using a logical standby database in your Data Guard configuration:

1. Prepare for the rolling upgrade as follows:
 - a. Stop SQL Apply by issuing the following statement on the logical standby database:


```
ALTER DATABASE STOP LOGICAL STANDBY APPLY;
```
 - b. Set the `COMPATIBLE` initialization parameter to the highest value. Ensure that the `COMPATIBLE` initialization parameter specifies the release number for the Oracle Database software running on the primary database prior to the upgrade.
2. Upgrade the Oracle Database software on the logical standby database to release y. While the logical standby database is being upgraded, it does not accept redo data from the primary database. See the *Oracle Database Upgrade Guide* for detailed information.

Performing a Rolling Upgrade by Using an Existing Logical Standby Database

3. Restart SQL Apply on the upgraded logical standby database.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

3. Restart SQL Apply by executing the following statement on the standby database:

```
ALTER DATABASE START LOGICAL STANDBY APPLY IMMEDIATE;
```

The redo data that was accumulating on the primary database system is automatically transmitted and applied on the logical standby database. To monitor how quickly the logical standby database is catching up to the primary database, query the `V$LOGSTDBY_PROGRESS` view on the logical standby database. For example:

```
SQL> ALTER SESSION SET NLS_DATE_FORMAT = 'DD-MON-YY HH24:MI:SS';
Session altered.
```

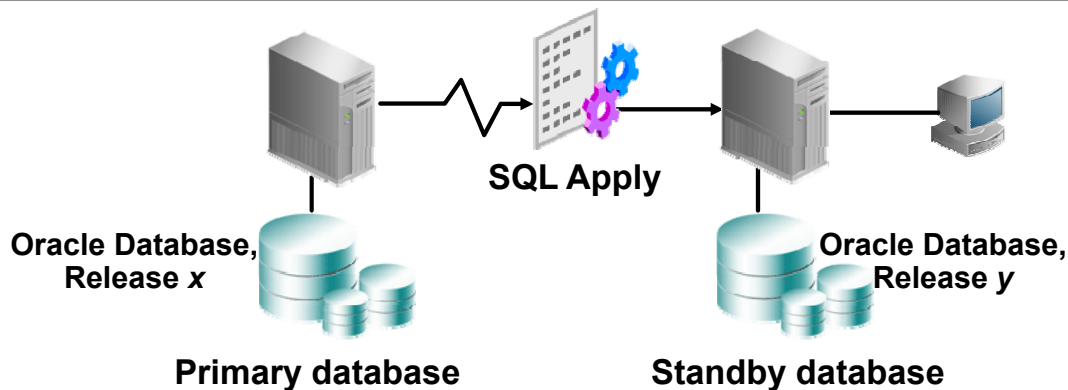
```
SQL> SELECT SYSDATE, APPLIED_TIME FROM V$LOGSTDBY_PROGRESS;
SYSDATE                APPLIED_TIME
```

```
-----
27-FEB-10 17:07:06 27-FEB-10 17:06:50
```


Performing a Rolling Upgrade by Using an Existing Logical Standby Database

4. Query `DBA_LOGSTDBY_EVENTS` to monitor the upgraded logical standby database.
5. Begin a switchover to the upgraded logical standby database:

```
SQL> ALTER DATABASE COMMIT TO SWITCHOVER TO LOGICAL
      STANDBY;
```



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

4. Query `DBA_LOGSTDBY_EVENTS` to determine whether there are any DDL and DML statements that were not applied on the logical standby database:


```
SELECT EVENT_TIMESTAMP, EVENT, STATUS
FROM DBA_LOGSTDBY_EVENTS
ORDER BY EVENT_TIMESTAMP;
```
5. Begin a switchover to the upgraded logical standby database by executing the following statement on the primary database:

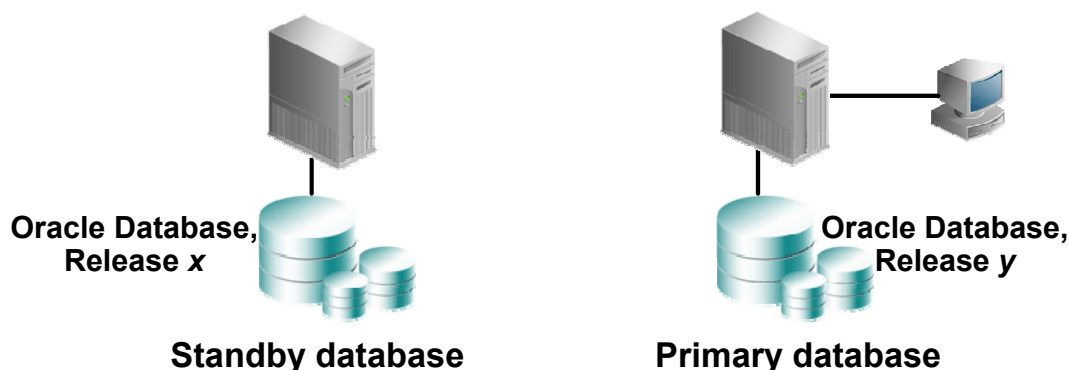

```
ALTER DATABASE COMMIT TO SWITCHOVER TO LOGICAL STANDBY;
```

This statement waits for existing transactions to complete. To minimize the time it takes to complete the switchover, users connected to the primary database should log off immediately and reconnect to the standby database.

Performing a Rolling Upgrade by Using an Existing Logical Standby Database

6. Import any tables that were unsupported and modified.
7. Complete the switchover and activate user applications:

```
SQL> ALTER DATABASE COMMIT TO SWITCHOVER TO LOGICAL  
PRIMARY;
```



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

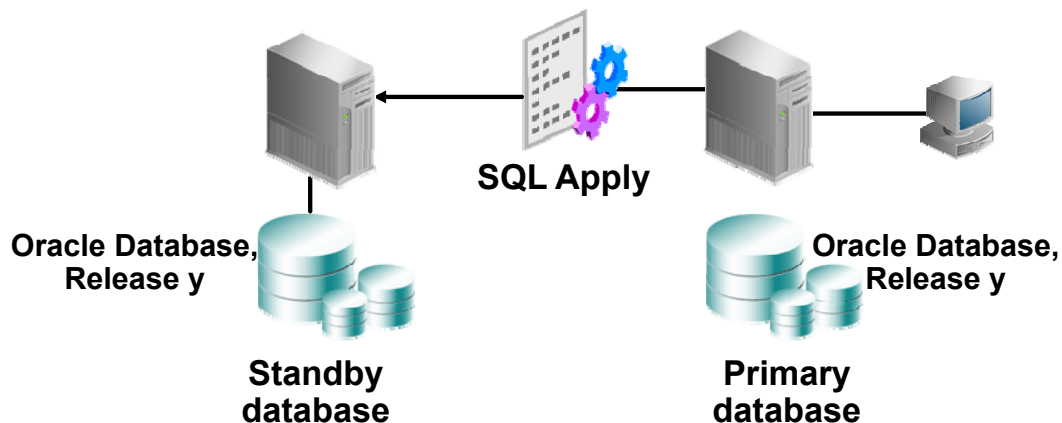
6. Import any tables that were modified during the upgrade from the primary database that were unsupported in the logical standby database.
7. Complete the switchover by executing the following statement on the logical standby database:

```
ALTER DATABASE COMMIT TO SWITCHOVER TO LOGICAL PRIMARY;
```

After the switchover, you cannot send redo data from the new primary database (using the new Oracle Database software release) to the new standby database (using the older Oracle Database software release).

Performing a Rolling Upgrade by Using an Existing Logical Standby Database

8. Upgrade the original primary database.
9. Start SQL Apply on the original primary database.



ORACLE

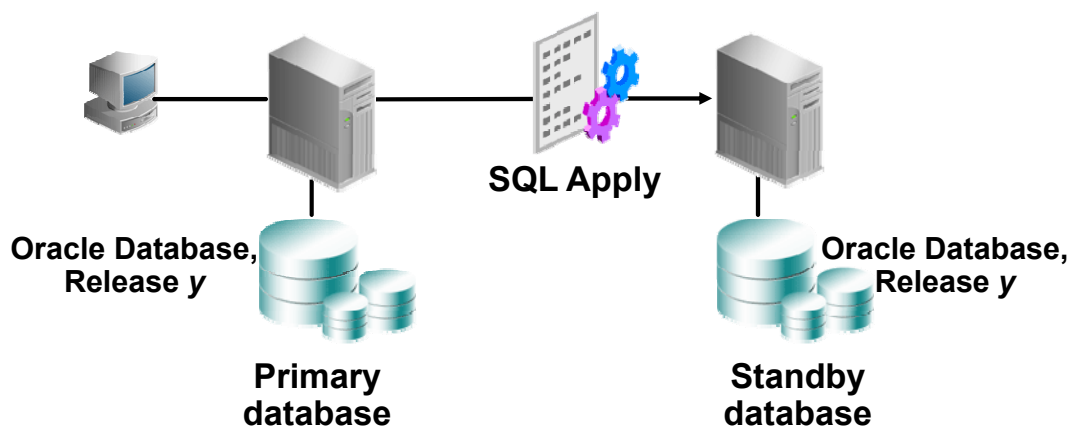
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

8. Upgrade the Oracle Database software on the original primary database to release y. See the *Oracle Database Upgrade Guide* for detailed information.
9. Start SQL Apply on the original primary database. You may also need to create a database link to the new primary database:

```
ALTER DATABASE START LOGICAL STANDBY APPLY IMMEDIATE
NEW PRIMARY db_link;
```

Performing a Rolling Upgrade by Using an Existing Logical Standby Database

10. Monitor events on the new logical standby database.
11. Optional: Perform a switchover to return to the original configuration.
12. Optional: Raise the compatibility level on both databases.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

10. Monitor events on the new logical standby database by querying the `DBA_LOGSTDBY_EVENTS` view.
11. **Optional:** Perform a switchover to return to the original configuration.
12. **Optional:** Raise the compatibility level on both databases by setting the `COMPATIBLE` initialization parameter on the standby database before you set it on the primary database.

Performing a Rolling Upgrade by Creating a New Logical Standby Database

1. Identify data types and storage attributes on the primary database that are not supported in a logical standby database.
2. Create a logical standby database.
3. Perform a rolling upgrade.

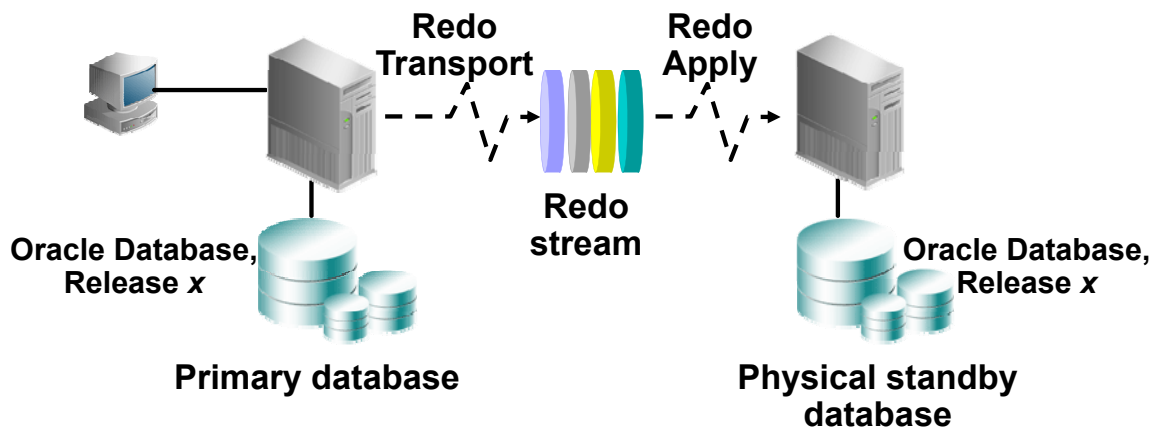
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

1. Identify data types and storage attributes on the primary database that are not supported in a logical standby database (as described earlier in this lesson).
2. Create a logical standby database (as described in the lesson titled “Creating a Logical Standby Database”).
3. Perform a rolling upgrade (as described earlier in this lesson).

Performing a Rolling Upgrade by Using a Physical Standby Database

1. Prepare the primary database for a rolling upgrade.
 - a. Enable Flashback Database.
 - b. Create a guaranteed restore point.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Note: Use the following technique only when you are upgrading an Oracle Database 11g database to a later release.

To perform a rolling upgrade when your configuration contains a physical standby database:

1. Prepare the primary database for a rolling upgrade:
 - a. Enable Flashback Database.
 - b. Create a guaranteed restore point:

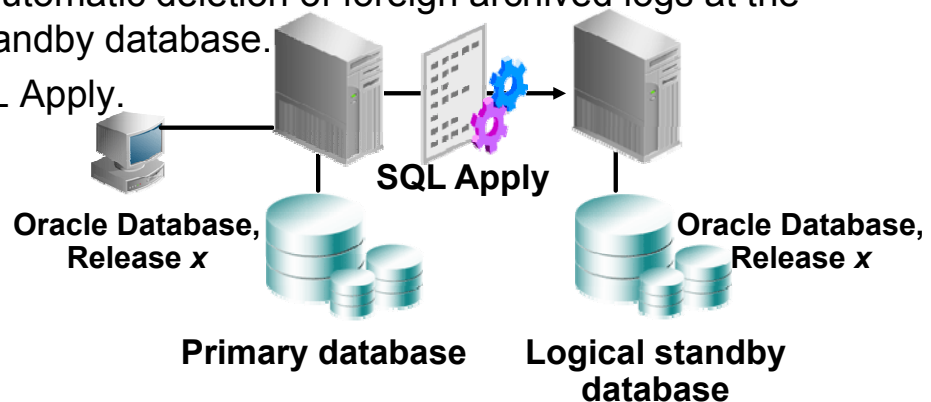
```
CREATE RESTORE POINT pre_upgrade GUARANTEE FLASHBACK
DATABASE;
```

Performing a Rolling Upgrade by Using a Physical Standby Database

2. Convert the physical standby database to a logical standby database.
 - a. Create a logical standby database and execute:

```
SQL> ALTER DATABASE RECOVER TO LOGICAL STANDBY KEEP IDENTITY;
```

- b. Disable automatic deletion of foreign archived logs at the logical standby database.
 - c. Start SQL Apply.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

2. Convert the physical standby database to a logical standby database. This is only temporarily done for the duration of the rolling upgrade.
 - a. Create a logical standby database and execute the following command:


```
ALTER DATABASE RECOVER TO LOGICAL STANDBY KEEP IDENTITY;
```

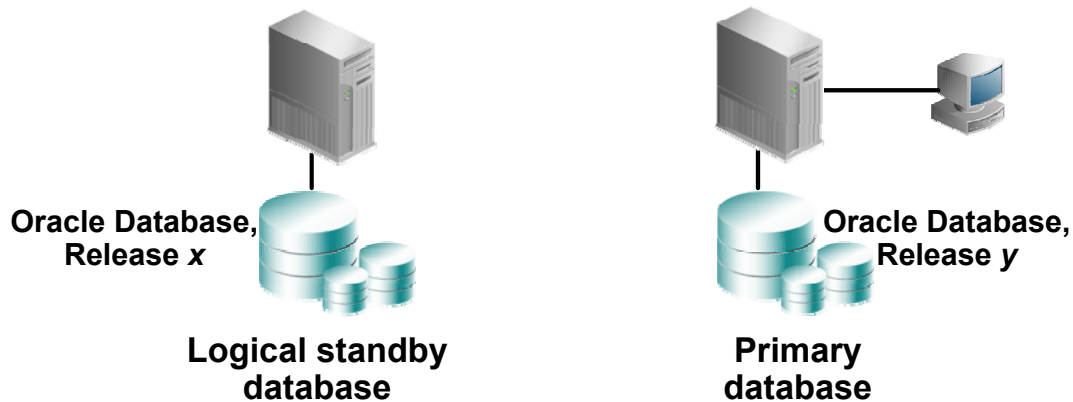
Note: A logical standby database created with the KEEP IDENTITY clause retains the same DB_NAME and DBID as those of its primary database.
 - b. Disable automatic deletion of foreign archived logs at the logical standby database:


```
execute DBMS_LOGSTDBY.APPLY_SET('LOG_AUTO_DELETE', 'FALSE');
```
 - c. Start SQL Apply:


```
ALTER DATABASE START LOGICAL STANDBY APPLY IMMEDIATE;
```

Performing a Rolling Upgrade by Using a Physical Standby Database

3. Upgrade the logical standby database (steps 1–7 of “Performing a Rolling Upgrade by Using a Logical Standby Database”).



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

After completing these steps, your original primary database will be the logical standby database and your original physical standby database will be your primary database with an upgraded version of the Oracle Database software.

Performing a Rolling Upgrade by Using a Physical Standby Database

- Flash back the original primary database to the guaranteed restore point:

```
SQL> SHUTDOWN IMMEDIATE
SQL> STARTUP MOUNT
SQL> FLASHBACK DATABASE TO RESTORE POINT pre_upgrade;
SQL> SHUTDOWN IMMEDIATE
```

- Mount the original primary database using the new version of the software:

```
SQL> startup mount
```

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

- Flash back the original primary database to the guaranteed restore point that you created in step 1.
- Mount the original primary database using the new version of the software. You will not run the upgrade scripts, because this database will be turned into a physical standby, and will be upgraded automatically as it applies redo data generated from the new primary database.

Performing a Rolling Upgrade by Using a Physical Standby Database

6. Convert the original primary database to a physical standby:

```
SQL> ALTER DATABASE CONVERT TO PHYSICAL STANDBY;  
SQL> SHUTDOWN IMMEDIATE;
```

7. Start managed recovery on the original primary database:

```
SQL> STARTUP MOUNT;  
SQL> ALTER DATABASE RECOVER MANAGED STANDBY DATABASE  
      USING CURRENT LOGFILE DISCONNECT FROM SESSION;
```

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is positioned on a solid red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

6. Convert the original primary database to a physical standby database:
ALTER DATABASE CONVERT TO PHYSICAL STANDBY;
SHUTDOWN IMMEDIATE;
7. Start the managed recovery process on the original primary database:
ALTER DATABASE RECOVER MANAGED STANDBY DATABASE
USING CURRENT LOGFILE DISCONNECT FROM SESSION;

Performing a Rolling Upgrade by Using a Physical Standby Database

8. Perform a switchover to return your original primary database to the primary database role. (Optional)
9. Clean up the guaranteed restore point created in step 1.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

8. To perform a switchover to return to your original configuration, execute the following commands on the new primary database:

```
ALTER DATABASE COMMIT TO SWITCHOVER TO PHYSICAL STANDBY WITH
SESSION SHUTDOWN;
SHUTDOWN IMMEDIATE;
STARTUP MOUNT;
```

On the original primary database, execute the following commands:

```
ALTER DATABASE COMMIT TO SWITCHOVER TO PRIMARY;
SHUTDOWN IMMEDIATE;
STARTUP
```

9. Clean up the guaranteed restore point created in step 1.
- ```
DROP RESTORE POINT PRE_UPGRADE;
```

## Rolling Upgrades Using DBMS\_ROLLING and Active Data Guard

- Database software upgrades using the DBMS\_ROLLING PL/SQL package will be usable starting with the first patchset of Oracle Database 12c to the second patchset.
- Database upgrades from Oracle Database 11g to Oracle Database 12c will need to use the classic Transient Logical Standby upgrade procedure.
- The DBMS\_ROLLING PL/SQL package can be used for other database maintenance tasks with the first release of Oracle Database 12c, such as:
  - Adding partitioning to nonpartitioned tables
  - Changing BasicFiles LOBs to SecureFiles LOBs
  - Changing XML-CLOB to binary XML
  - Altering a table to be OLTP-compressed

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Rolling Upgrade Using Active Data Guard feature, new as of Oracle Database 12c Release 1 (12.1), provides a streamlined method of performing rolling upgrades. It is implemented using the new DBMS\_ROLLING PL/SQL package, which allows you to upgrade the database software in a Data Guard configuration in a rolling fashion. The Rolling Upgrade Using Active Data Guard feature requires the Oracle Active Data Guard option.

You can use this feature for database version upgrades starting with the first patchset of Oracle Database 12c. This means that the manual Transient Logical Standby upgrade procedure must still be used when upgrading from Oracle Database 11g to Oracle Database 12c, or when upgrading from the initial Oracle Database 12c release to the first patchset of Oracle Database 12c.

Additionally, you can use this feature immediately for other database maintenance tasks beginning with Oracle Database 12c Release 1 (12.1). The database where maintenance is performed must be operating at a minimum of Oracle 12.1. Such maintenance tasks include adding partitioning to nonpartitioned tables, changing BasicFiles LOBs to SecureFiles LOBs, changing XMLType stored as CLOB to XMLType stored as binary XML, and altering tables to be OLTP-compressed.

## DBMS\_ROLLING: Concepts

- Rolling changes can be applied on the whole Data Guard configuration.
  - Three stages: Specification, Compilation, and Execution
  - Execution has three phases: Start, Switchover, and Finish
- Two key groups:
  - Leading group (LG)
    - These databases are upgraded first (before switchover).
    - The LG has a master database (the future primary database).
  - Trailing group (TG)
    - These databases are upgraded last (after switchover).
    - The TG has a master database (the original primary database).

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Rolling changes can be applied to the whole Data Guard configuration using all standby databases identified by the `LOG_ARCHIVE_CONFIG` parameter. There are three stages to the rolling upgrade process using the `DBMS_ROLLING` PL/SQL package: Specification, Compilation, and Execution. In the Specification stage, you identify how you want to implement the rolling upgrade process and designate which standby will become the future primary database. The Compilation stage builds an upgrade plan by performing validation checks and reports any errors that require corrective actions before the Execution stage begins. The Execution stage performs the actual upgrade and has three phases to it: Start, Switchover, and Finish.

Conceptually, the rolling upgrade process splits the Data Guard configuration into two groups: the leading group (LG) and the trailing group (TG). The leading group contains a master database that is designated to be the future primary database. The leading group can contain additional standby databases designed to protect the master database during the upgrade process. Databases in the leading group are upgraded first before switchover. The trailing group contains the original primary database along with any additional databases designed to protect it during the upgrade process. Databases in the trailing group are upgraded last after the switchover is performed.

## DBMS\_ROLLING: Concepts

- Leading group
  - The Leading Group Master database (LGM) must be identified during Specification.
  - The LGM starts as a physical standby, converted into a logical standby (START), and then becomes the primary database (SWITCHOVER).
  - Other databases in the leading group protect the LGM.
  - LGM responsibility is transferrable on failure.
- Trailing group
  - The TG contains the original primary database (Trailing Group Master [TGM]).
  - Other databases in the Trailing Group protect the TGM.
  - TGM responsibility is transferrable on failure.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The leading group contains the designated future primary database, known as the Leading Group Master database (LGM), and the physical standbys that you can configure to protect the designated future primary. All other standbys in the leading group can only be physical standbys. The LGM is first converted into a logical standby database and then the new database software is installed on it and the upgrade process is run. Other standby databases in the leading group also must have their software upgraded at this point. In the event of a failure during the upgrade process, you can fail over to any of the additional standby databases in the leading group and then designate the failover target database to take over the role of the LGM.

The trailing group contains the original primary database known as the Trailing Group Master database (TGM) and standby databases that will protect the original primary during the rolling upgrade process. While the databases in the leading group are going through the upgrade process, user applications can still be connected to the original primary and making changes. The trailing group databases continue running the old database software until all the databases in the leading group are upgraded and the future primary has caught up with the original primary by applying the changes that were generated at the original primary database during the upgrade window. New software is then installed on the databases that are part of the trailing group after switchover, and they are reinstated into the configuration as standbys to the new primary database. The role of TGM can be transferred to other standby databases in the trailing group in the event a failure occurs during the upgrade.

## DBMS\_ROLLING: Key Features

- Specify – Compile – Execute protocol
  - Configuration errors are detected during the Compilation stage.
  - Runtime errors are detected during the Execution stage.
- State is kept in the database.
  - Provides robustness
- Runtime steps are constant.
  - Regardless of how many databases are involved
- Handles failure at the original primary database
- Allows data protection for the upgraded primary right from the start

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The DBMS\_ROLLING package provides for a Specify – Compile - Execute protocol, all driven from a centralized interface using procedures. Configuration errors that would prevent execution of the upgrade plan can be identified during the Compilation stage and resolved before beginning the actual upgrade. DBA\_ROLLING\_\* views provide diagnostic information should any runtime errors occur during the Execution stage.

Plan parameters are persisted in the database, and remain even after completion of a rolling upgrade. After the rolling upgrade has been successfully executed, you can remove your rolling upgrade specification by calling the DBMS\_ROLLING.DESTROY\_PLAN procedure.

In the previous versions of Oracle Data Guard, a rolling upgrade using a transient logical standby database required at least 42 steps for a simple two-database configuration. The number of steps would significantly increase for each additional standby database contained in the Data Guard configuration. Using the DBMS\_ROLLING package, the runtime steps of the Execution stage are constant, regardless of how many standby databases exist.

The concept of leading group standbys and trailing group standbys allow for multiple databases in each group to provide fault tolerance for the group. In addition, the additional databases, if present, can provide protection for the upgraded primary immediately following the upgrade.

# Database Rolling Upgrade: Specification and Compilation Stages

Generate an *upgrade plan*:

- Specify parameters of the rolling upgrade, such as target software versions, participating databases, apply lag requirements, and logging levels.
  - `DBMS_ROLLING.INIT_PLAN` procedure
  - `DBMS_ROLLING.SET_PARAMETER` procedure
  - `DBA_ROLLING_PARAMETERS` view
- Use the `DBMS_ROLLING.BUILD_PLAN` procedure to generate an upgrade plan and perform validations.
- Use the `DBA_ROLLING_PARAMETERS`, `DBA_ROLLING_PLAN`, and `DBA_ROLLING_EVENTS` views to display the current plan and diagnose any problems with the plan.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Specification and Compilation stages involve the following six steps to create a centralized upgrade plan that drives the rolling upgrade process:

1. Initialize the upgrade parameters.
2. View the current upgrade parameters.
3. Modify the upgrade parameter values as necessary.
4. Build the upgrade plan.
5. View the current plan.
6. Revise the upgrade plan as necessary.

The `DBMS_ROLLING.INIT_PLAN` procedure generates system default parameters for all databases specified in the `DG_CONFIG` parameter. You can adjust parameters with the `DBMS_ROLLING.SET_PARAMETER` procedure. All parameters for the rolling upgrade are visible with the `DBA_ROLLING_PARAMETERS` view.

When you finalize the parameters, use the `DBMS_ROLLING.BUILD_PLAN` procedure to generate the actual plan and perform validation against the plan. If the validation identifies any errors with the plan, the `DBMS_ROLLING_EVENTS` view displays the errors that need attention.



## Specification Stage Examples

- Initialize the upgrade parameters:

```
SQL> exec DBMS_ROLLING.INIT_PLAN(future_primary=>'london');
```

- View the current upgrade parameter values:

```
SQL> select scope, name, curval from dba_rolling_parameters order by
 scope, name;
SCOPE NAME CURVAL

boston INVOLVEMENT FULL
 SWITCH_LGM_LAG_WAIT 60
...
```

- Configuring the plan to wait for the apply lag to fall below 60 seconds before switching over to the future primary:

```
SQL> exec DBMS_ROLLING.SET_PARAMETER('SWITCH_LGM_LAG_WAIT','1');
SQL> exec DBMS_ROLLING.SET_PARAMETER('SWITCH_LGM_LAG_TIME','60');
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Plan parameters must be initialized to system-generated default values before they can be customized. To initialize plan parameters, call the `DBMS_ROLLING.INIT_PLAN` procedure. This procedure identifies the `DB_UNIQUE_NAME` of the future primary database (that is, the Leading Group Master or LGM). The `INIT_PLAN` procedure returns an initial set of system-generated plan parameters. Once the database-related parameters have been defined, the `INIT_PLAN` procedure defines operational parameters with system-supplied defaults. In most cases, the plan parameters will be ready for plan validation; however, to ensure that they meet your needs, you should review each parameter. Plan parameters are persisted in the database until you call the `DESTROY_PLAN` procedure to remove all states related to the rolling upgrade.

You can query the `DBA_ROLLING_PARAMETERS` view to see the plan parameters and their current values. Plan parameters are either global or local in scope. Global parameters are attributes of the rolling upgrade as a whole and are independent of the database participants. Global parameters have a `NULL` value in the `SCOPE` column. Local parameters are associated with a specific database name in the `SCOPE` column.

To modify any existing rolling upgrade parameter, use the `DBMS_ROLLING.SET_PARAMETER` PL/SQL procedure.

## Compilation Stage Examples

- Build the upgrade plan:

```
SQL> exec DBMS_ROLLING.BUILD_PLAN;
```

- View the current upgrade plan:

```
SQL> select instid, target, phase, description from dba_rolling_plan;
```

| INSTID | TARGET | PHASE | DESCRIPTION                                           |
|--------|--------|-------|-------------------------------------------------------|
| 1      | boston | START | Verify database is a primary                          |
| 2      | boston | START | Verify MAXIMUM PROTECTION is disabled                 |
| 3      | london | START | Verify database is a physical standby                 |
| 4      | london | START | Verify physical standby is mounted                    |
| 5      | boston | START | Verify server parameter file exists and is modifiable |
| ...    |        |       |                                                       |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

After all the necessary parameters are specified, you build an upgrade plan. An upgrade plan is a custom-generated set of instructions that guides your Data Guard configuration through a rolling upgrade. To build an upgrade plan, use the `DBA_ROLLING.BUILD_PLAN` PL/SQL procedure. This procedure requires the configuration to be exactly as described by the plan parameters with all of the instances started and reachable through the network. There are no arguments to specify, because the procedure gets all its input from the `DBA_ROLLING_PARAMETERS` view. The procedure validates plan parameters and performs site-specific validations of resources such as log transport and flash recovery area settings. In general, configuration settings that do not meet best-practice criteria generate a warning message.

After the `BUILD_PLAN` procedure successfully returns, the complete upgrade plan is viewable in the `DBA_ROLLING_PLAN` view. Each record in the view identifies a specific instruction that is scheduled for execution and recorded in the `DBA_ROLLING_EVENTS` view.

## Database Rolling Upgrade: Execution Stage

1. Call `DBMS_ROLLING.START_PLAN` to configure the primary and standby databases participating in the upgrade.
2. Upgrade the RDBMS software for leading group databases.
3. Call `DBMS_ROLLING.SWITCHOVER` to swap roles between the current primary database and the new primary database. Switchover is the only required down time.
4. Restart the former primary and any bystander standby databases by using new binaries.
5. Call `DBMS_ROLLING.FINISH_PLAN` to complete the upgrade of the former primary and any bystanders and resynchronize with the new primary.

No arguments are needed on these three procedures.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Execution stage of the rolling upgrade process involves the following five steps:

1. Call the `DBMS_ROLLING.START_PLAN` procedure to configure the future primary and physical standbys designated to protect the future primary.
2. Manually upgrade the Oracle Database software at the future primary database and standbys that protect it.
3. Call the `DBMS_ROLLING.SWITCHOVER` procedure to switch roles between the current primary database and future primary database.
4. Manually restart the former primary and remaining standby databases on the higher version of the Oracle Database software.
5. Call the `DBMS_ROLLING.FINISH_PLAN` procedure to convert the former primary to a physical standby and to configure the remaining standby databases for recovery of the upgrade redo.

No arguments are required for the `START_PLAN`, `SWITCHOVER`, and `FINISH_PLAN` procedures.

## Quiz

You can perform a rolling upgrade using a logical standby SQL apply technique with zero down time.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: b**

## Quiz

The Data Guard broker can be enabled and running throughout the upgrade procedures.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: b**

## Summary

In this lesson, you should have learned how to patch and upgrade databases in your Data Guard configuration:

- By using Standby-First Patch Apply methods
- By using traditional upgrade methods
- By performing rolling upgrades with `DBMS_ROLLING`

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# 18

## Optimizing a Data Guard Configuration

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# Objectives

After completing this lesson, you should be able to:

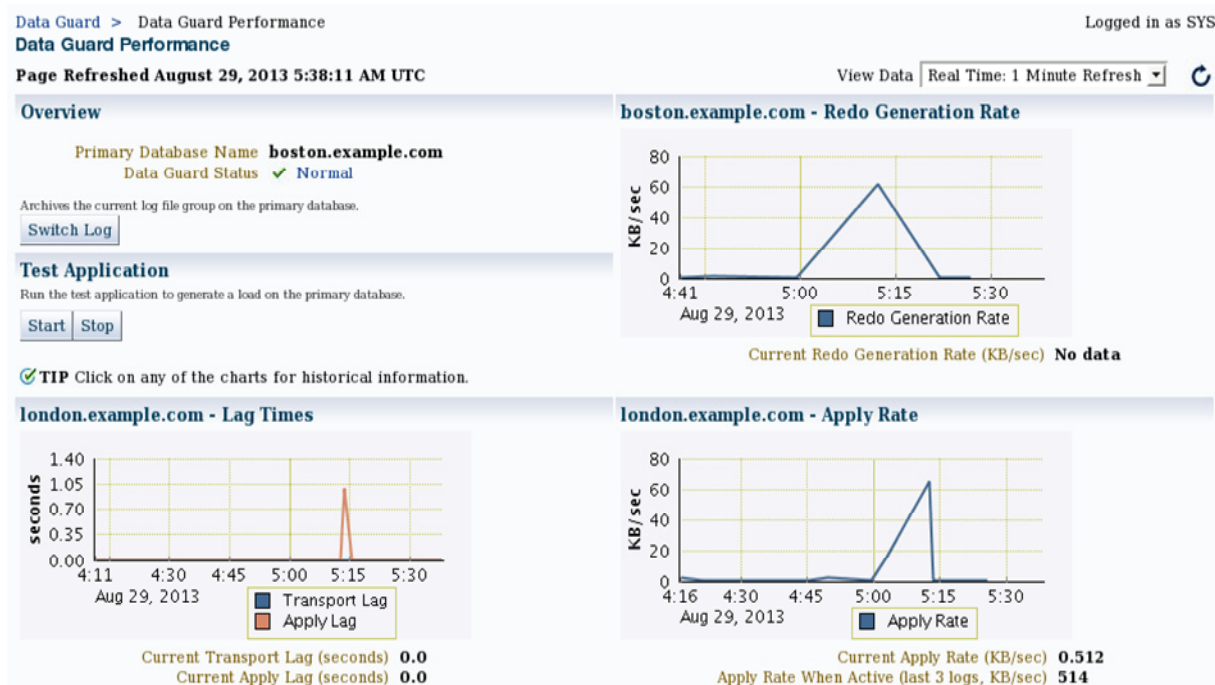
- Monitor configuration performance
- Optimize redo transport for best performance
- Optimize SQL Apply

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



# Monitoring Configuration Performance by Using Enterprise Manager Cloud Control



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Graphical charts on the Performance Overview page:

- **Redo Generation Rate:** Shows the redo generation rate (in KB per second) on the primary database.
- **Apply Rate:** Shows the apply rate (in KB per second) on the standby database. The “Apply Rate When Active” statistic indicates the actual apply rate averaged over the last three log files.
- **Lag Time:** Shows the transport lag and apply lag. Transport lag is the approximate amount of redo (in seconds) that is not yet available on the standby database. Apply lag is the approximate number of seconds by which the standby database is behind the primary database.

On the Performance Overview page, you can invoke a test application to generate a workload on the primary database. This provides a way to view performance metrics when the primary database is operating under a load.

## Optimizing Redo Transport Services

Optimize redo transport with the following techniques:

- Optimizing asynchronous redo transmission by using multiple archiver processes
- Compressing redo data

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Information about these techniques is provided in the following slides.

## Setting the ReopenSecs Database Property

- This property specifies the minimum number of seconds before the archiver process tries to access a previously failed destination.
- Broker default: 300
- Set for the standby database or Far Sync instance.
- The setting is propagated to the REOPEN attribute of the LOG\_ARCHIVE\_DEST\_ *n* initialization parameter of the sending instance.

```
DGMGRL> EDIT DATABASE 'london' SET PROPERTY
'ReopenSecs'=600;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When you specify a value for the ReopenSecs database property, it is propagated to the REOPEN attribute of the LOG\_ARCHIVE\_DEST\_ *n* initialization parameter of the sending instance. The sending instance can be a primary database instance or a Far Sync instance. The REOPEN attribute of the LOG\_ARCHIVE\_DEST\_ *n* parameter specifies the minimum number of seconds before the process that is shipping the redo should try again to access a previously failed destination.

REOPEN applies to all errors and not only connection failures. These errors include (but are not limited to) network failures, disk errors, and quota exceptions.

## Setting the NetTimeout Database Property

- This property specifies the number of seconds that the log writer process (LGWR) waits for Oracle Net Services to respond to a request.
- Broker default: 30
- The setting is propagated to the `NET_TIMEOUT` attribute of the `LOG_ARCHIVE_DEST_n` initialization parameter of the sending instance.

```
DGMGRL> EDIT DATABASE 'london' SET PROPERTY
'NetTimeout'=20;
```

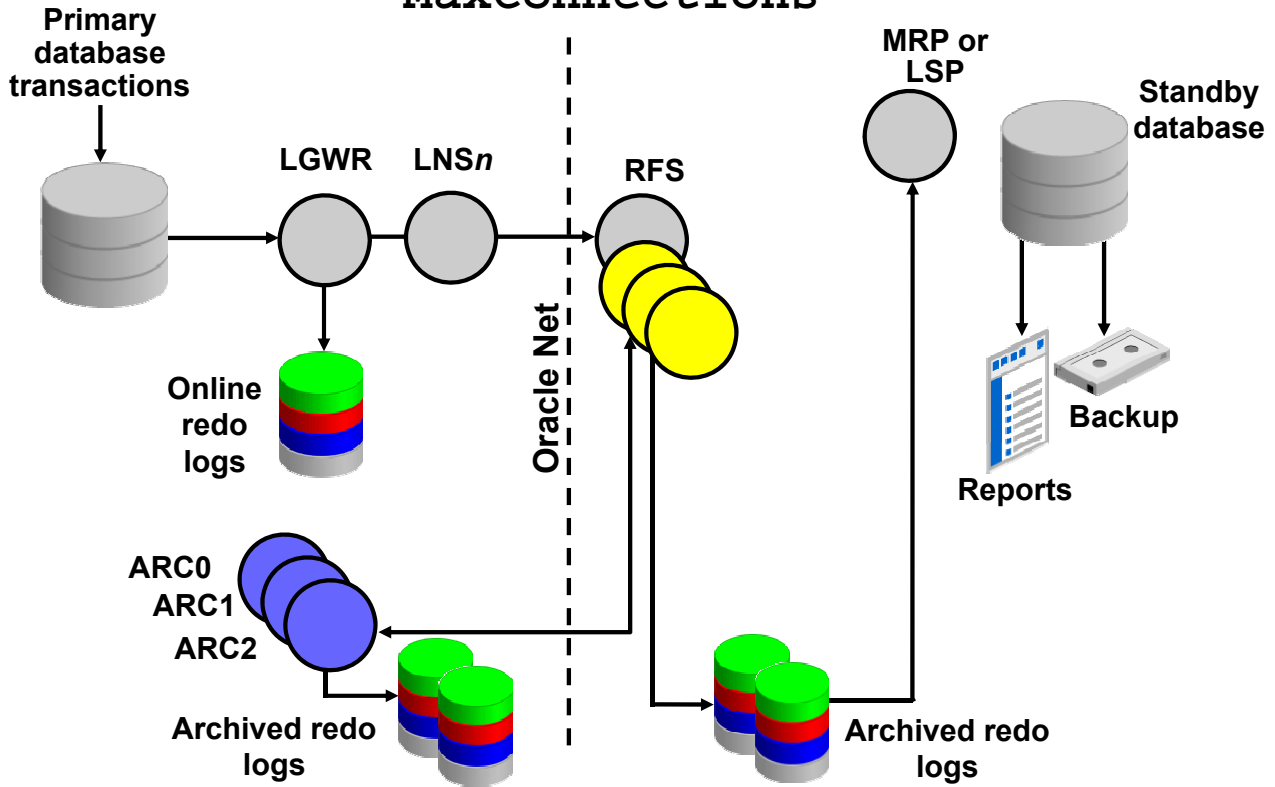
The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When you specify a value for the `NetTimeout` database property, it is propagated to the `NET_TIMEOUT` attribute of the `LOG_ARCHIVE_DEST_n` initialization parameter for your primary database or Far Sync. The `NET_TIMEOUT` attribute enables you to bypass the default network timeout interval that was established for the system on which the primary database resides. Without the `NET_TIMEOUT` attribute, the primary database can potentially stall for the default network timeout period. By specifying a smaller, nonzero value for `NET_TIMEOUT`, you can enable the primary database to mark a destination as “failed” after the user-specified timeout interval expires.

**Note:** Remember to specify a reasonable value when running in maximum protection mode. False network failure detection may cause the primary instance to shut down if there are no other standby databases in the correct mode that can be contacted by the primary database instance.

# Optimizing Redo Transmission by Setting MaxConnections



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The redo transport mechanism uses all available bandwidth by allowing a single large redo log file to be transferred in parallel by multiple archiver processes. This behavior is controlled by the `MaxConnections` database property.

This architecture increases the redo transfer rate and enables faster redo transmission to standby databases for bulk batch updates on the primary database. As a result of the improvement in transfer rates, there is an increased availability of data at the standby database site.

## Setting the MaxConnections Database Property

- This property specifies the number of archiver processes that are used to transmit redo data from a single archived redo log on the primary database to the archived redo log at the remote site for gap resolution.
- Broker default: 1
- The setting is propagated to the MAX\_CONNECTIONS attribute of the LOG\_ARCHIVE\_DEST\_ *n* initialization parameter of the primary database or Far Sync instance.

```
DGMGRL> EDIT DATABASE 'london' SET PROPERTY
'MaxConnections'= 15;
```

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When you specify a value for the MaxConnections database property, it is propagated to the MAX\_CONNECTIONS attribute of the LOG\_ARCHIVE\_DEST\_ *n* initialization parameter for your primary database or Far Sync instance. The MAX\_CONNECTIONS attribute of LOG\_ARCHIVE\_DEST\_ *n* is used to set the number of parallel connections that are used for transmitting archived redo log files to a remote destination. The MAX\_CONNECTIONS attribute defaults to 1, indicating that a single connection is established for the communication and transfer of data. The maximum value for MAX\_CONNECTIONS is 20.

**Note:** You must set the LOG\_ARCHIVE\_MAX\_PROCESSES initialization parameter to be greater than or equal to the value of MAX\_CONNECTIONS to achieve the desired number of parallel connections. If the value of the MAX\_CONNECTIONS attribute exceeds the value of LOG\_ARCHIVE\_MAX\_PROCESSES, Data Guard uses the available archiver processes.

## Compressing Redo Data by Setting the RedoCompression Property

- This can be enabled for all redo transport methods (ASYNCR and SYNC).
- The setting is propagated to the COMPRESSION attribute of the LOG\_ARCHIVE\_DEST\_n initialization parameter.
- Determine whether redo compression is enabled by querying the COMPRESSION column of the V\$ARCHIVE\_DEST view.
- Enable by using DGMGRL:

```
DGMGRL> EDIT DATABASE 'london' SET PROPERTY
 'RedoCompression'='ENABLE';
```

- Enable by using SQL:

```
SQL> ALTER SYSTEM SET LOG_ARCHIVE_DEST_3 =
 'SERVICE=london SYNC COMPRESSION=ENABLE';
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When the communication network to remote databases is a high-latency, low-bandwidth WAN link and the redo data that is transferred to standby databases is substantial, you want to make the most effective use of network bandwidth. Redo transport compression can be enabled on any remote destination for all redo transport methods to reduce network bandwidth usage.

Redo compression can be enabled or disabled by setting the Oracle Data Guard broker's RedoCompression property. The setting is propagated to the COMPRESSION attribute of the LOG\_ARCHIVE\_DEST\_n initialization parameter. Redo compression is disabled by default.

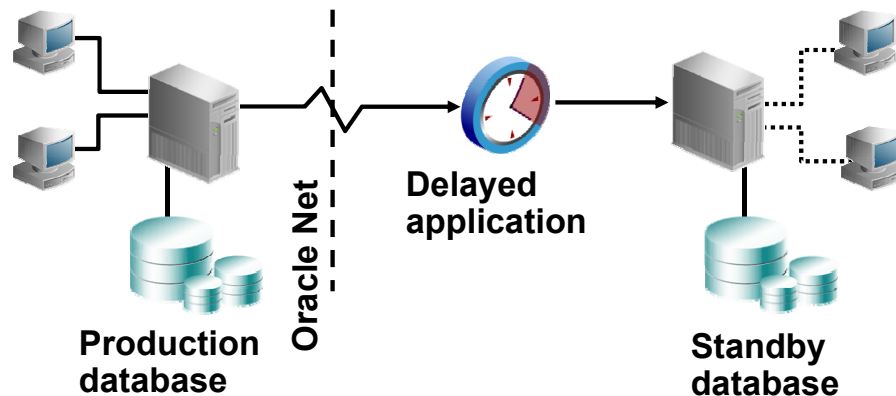
When you add a database to the Data Guard configuration, the Data Guard broker automatically detects whether network compression is enabled or disabled for the standby database being added. The property is set accordingly.

**Note:** Use of this feature requires the Oracle Advanced Compression option.

## Delaying the Application of Redo

Delaying the application of redo helps safeguard against:

- Data corruption
- User errors



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can delay the application of changes to standby databases, thereby providing protection from user errors and corruptions. You can protect against the application of corrupted or erroneous data to the standby database. The apply process also revalidates the log records to prevent application of log corruptions.

For example, if a critical table is accidentally dropped from the primary database, you can prevent this action from affecting the standby database by delaying the application of the change in the standby database.

When operating in maximum protection or maximum availability mode, Data Guard ensures zero data loss even with the delayed apply in effect.

If you define a delay for a destination that has real-time apply enabled, the delay is ignored.

**Note:** You can use Flashback Database as an alternative to the Apply Delay configuration option. Using Flashback Database is an Oracle best practice. See the lesson titled “Using Flashback Database in a Data Guard Configuration” for additional information.



## Setting the DelayMins Database Property to Delay the Application of Redo

- This property specifies the number of minutes that log apply services must wait before applying redo data to the standby database.
- Broker default: 0 (meaning that apply services applies redo data as soon as possible)
- The setting is propagated to the DELAY attribute of the LOG\_ARCHIVE\_DEST\_ *n* initialization parameter of the primary database or Far Sync.

```
DGMGRL> EDIT DATABASE 'london' SET PROPERTY
 'DelayMins'=5;
```

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Use the DelayMins configurable database property to specify the number of minutes that log apply services must wait before applying redo data to the standby database. This setting is propagated to the DELAY attribute of the LOG\_ARCHIVE\_DEST\_ *n* initialization parameter.

# Using Enterprise Manager to Delay the Application of Redo

london2.example.com

Oracle Database ▾ Performance ▾ Availability ▾ Schema ▾ Administration ▾

Data Guard > Edit Standby Database Properties: london2.example.com  
Edit Standby Database Properties: london2.example.com

General Standby Role Properties Common Properties

**TIP** The database is currently in the standby role. Any modifications take effect immediately.

**Redo Transport Mode** ASYNC  
Method used to transport redo to this standby database.

**Redo Compression** DISABLE  
Specifies whether redo data is transmitted in compressed form.

**Net Timeout** 30  
Amount of time the primary database will wait for a response from the standby database over the network connection.

**Apply Delay (minutes)** 0  
Number of minutes to delay applying archived redo data to the standby database.

**Standby Archive Location** USE\_DB\_RECOVERY\_FILE\_DEST  
Location on the standby host for archived redo log files.

**Log Shipping** ON  
Disables log shipping to this standby database.

[Hide Advanced Properties](#)

**Specify the delay in minutes.**

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

1. On the Data Guard page, select your standby database and click Edit.
2. On the Edit Standby Database Properties page, click Standby Role Properties.
3. In the Apply Delay field, enter the delay value (in minutes).
4. Click Apply.

## Optimizing SQL Apply

- Adjust the number of processes allocated to SQL Apply.
  - APPLIER processes
  - PREPARER processes
- Modify SQL Apply parameters to control the number of processes allocated to SQL Apply.
  - APPLY\_SERVERS: The number of APPLIER processes that are used to apply changes
  - MAX\_SERVERS: The number of processes that SQL Apply uses to read and apply redo
  - PREPARE\_SERVERS: The number of PREPARER processes that are used to prepare changes
- Set SQL Apply parameters by using the DBMS\_LOGSTDBY package.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The MAX\_SERVERS, APPLY\_SERVERS, and PREPARE\_SERVERS parameters can be modified to control the number of processes that are allocated to SQL Apply. Because SQL Apply allocates one process for the READER, BUILDER, and ANALYZER roles, the following relationship between the three parameters is required:

$\text{APPLY\_SERVERS} + \text{PREPARE\_SERVERS} = \text{MAX\_SERVERS} - 3$

Use the DBMS\_LOGSTDBY.APPLY\_SET procedure to change the APPLY\_SERVERS, MAX\_SERVERS, and PREPARE\_SERVERS parameters.

Query DBA\_LOGSTDBY\_PARAMETERS to view the SQL Apply parameter settings.

**Note:** See the *Oracle Database PL/SQL Packages and Types Reference* for detailed information about the DBMS\_LOGSTDBY.APPLY\_SET procedure. Also consult the "SQL Apply Best Practices: Oracle Data Guard 11g Release 1" whitepaper found at:

<http://www.oracle.com/technetwork/database/features/availability/maa-wp-11gr1-sqlapplybestpractices-131426.pdf>

At the time of developing this course, the above whitepaper has not been rewritten for Oracle Database 12c, and therefore is still relevant.

## Adjusting the Number of APPLIER Processes

- Determine whether adjusting the number of APPLIER processes achieves greater throughput.
  - Determine whether APPLIER processes are busy:

```
SQL> SELECT COUNT(*) AS IDLE_APPLIER
2 FROM V$LOGSTDBY_PROCESS
3 WHERE TYPE = 'APPLIER' and status_code = 16166;
```

- Determine whether there is enough work available for additional APPLIER processes:

```
SQL> SELECT NAME, VALUE FROM V$LOGSTDBY_STATS
2 WHERE NAME LIKE 'transactions%';
```

- Adjust the MAX\_SERVERS parameter (if necessary).
- Adjust the APPLY\_SERVERS parameter to increase the number of APPLIER processes.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Before changing the number of APPLIER processes, perform the following steps to determine whether adjusting the number of APPLIER processes will help you achieve greater throughput:

1. Determine whether APPLIER processes are busy by issuing the following query:

```
SELECT COUNT(*) AS IDLE_APPLIER FROM V$LOGSTDBY_PROCESS
WHERE TYPE = 'APPLIER' and status_code = 16166;
```

2. After ensuring that there are no idle APPLIER processes, determine whether there is enough work available for additional APPLIER processes by issuing the following query:

```
SELECT NAME, VALUE FROM V$LOGSTDBY_STATS
WHERE NAME LIKE 'transactions%';
```

The second query returns two statistics that provide a cumulative total: the number of transactions that are ready to be applied by the APPLIER processes and the number of transactions that have already been applied. If the difference between “transactions mined” and “transactions applied” is higher than twice the number of available APPLIER processes, an improvement in throughput is possible if you increase the number of APPLIER processes.

Before you increase the number of APPLIER processes, consider the following requirement:

$\text{APPLY\_SERVERS} + \text{PREPARE\_SERVERS} = \text{MAX\_SERVERS} - 3$

## Adjusting the Number of PREPARER Processes

- Determine whether adjusting the number of PREPARER processes is required.
  - Determine whether all PREPARER processes are busy:

```
SQL> SELECT COUNT(*) AS IDLE_PREPARER
2 FROM V$LOGSTDBY_PROCESS
3 WHERE TYPE = 'PREPARER' and status_code = 16166;
```

- Determine whether the number of transactions ready to be applied is less than the number of available APPLIER processes:

```
SQL> SELECT NAME, VALUE FROM V$LOGSTDBY_STATS
2 WHERE NAME LIKE 'transactions%';
SQL> SELECT COUNT(*) AS APPLIER_COUNT
2 FROM V$LOGSTDBY_PROCESS WHERE TYPE = 'APPLIER';
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

On rare occasions, you may need to adjust the number of PREPARER processes. Before increasing the number of PREPARER processes, verify that the following conditions are true:

- All PREPARER processes are busy.
- The number of transactions ready to be applied is less than the number of available APPLIER processes.
- There are idle APPLIER processes.

Perform the following queries to verify the preceding conditions:

1. Determine whether all PREPARER processes are busy:

```
SELECT COUNT(*) AS IDLE_PREPARER
FROM V$LOGSTDBY_PROCESS
WHERE TYPE = 'PREPARER' and status_code = 16166;
```

2. Determine whether the number of transactions ready to be applied is less than the number of APPLIER processes:

```
SELECT NAME, VALUE FROM V$LOGSTDBY_STATS
WHERE NAME LIKE 'transactions%';
SELECT COUNT(*) AS APPLIER_COUNT
FROM V$LOGSTDBY_PROCESS WHERE TYPE = 'APPLIER';
```

## Adjusting the Number of PREPARER Processes

- Determine whether there are idle APPLIER processes:

```
SQL> SELECT COUNT(*) AS IDLE_APPLIER
2 FROM V$LOGSTDBY_PROCESS
3 WHERE TYPE = 'APPLIER' and status_code = 16166;
```

- Adjust the MAX\_SERVERS parameter if necessary.
- Adjust the PREPARE\_SERVERS parameter to increase the number of PREPARER processes.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

- Determine whether there are idle APPLIER processes:

```
SELECT COUNT(*) AS IDLE_APPLIER
FROM V$LOGSTDBY_PROCESS
WHERE TYPE = 'APPLIER' and status_code = 16166;
```

Before you increase the number of PREPARER processes, consider the requirement:

$APPLY\_SERVERS + PREPARE\_SERVERS = MAX\_SERVERS - 3$

You may need to increase the value of MAX\_SERVERS before increasing the value of PREPARE\_SERVERS.

Use the DBMS\_LOGSTDBY.APPLY\_SET procedure to increase the values of MAX\_SERVERS and PREPARE\_SERVERS, as shown in the following example:

```
SQL> EXECUTE DBMS_LOGSTDBY.APPLY_SET('MAX_SERVERS', 26);
SQL> EXECUTE DBMS_LOGSTDBY.APPLY_SET('PREPARE_SERVERS', 3);
```

## Quiz

Enabling Data Guard network redo compression requires the Oracle database Advanced Compression option.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: a**

## Quiz

The Data Guard MAX\_SERVERS, APPLY\_SERVERS, and PREPARE\_SERVERS parameters can be adjusted with the database initialization parameter file.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: b**



## Summary

In this lesson, you should have learned how to:

- Monitor configuration performance
- Optimize redo transport for best performance
- Optimize SQL Apply

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Practice 18: Overview

This practice covers configuring network compression of redo data.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.